PROGRAMMING IN JAVA

ASSIGNMENT 5 – CO5

DATA PERSISTENCE ENGINEERING – FILES AND DATABASES

Course Outcome – CO5: Java I/O – byte, character and buffered streams, the File class, serialization; JDBC – architecture and drivers, Statement, PreparedStatement, CallableStatement, transactions, batch processing and metadata (Sessions 39–48) **Nature:** Individual
Duration: One Week
Total Marks: 50

Bloom's Levels Assessed: K4 – Analyse and K5 – Evaluate (the level of each task is statedunder the task heading)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, includingall "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment5** andshareit with the course faculty (public, or private with the faculty added as a collaborator). • Keep one folder for each task (for example: Task01, Task02 …or Q1, Q2 …), containingthe Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps tocompile and run each program, and a short note of the result of each task. • Commit your work progressively – at least one commit for each task. The commit historywill be checked; a single upload of all files on the last day will not be accepted. • Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____ **How to Answer the Additional Questions**

Complete the Code: Copy the given code into your repository, replace every TODOwithworking code without changing the rest of the program, execute it and compare your output withthe expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would changeif thealternative concept were used, with code or pseudocode, and analyse and compare thetimecomplexity and space complexity of both solutions using Big-O notation, supported by your ownmeasurements wherever possible.

Assessment Components

PART A – Persistence Design & Pseudocode: 25 Marks
PART B – Implementation & Experimental Investigation: 25 Marks

PURPOSE OF THE ASSIGNMENT

Almost every real application must store data so that it survives after the programends. Javaoffers several ways to do this – text files, binary streams, object serialization and relational databases through JDBC.

This assignment investigates a fundamental question:

> **How should a Java engineer decide where and how data is stored, and howcan the program be made correct, safe and efficient as the data grows?**

You are expected to demonstrate the following progression:

> **Model → Choose a Store → Design → Protect → Implement →Measure →Compare → Decide**

Simply obtaining a program that saves and loads data is not sufficient. Your design reasoning, experimental evidence and interpretation are all part of the assessment.

PART A – PERSISTENCE DESIGN & PSEUDOCODE 25MarksAll pseudocode must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode.

TASK A1 – DESIGN A FILE FORMAT 7MarksBloom's Level: K4 – Analyse

Transport Office Bus-Pass Records

The university transport office issues bus passes. Each pass record contains: pass ID, student register number, name, route number, boarding stop, validity date and amount paid. About 8,000passes are issued each year. The office wants the records stored in a file and needs to (i) addanew pass, (ii) find a pass by pass ID, and (iii) list all passes on a given route.

A1.1 – Represent the Record | 2 Marks

Design the record layout for (a) a delimited text file and (b) a binary file writtenwithDataOutputStream. State the field order, delimiter or field sizes, and how a name containingthedelimiter is handled.

A1.2 – Design the Operations | 3 Marks

Write pseudocode for the three operations for the text-file design, stating which streamclasses(FileReader/FileWriter, BufferedReader/BufferedWriter, FileInputStream/FileOutputStream) youwould use and why.

A1.3 – Design for Damaged Data | 2 Marks

Explain how your design detects and handles a damaged line, a missing file and an incompletewrite caused by a power failure.

Alternative Concept and Complexity Analysis

If the passes were stored in a RandomAccessFile with fixed-length records, so that recordi isfound by seeking to i × recordSize, instead of a delimited text file, what would the solutionbe?Analyse the time to find a pass by ID (O(n) scan versus O(1) seek) and the space wastedbypadding fixed-length fields.

Evidence Expected

Two record layouts, pseudocode for three operations with justified stream choices, and adesignfor damaged data.

TASK A2 – DON'T READ WHAT YOU DON'T NEED 5MarksBloom's Level: K4 – Analyse

A program reads a 50 MB attendance log one byte at a time using FileInputStream.read() tocount the number of lines.

A2.1 – Predict the Cost | 2 Marks

Predict, with reasons, how many read calls the program makes and why it is slow. **A2.2 – Redesign Using Buffering | 2 Marks**

Redesign the program using a buffered stream or a byte array buffer, and explain howbufferingchanges the number of calls to the operating system.

A2.3 – When Buffering Does Not Help | 1 Mark

Describe one situation in which adding a buffer gives little or no benefit. **Alternative Concept and Complexity Analysis**

If the whole file were read at once using Files.readAllLines() instead of a buffered stream, what would the solution be? Analyse the time complexity and the memory used (O(file size) versusO(buffer size)) for a 50 MB file, and state when each approach is appropriate.

Evidence Expected

A reasoned prediction, a redesigned read strategy and a justified limitation.

TASK A3 – DISCOVER THE DATA-ACCESS DESIGN PATTERN 5MarksBloom's Level: K4 – Analyse

A student project has three classes – StudentDB, CourseDB and MarksDB. Each one opensaconnection, builds an SQL string, executes it, reads the ResultSet into objects and closestheconnection. Large parts of the three classes are almost identical.

A3.1 – Identify the Common Pattern | 2 Marks

Identify the steps that are repeated in all three classes and the parts that differ. **A3.2 – Design a Generic Data-Access**

Interface | 2 Marks

Design a generic interface such as Dao<T, K> with methods for insert, findById, findAll, updateand delete, and show how StudentDao implements it.

A3.3 – Map Objects to Tables | 1 Mark

Show how the fields of the Student class map to the columns of the student table, includingtheprimary key.

Alternative Concept and Complexity Analysis

If the SQL were written directly inside the service or GUI classes instead of a generic Dao<T, K>, what would the solution be? Compare the number of places that must change when atablecolumn is renamed, and analyse whether the time and space cost per database operationdiffersbetween the two designs.

Evidence Expected

An identified common pattern, a generic data-access design and an object–table mapping.

TASK A4 – WHAT SHOULD BE SERIALIZED? 4MarksBloom's Level: K4 – Analyse

A hostel management program saves its in-memory state by serializing a Hostel object, whichholds a list of Room objects, each holding a list of Resident objects. Each Resident has aname, register number, password and a reference to a live database Connection.

A4.1 – Decide What to Persist | 2 Marks

Decide which fields must be marked transient and why, and explain what happens to theobject graph when the Hostel object is serialized.

A4.2 – Plan for Change | 2 Marks

A new field (phone number) is added to Resident after files have already been saved. Explainwhat happens when an old file is read, and how serialVersionUID affects the outcome.

Alternative Concept and Complexity Analysis

If the hostel data were saved as a CSV or JSON text file instead of Java serialization, what wouldthe solution be? Compare the file size, the time to save and load n residents, and howeachapproach copes when a new field is added.

Evidence Expected

A justified transient design and a correct explanation of object-graph serialization and versioning.

TASK A5 – DESIGN FOR CORRECTNESS AND SAFETY 4MarksBloom's Level: K4 –

Analyse

A5.1 – Transaction Design | 2 Marks

A student transfers ₹500 from a hostel wallet to a canteen wallet. Write pseudocode usingJDBCcommit and rollback so that money is never lost or created if the program fails betweenthetwoupdates.

A5.2 – Injection Analysis | 2 Marks

The login check is written as:

```
String sql = "SELECT * FROM users WHERE id='" + id + "' AND pwd='" + pwd + "'";
```

Explain how a crafted input can make the query succeed without a valid password, and redesignthe check using PreparedStatement.

Alternative Concept and Complexity Analysis

If the two wallet updates were executed as two separate auto-commit statements insteadof onetransaction, what would the outcome be when the program fails between them? Analyse thetimecost of a transaction versus two independent updates, and the risk to data correctness.

Evidence Expected

Correct transaction pseudocode and a reasoned injection analysis with a safe redesign.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION25Marks

Part B assesses whether your design reasoning survives contact with actual computation. Youare expected to:

Implement → Instrument → Stress → Measure → Compare →Interpret

Execution screenshots alone are insufficient.

TASK B1 – IMPLEMENT AND INSTRUMENT THE FILE STORE 7MarksBloom's Level: K4 – Analyse

Implement the bus-pass file store designed in Task A1 (text format) and the buffered linecounter from Task A2.

B1.1 – Validate the Implementation | 2 Marks

Test add, find and list using at least one normal case, one pass ID that does not exist andonedamaged line. Explain what each test verifies.

B1.2 – Compare with Your Design | 1 Mark

Compare your implementation with your Part A pseudocode and explain any change youhadtomake.

B1.3 – Measure the Effect of Buffering | 2 Marks

Measure the time to count lines in files of 1 MB, 10 MB and 50 MB with the unbufferedandbuffered versions. Compare the results with your prediction in A2.1.

B1.4 – Measure Growth | 2 Marks

Measure the time to find a pass as the file grows from 1,000 to 100,000 records, and explainthetrend.

Complete the Code

Complete TODO 1 below and execute the program.

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.Arrays;

public class PassStore {
    private final Path file;

    PassStore(Path file) { this.file = file; }

    public void add(String passId, String regNo, String name, int route, double amount)throws
            IOException {
        try (BufferedWriter w = Files.newBufferedWriter(file, StandardOpenOption.CREATE,
            StandardOpenOption.APPEND)) { w.write(String.join(",", passId, regNo, name,
                                    String.valueOf(route), String.valueOf(amount)));
            w.newLine();
        }
    }

    /** Returns the five fields of the pass with the given ID, or null if not found.public
    String[] find(String passId) throws IOException { // TODO 1: Read the file line by line
    with a BufferedReader inside try-with-// resources; skip damaged lines (not exactly 5
    fields); return the fields// the matching pass


        return null;
    }


        public static void main(String[] args) throws IOException { Path f =
                        Files.createTempFile("passes", ".csv");
        PassStore store = new PassStore(f);
        store.add("P1001", "192311001", "Arun", 12, 4500);
        Files.writeString(f, "P1002,DAMAGED_LINE\n",
        StandardOpenOption.APPEND);store.add("P1003", "192311003", "Divya",
        7, 3800);
        System.out.println(Arrays.toString(store.find("P1003")));
        System.out.println(Arrays.toString(store.find("P1002"))); Files.delete(f);
    }
}
```

Expected output:

```
[P1003, 192311003, Divya, 7, 3800.0]
null
```

Explain why the damaged line for P1002 does not stop the search. **Alternative Concept and Complexity Analysis**

Your find() scans the file every time. If the file were read once at start-up into a HashMap<String, String[]> keyed by pass ID, what would the solution be? Analyse the time for q searchesonnpasses (O(q × n) versus O(n + q)) and the memory used in both approaches.

Evidence Expected

Working code, test table, measured buffering and growth evidence, and interpretation.

TASK B2 – ENGINEER AN ADVERSARIAL INPUT 5MarksBloom's Level: K4 – Analyse

Using a **local test database created only for this assignment**, implement the unsafelogincheck from Task A5.2 and the PreparedStatement version.

- Construct an input that makes the unsafe version accept an invalid login, and showthat the PreparedStatement version rejects it
- Construct a damaged bus-pass file (wrong number of fields, non-numeric amount, emptyline) and show how your file store responds
- Explain what each adversarial input reveals about the original design

Alternative Concept and Complexity Analysis

If, instead of PreparedStatement, the program escaped quote characters in the input or validatedit with a regular expression before building the SQL string, what would the solution be?Explainwhy this is weaker, and analyse the time cost of validation per query comparedwithaPreparedStatement.

Evidence Expected

Deliberately constructed adversarial inputs, observed behaviour of the unsafe and safe versions, and a reasoned explanation.

TASK B3 – STATEMENT vs PREPAREDSTATEMENT vs BATCH 8MarksBloom's Level: K4 – Analyse

Move the bus-pass records into a MySQL table using JDBC.

B3.1 – Correctness on Small Data | 2 Marks

Insert 10 records using each approach and verify, using a SELECT and ResultSetMetaData, that the table contents and column types are correct.

B3.2 – Increase the Data Size | 2 Marks

Insert 1,000, 10,000 and 100,000 records using (a) Statement, (b) PreparedStatement and(c) PreparedStatement with addBatch()/executeBatch(). Record the time for each.

B3.3 – Auto-commit vs Transaction | 2 Marks

Repeat the batch insert with auto-commit on and with a single transaction. Then force afailurehalf-way and show the table contents in both cases.

B3.4 – Engineering Trade-Off | 2 Marks

Explain the differences you observed, and state which approach you would use for (a) anightlyimport of 100,000 passes and (b) a single pass issued at the counter.

Complete the Code

Complete TODO 1 to TODO 3 below, create the bus_pass table in your test database, andcall loadAll() with 1,000, 10,000 and 100,000 generated passes.

```
import java.sql.Connection;
import java.sql.PreparedStatement;                    "
import java.sql.SQLException;
import java.util.List;

public class PassLoader {
    /** Loads all passes in one transaction, in batches of 1,000 rows. * Returns the
     number of rows loaded. */
    public static int loadAll(Connection con, List<String[]> passes) throws
            SQLException {                                    every
            String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount)+
                                    "VALUES (?, ?, ?, ?, ?)";
        boolean oldAutoCommit = con.getAutoCommit();
        con.setAutoCommit(false);
        try (PreparedStatement ps = con.prepareStatement(sql)) { int n = 0;
            for (String[] p : passes) {
                // TODO 1: Set the five parameters from p (route as int, amount as// double),
                add the row to the batch, and execute the batch after// 1,000 rows


            }
            ps.executeBatch();
            // TODO 2: Make all inserted rows permanent and return the number of rows// loaded

        } catch (SQLException | RuntimeException e) {
            // TODO 3: Undo every insert made in this transaction, then re-throw the// exception

        } finally {
            con.setAutoCommit(oldAutoCommit);
        }
    }
}
```

Insert a pass with an invalid amount half-way through the list and show that no rows remaininthe table after the failure.

Alternative Concept and Complexity Analysis

If, instead of PreparedStatement batches, the rows were inserted using one multi-rowINSERTstatement or the MySQL LOAD DATA command, what would the solution be? Analyse thetotal time for n rows and the memory used for a batch of b rows in each approach, usingyour measurements.

Evidence Expected

Verified correctness, measured comparison at three sizes, transaction evidence and a reasonedtrade-off.

TASK B4 – FINAL ENGINEERING DECISION 5MarksBloom's Level: K5 – Evaluate

The transport office will now serve all 12 campuses of the university, with 100× thecurrent number of passes and several counters issuing passes at the same time.

B4.1 – Implement a Practical Strategy | 2 Marks

Implement a migration program that reads the existing pass file and loads it into the databaseusing batches and a transaction, reporting the records that failed validation.

B4.2 – Inspect the Database | 1 Mark

Use DatabaseMetaData to report the database product, driver version and the tables present, and explain why this is useful when the application is moved to another server.

B4.3 – Production Decision | 2 Marks

Decide whether the office should use text files, serialization or a database for the newscale. Justify your decision using your measurements and the requirements for concurrent counters, searching and safety.

Alternative Concept and Complexity Analysis

If each campus kept its own serialized file instead of sharing one database, what wouldthesolution be for the 12-campus requirement? Analyse the time to search all passes and thespaceused, and explain how concurrent counters would be handled in each design.

Evidence Expected

A working migration with a failure report, metadata output and an evidence-based productiondecision.

MARK DISTRIBUTION

Component	Task	Bloom's Level	Marks
Part A – Design	A1 – File Format Design		Analyse 7
	A2 – Buffered I/O Design K4		Analyse 5
	A3 – Data-Access Pattern		Analyse 5
	A4 – Serialization Design		Analyse 4
	A5 – Transactions & Injection		Analyse 4
	Part A Total		**25**
Part B – Implementation	B1 – Instrumented File Store		Analyse 7
	B2 – Adversarial Inputs		Analyse 5
	B3 – Statement vsPreparedStatement vs Batch		Analyse 8
	B4 – Final Engineering Decision		Evaluate 5
	Part B Total		**25**
	TOTAL		**50**

SUBMISSION REQUIREMENTS

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots andaREADME.md.

For **Part A**, submit the record layouts, pseudocode, diagrams and reasoning. Java codewill not be accepted in place of pseudocode.

For **Part B**, submit Java source code, the SQL script used to create the test database, test inputs, expected behaviour, actual outputs, measurement data and interpretation. A screenshot showingonly a final message is not sufficient evidence.

Students are expected to use the same engineering workflow throughout the assignment:

UNDERSTAND → DESIGN → PROTECT → IMPLEMENT → MEASURE →CHALLENGE → INTERPRET

The key question is not simply “Does the program save the data?” It is also:

How safely, how efficiently and at what scale does it save the data, and what should an engineer change when the scale changes?

<YOUR COLLEGE / DEPARTMENT NAME>

(write your college and department here)

CSA09 – PROGRAMMING IN JAVA

ASSIGNMENT 5 – CO5 REPORT

Persistence: Files, Streams, Serialization and JDBC – Design, Implementation and Experimental Investigation

Item	Details
Student name	<YOUR NAME>
Register number	<REGISTER NUMBER>
Course	CSA09 – Programming in Java
Assignment	Assignment 5 – CO5 (Part A 25 marks + Part B 25 marks = 50 marks)
Java version used	OpenJDK 21 (java -version)
GitHub repository	https://github.com/<your-username>/CSA09-<RegisterNumber>-Assignment5
Submission date	<DATE>

The pages before this cover page are the assignment as issued. The answers follow in the order of the assignment: **Part A** (design and pseudocode: A1–A5) and **Part B** (implementation and experiments: B1–B4). Every answer is placed under the marked sub-question it belongs to, and the **Marks Map** on the next pages shows, for all 50 marks, where each item is answered and what evidence supports it.

Contents

Part / task	Topic	Marks
Marks Map	Rubric alignment: every marked sub-question → section, evidence and figures	–
Part A	Persistence Design & Pseudocode	25
A1	Design a File Format (bus-pass records)	7
A2	Don't Read What You Don't Need (buffered I/O)	5
A3	Discover the Data-Access Design Pattern (DAO)	5
A4	What Should Be Serialized?	4
A5	Design for Correctness and Safety (transactions, injection)	4
Part B	Implementation & Experimental Investigation	25
B1	Implement and Instrument the File Store (+ Complete the Code: PassStore)	7
B2	Engineer an Adversarial Input	5
B3	Statement vs PreparedStatement vs Batch (+ Complete the Code: PassLoader)	8
B4	Final Engineering Decision (12 campuses)	5
Appendix A	Repository layout, how to run (also with MySQL), GitHub steps	–
Appendix B	The test-only JDBC driver and how the evidence was produced	–
Appendix C	Complete list of programs, viva preparation, final checklist	–

How the evidence in this report was produced (please read)

The assignment states that **execution screenshots alone are insufficient**: design reasoning, measurements and interpretation are assessed. Every program in this report was compiled and executed (OpenJDK 21), and every number in the tables is read from a file written by a program (CSV or console output). Two limitations of the preparation machine must be stated openly:

Topic	What was done	Consequence for the reader
Database	The machine has no MySQL server and no Connector/J jar. All JDBC programs use only the standard java.sql interfaces and were run against the real SQLite engine through a small test-only JDBC driver (jdbc:minisqlite:, Appendix B, built on Java's foreign-function API). SQL scripts are written in the common subset that MySQL and SQLite both accept.	Statement / PreparedStatement / batch / commit / rollback / constraints / ResultSetMetaData / DatabaseMetaData behave as in JDBC. Not measured: network round trips, MySQL's rewriteBatchedStatements, LOAD DATA, MySQL-specific escaping (backslash). Each place where this matters is marked in the text. For the final submission: run the same programs with the MySQL driver (only the JDBC URL changes).
Pseudocode standard	The text of "SIMATS Engineering – Standard for Algorithmic Pseudocode" was not available. Pseudocode in Part A uses numbered steps and the keywords ALGORITHM, INPUT, OUTPUT, BEGIN, END, IF/THEN/ELSE/ENDIF, FOR EACH/ENDFOR, WHILE/ENDWHILE, TRY/CATCH, CALL, SET, RETURN.	If the official standard prescribes other keywords, only the keywords need to change. Part A contains pseudocode only, as required; the matching Java is in Part B and the appendix.
Timings	Measured on one Linux machine with a real disk (commits are made durable). Repeats and warm-up are stated for each experiment.	Absolute times will differ on another computer; trends and operation counts do not.

For the final submission: run every program on your own computer (the assignment requires individual execution), re-run the JDBC programs with MySQL, retake the screenshots, replace the placeholders, and commit to GitHub task by task.

Marks Map – Rubric Alignment (50 marks)

The assignment awards marks per sub-question and states the **evidence expected** for each task; it does not publish band descriptors. The table below therefore uses (1) the sub-question marks, (2) the "Evidence Expected" line of the task, and (3) the "How to answer" rules (Complete the Code: execute it and compare with the expected output; Alternative Concept: describe the alternative with code or pseudocode, analyse time and space in Big-O, and support it with your own measurements; Bloom level K4 = analyse, K5 = evaluate). "Where" gives the section of this report.

Task	Sub-question	Marks	What must be shown (criterion)	Where in this report	Evidence
A1	A1.1 Represent the record	2	Two layouts (text and DataOutputStream): field order, delimiter / sizes, name containing the delimiter	A1.1, Figures A1.1–A1.2, Table A1.1	Design + verified in B1 tests
A1	A1.2 Design the operations	3	Pseudocode for add / find / list; stream classes chosen and justified	A1.2, pseudocode, Table A1.2	Pseudocode + B1 implementation
A1	A1.3 Design for damaged data	2	Damaged line, missing file, incomplete write (power failure)	A1.3, Table A1.3	Design + B1 tests T0, T7–T10, B2 file demo
A1	Alternative: RandomAccessFile	in task	Solution, O(n) vs O(1), padding waste	A1 Alternative, Figure A1.3, Table A1.4	Measured (B1.4 growth data)
A2	A2.1 Predict the cost	2	Number of read calls and why slow, with reasons	A2.1	Prediction checked in B1.3
A2	A2.2 Redesign using buffering	2	Buffered stream or byte[] buffer; change in operating-system calls	A2.2, Figure A2.1, Table A2.1	Measured call counts + times
A2	A2.3 When buffering does not help	1	One justified situation	A2.3	Measured (two situations)
A2	Alternative: Files.readAllLines()	in task	O(file size) vs O(buffer) memory, when each is appropriate	A2 Alternative, Table A2.2	Measured time and memory
A3	A3.1 Common pattern	2	Repeated steps and the parts that differ	A3.1, Table A3.1	Analysis
A3	A3.2 Generic Dao<T, K>	2	insert, findById, findAll, update, delete; StudentDao	A3.2, Figure A3.1, pseudocode	Design + runnable Java in Part B/appendix
A3	A3.3 Object–table mapping	1	Student fields → student columns, primary key	A3.3, Table A3.2, ER diagram	Design + ER
A3	Alternative: SQL in services	in task	Places to change on a column rename; time and space per operation	A3 Alternative, Tables A3.3–A3.4	Counted from source + measured
A4	A4.1 Decide what to persist	2	transient fields and why; object-graph behaviour	A4.1, Figure A4.1, Table A4.1	Reasoned + executed demo
A4	A4.2 Plan for change	2	Old file read by new class; role of serialVersionUID	A4.2, Table A4.2	Executed experiment (two class versions)
A4	Alternative: CSV / JSON	in task	File size, save/load time, new-field behaviour	A4 Alternative, Table A4.3, Figure A4.2	Measured
A5	A5.1 Transaction design	2	JDBC commit / rollback pseudocode; no money lost or created	A5.1, pseudocode, Figure A5.1	Pseudocode + executed in A5.3
A5	A5.2 Injection analysis	2	Crafted input; PreparedStatement redesign	A5.2, Figure A5.2, pseudocode	Reasoned + executed in B2
A5	Alternative: two auto-commit updates	in task	Outcome on failure; time cost; data risk	A5 Alternative, Table A5.1	Executed failure + timing
B1	B1.1 Validate the implementation	2	Tests: normal, id not found, damaged line; what each verifies	B1.1, Table B1.1	Executed test table (11 tests)
B1	B1.2 Compare with the design	1	Changes made to the Part A pseudocode and why	B1.2, Table B1.2	Analysis
B1	B1.3 Measure buffering	2	1, 10, 50 MB, unbuffered vs buffered; compare with A2.1	B1.3, Table B1.3, Figure B1.1	Measured
B1	B1.4 Measure growth	2	1,000 → 100,000 records; explain trend	B1.4, Table B1.4, Figure B1.2	Measured
B1	Complete the Code: PassStore	in task	TODO 1; output; why damaged P1002 does not stop the search	B1 Complete the Code	Executed, output equals expected
B1	Alternative: HashMap	in task	O(q·n) vs O(n+q); memory of both	B1 Alternative, Table B1.5, Figure B1.3	Measured

Task	Sub-question	Marks	What must be shown (criterion)	Where in this report	Evidence
B2	Adversarial login input	5 (total for B2)	Input that fools the unsafe check; PreparedStatement rejects it	B2.1, Tables B2.1–B2.2	Executed against a real SQL engine
B2	Damaged bus-pass file	(B2)	Wrong field count, non-numeric amount, empty line: store response	B2.2, Table B2.3	Executed
B2	What each input reveals	(B2)	Explanation per input	B2.3, Table B2.4	Reasoned
B2	Alternative: escaping / regex	in task	Why weaker; validation cost vs PreparedStatement	B2 Alternative, Tables B2.5–B2.6, Figure B2.1	Executed bypasses + measured
B3	B3.1 Correctness on small data	2	10 records per approach; SELECT and ResultSetMetaData	B3.1	Executed, 3/3 pass
B3	B3.2 Increase the data size	2	1,000 / 10,000 / 100,000 rows × Statement, PreparedStatement, batch	B3.2, Table B3.1, Figure B3.1	Measured
B3	B3.3 Auto-commit vs transaction	2	Batch with both modes; failure half-way; table contents	B3.3, Tables B3.2–B3.3	Measured + executed failures
B3	B3.4 Engineering trade-off	2	Choice for (a) nightly import of 100,000, (b) one pass at the counter	B3.4	Reasoned from the measurements
B3	Complete the Code: PassLoader	in task	TODO 1–3, loadAll with 1,000 / 10,000 / 100,000, invalid amount half-way → no rows	B3 Complete the Code	Executed
B3	Alternative: multi-row INSERT / LOAD DATA	in task	Time for n rows, memory for a batch of b rows	B3 Alternative, Table B3.4	Measured (LOAD DATA: design only)
B4	B4.1 Practical strategy	2	Migration with batches + transaction; failure report	B4.1, migration output	Executed
B4	B4.2 Inspect the database	1	DatabaseMetaData: product, driver version, tables; why useful	B4.2	Executed
B4	B4.3 Production decision	2	Text vs serialization vs database, justified by measurements and requirements	B4.3, Tables B4.1–B4.2, Figure B4.1	Measured + executed concurrency
B4	Alternative: one serialized file per campus	in task	Search time, space, concurrent counters in each design	B4 Alternative	Measured + executed concurrency

Table M.1 – Marks Map. Part A: 7 + 5 + 5 + 4 + 4 = 25 marks. Part B: 7 + 5 + 8 + 5 = 25 marks. Total 50.

Reading guide for an evaluator. In each task, the sub-questions appear in the assignment order with their marks in the heading. Part A contains the design reasoning and pseudocode (no Java in place of pseudocode, as the assignment requires); Part B contains the implementations (full source listings), the measurements (tables and figures generated from CSV files written by the programs), and the interpretation of each result.

PART A – Persistence Design & Pseudocode (25 marks)

All pseudocode follows the numbered-step style described in the front matter. Part A contains design reasoning and pseudocode only; the corresponding Java programs and all measurements are in Part B. Where a Part A claim is verified by an experiment, the text says which one.

Task A1 – Design a File Format (7 marks, K4)

Context: the transport office issues about 8,000 bus passes a year. A pass has: pass ID, student register number, name, route number, boarding stop, validity date, amount paid. The office must (i) add a pass, (ii) find a pass by pass ID, (iii) list all passes on a route.

A1.1 – Represent the record (2 marks)

(a) Delimited text file

A1.1(a) Delimited text file: one pass = one line, 8 fields separated by commas

passId	regNo	name	route	stop	validity	amount	crc32
P1001	192311001	Kumar\, Meena	12	Main Gate	2027-06-30	4500.00	84b0f00e

Delimiter = comma. Escape rules inside a field: , -> \, \ -> \\ line break -> \n
Split only at commas that are NOT escaped. crc32 = checksum of the first seven fields.

Example line (the name is Kumar, Meena):
P1001,192311001,Kumar\, Meena,12,Main Gate,2027-06-30,4500.00,84b0f00e
(file is ended by a line break after every record)

Figure A1.1 – Text layout: one pass = one line, eight comma-separated fields

#	Field	Type / format	Example	Check on reading
1	passId	text, e.g. P1001	P1001	not empty
2	regNo	text (9 digits)	192311001	–
3	name	text, may contain a comma	Kumar\, Meena	escapes removed after splitting
4	route	integer	12	must be an integer
5	stop	text	Main Gate	–
6	validity	date yyyy-mm-dd	2027-06-30	must be a real date
7	amount	decimal with 2 places	4500.00	must be a positive number
8	crc32	checksum of fields 1–7 (hex)	84b0f00e	must match (detects damage)

Table A1.1 – Field order of the text record

Delimiter and names that contain the delimiter. The delimiter is the comma and the record ends with a line break. Inside a field, a comma is written as \, a backslash as \\ and a line break as \n (backslash escaping). The reader splits only at commas that are not preceded by an escaping backslash and then removes the escapes. The name “Kumar, Meena” is therefore stored as Kumar\, Meena and is read back unchanged (verified in test T2 of Task B1). Without escaping, the line would split into nine fields and the record would be lost or mis-read.

(b) Binary file written with DataOutputStream

A1.1(b) Binary file written with DataOutputStream (variable length records)

passId writeUTF: 2-byte length + UTF-8	regNo writeUTF	name writeUTF	route writeInt: 4 bytes	stop writeUTF	validity writeLong: epoch day, 8 bytes	amount writeDouble: 8 bytes

No delimiter is needed: every text field starts with its own length, so a name that contains a comma (or any character) is stored unchanged.
The reader must read the fields in EXACTLY this order (readUTF, readUTF, readUTF, readInt, readUTF, readLong, readDouble).
A record is about 2+5 + 2+9 + 2+12 + 4 + 2+9 + 8 + 8 = 63 bytes for typical values.

Figure A1.2 – Binary layout (variable-length records)

Fields are written in this order: writeUTF(passId), writeUTF(regNo), writeUTF(name), writeInt(route), writeUTF(stop), writeLong(validity as epoch day), writeDouble(amount). writeUTF writes a 2-byte length followed by the text, so **no delimiter is needed and a name containing a comma needs no special handling**; the price is that the file cannot be read with a text editor and the reader must read

the fields in exactly the same order. A typical record is about 65 bytes (against about 72 bytes for the text line measured in Task B1).

A1.2 – Design the operations (3 marks)

```
ALGORITHM AddPass
INPUT   : pass (passId, regNo, name, route, stop, validity, amount), the pass file
OUTPUT  : the pass is appended as the last line, or an error is reported
BEGIN
  1.  IF any field is invalid (empty passId, route not an integer, impossible date, amount <= 0)
THEN RETURN error ENDIF
  2.  IF FindPass(passId) is not "not found" THEN RETURN error "duplicate pass id" ENDIF       //
O(n) scan
  3.  SET line <- join with commas: escape(passId), escape(regNo), escape(name), route,
escape(stop), validity, amount with 2 decimals
  4.  SET line <- line + "," + crc32(line)
  5.  IF the file exists AND its last byte is not a line break THEN append a line break ENDIF   //
repair after a half-written line
  6.  OPEN a BufferedWriter on the file in APPEND mode, creating the file if it is missing      //
try-with-resources
  7.  WRITE line followed by a line break in ONE write call
  8.  CLOSE the
writer                                                                          // flushes the
buffer
END

ALGORITHM FindPass
INPUT   : passId, the pass file
OUTPUT  : the pass, or "not found"
BEGIN
  1.  IF the file does not exist THEN RETURN "not found" ENDIF
  2.  OPEN a BufferedReader on the
file                                                        // try-with-resources
  3.  SET lineNo <- 0
  4.  WHILE readLine() returns a line DO
  5.      SET lineNo <- lineNo + 1
  6.      TRY    SET pass <- Parse(line)          // split at unescaped commas, expect 8 fields,
check crc32, convert route / date / amount
  7.      CATCH DamagedRecord(reason)    CALL report("line " + lineNo + ": " + reason);  CONTINUE
with the next line    ENDTRY
  8.      IF pass.passId = passId THEN CLOSE the reader; RETURN pass ENDIF
  9.  ENDWHILE
  10. CLOSE the reader;  RETURN "not found"
END

ALGORITHM ListByRoute
INPUT   : route number, the pass file
OUTPUT  : list of all passes whose route equals the given route
BEGIN
  1.  SET result <- empty list
  2.  OPEN a BufferedReader on the file (if the file is missing, RETURN result)
  3.  WHILE readLine() returns a line DO
  4.      TRY SET pass <- Parse(line)   CATCH DamagedRecord  report and CONTINUE   ENDTRY
  5.      IF pass.route = route THEN add pass to result ENDIF                       // no early
exit: every line must be examined
  6.  ENDWHILE
  7.  CLOSE the reader;  RETURN result
END
```

Operation	Stream classes chosen	Why this choice (and what was rejected)
Add	Files.newBufferedWriter(path, UTF-8, CREATE, APPEND) = BufferedWriter over OutputStreamWriter over FileOutputStream	The record is text, so a character writer is natural; the charset is stated explicitly (FileWriter used the platform default charset in older Java versions, so a file could be unreadable on another computer). APPEND means existing data is never overwritten. BufferedWriter collects the characters so the record reaches the operating system in one write. FileOutputStream alone writes bytes and would need manual encoding.
Find / List	Files.newBufferedReader(path, UTF-8) = BufferedReader over InputStreamReader over FileInputStream	readLine() returns one record at a time, which is exactly the unit of this format. The buffer reduces operating-system reads from one per byte to one per 8 KB (measured in Task A2/B1.3). A plain FileInputStream.read() or FileReader.read() per character would be thousands of times slower for no gain.
Repair check before add	RandomAccessFile (read the last byte)	Reading the last byte needs a seek, which stream classes do not offer; it costs O(1).

Table A1.2 – Stream choices and justification

A1.3 – Design for damaged data (2 marks)

Problem	How it is detected	How it is handled
A damaged line (edited by hand, cut, wrong number of fields, bad number or date)	Parse() checks four things in order: the number of fields (8), the CRC-32 of the first seven fields, the integer route, the date and the positive amount. Any failure throws DamagedRecord(reason).	The line is skipped and reported with its line number and reason; the scan continues with the next line, so one bad line never hides the good records. The report is available to the caller (verified: Task B1 tests T7–T8 and the nine damaged lines of Task B2).
A missing file	Files.exists(path) is false (or NoSuchFileException).	find() returns “not found” and list() returns an empty list: a missing file means “no passes yet”, not a crash. add() creates the file (CREATE option). Verified: test T0.
An incomplete write caused by a power failure	The last line is cut: it fails the field count or the CRC check, and the file does not end with a line break.	Each record is written with one write call, so a power cut can damage only the LAST line, never an earlier one. The damaged last line is skipped on reading. Before the next add, the program checks the last byte of the file and, if it is not a line break, writes one, so the new record does not get glued to the broken line (verified: test T10). Operations that rewrite the whole file (delete / update) write a temporary file and replace the old one with an atomic move, so there is always one complete file.

Table A1.3 – Design for damaged data

Limit of the design. Closing the writer hands the data to the operating system, not necessarily to the disk. A stronger guarantee needs FileChannel.force(true) after every record; Task B3 shows what a durable commit costs on this disk (about 0.8 ms per commit), which is why the design does not force every record.

Alternative Concept and Complexity Analysis – RandomAccessFile with fixed-length records

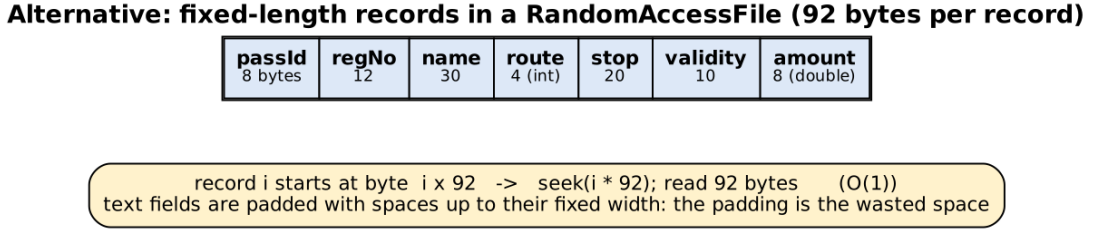


Figure A1.3 – Fixed-length record layout (92 bytes)

```
ALGORITHM FindPassFixed
INPUT   : passId (for example "P1003"), the record file, RECORD_SIZE = 92
OUTPUT  : the pass, or "not found"
```

```
BEGIN
  1.  SET i <- (number in passId) - 1001                    // pass ids are dense: P1001 is
record 0
  2.  IF i < 0 OR (i + 1) * RECORD_SIZE > file length THEN RETURN "not found" ENDIF
  3.  CALL seek(i * RECORD_SIZE)
  4.  CALL readFully(record of RECORD_SIZE bytes)
  5.  IF the passId field of the record is not passId THEN RETURN "not found" ENDIF   // empty
slot
  6.  RETURN decode(record)
END
ALGORITHM AddPassFixed:  SET i from passId; seek(i * RECORD_SIZE); write the 92 bytes padded with
spaces.
```

Time. The delimited file must be scanned: O(n) lines. The fixed-length file needs one seek and one read of 92 bytes: **O(1)**. Measured in Task B1.4 (n = 1,000 → 100,000): the scan grew from 1.83 ms to 103.1 ms while the seek stayed at 3.7–3.7 µs for every n (Table B1.4).

Space wasted by padding. Every record occupies 92 bytes even if the fields are short. In the benchmark data the real content of a record averages 59 bytes, so **33 bytes (35%) of every record is padding**. The text line needs only 72 bytes on average (measured: 7,211,928 bytes for 100,000 passes against 9,200,000 bytes for the fixed file, i.e. the fixed file is 28% larger). For 8,000 passes the difference is only about 155 KB, so the space cost is small in absolute terms.

	Delimited text file	**Fixed-length RandomAccessFile**
Find by pass ID	O(n) scan	O(1) seek (needs dense, numbered ids)
List by route	O(n) scan	O(n) scan (every record must still be read)
Space	only the real content (about 72 bytes per record)	92 bytes per record, padding about 36%; fixed field widths (a longer name is cut)
Delimiters inside data	need escaping	not an issue
Deleting a record	rewrite the file	leaves a hole (needs a free-slot marker)
Human readable	yes	no

Table A1.4 – Comparison. Conclusion: the fixed-length file is better only when pass ids are dense and lookup by id dominates; for the requirement "list by route" both designs scan.

Task A2 – Don't Read What You Don't Need (5 marks, K4)

A program reads a 50 MB attendance log one byte at a time with FileInputStream.read() to count the lines.

A2.1 – Predict the cost (2 marks)

Prediction. A 50 MB file has 52,428,800 bytes. FileInputStream.read() returns one byte per call and the last call returns −1, so the program makes **about 52.4 million read calls (file size + 1)**. Each call crosses from Java into native code and into the operating system (a system call), which costs a few hundred nanoseconds even though only one byte is delivered; 52 million calls therefore cost several seconds to tens of seconds, and almost all of the time is call overhead, not disk transfer. I predicted a time of roughly 10 s (about 200 ns per call). **Checked in Task B1.3:** the real log file of 52,428,783 bytes caused **52,428,784 read calls** (exactly file size + 1) and took 16.2 s, i.e. 309 ns per call. The number of calls was predicted exactly and the time within the predicted order of magnitude.

A2.2 – Redesign using buffering (2 marks)

```
ALGORITHM CountLinesBuffered
INPUT   : file name
OUTPUT  : number of line breaks
BEGIN
  1.  SET buffer <- array of 65,536 bytes;  SET lines <- 0
  2.  OPEN a FileInputStream on the file                                    // try-with-
resources
  3.  REPEAT
  4.      SET n <- read(buffer)                                             // ONE system call
fills up to 65,536 bytes
  5.      FOR i FROM 0 TO n - 1 DO  IF buffer[i] is a line break THEN lines <- lines + 1 ENDIF
ENDFOR
  6.  UNTIL n = -1
  7.  CLOSE the stream;  RETURN lines
END
// Variant: wrap the FileInputStream in a BufferedInputStream and keep the one-byte read(): the
program still calls read()
// 52 million times, but each call is served from a memory buffer; the operating system is asked
only once per 8,192 bytes.
```

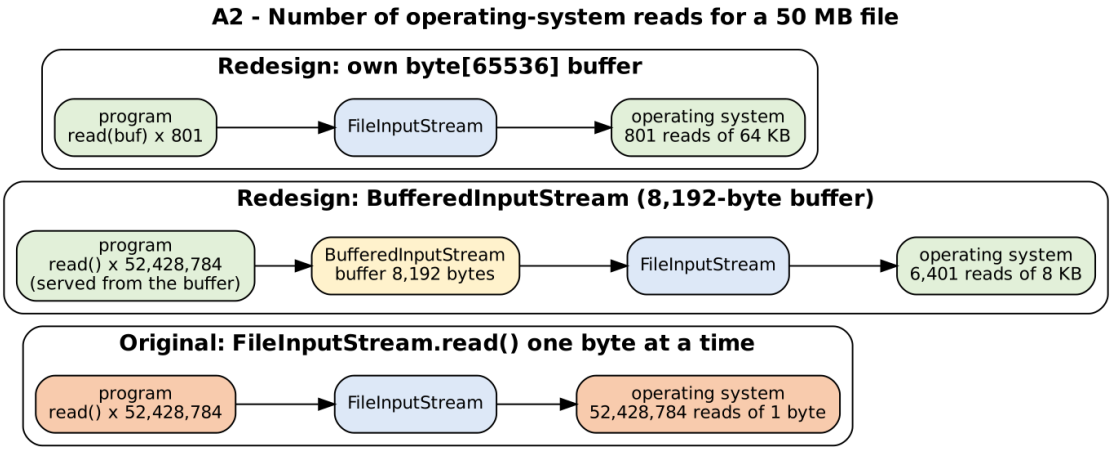


Figure A2.1 – How buffering changes the number of operating-system reads

Version (50 MB)	Calls to the file stream (= operating-system reads)	Time	Calls compared with the original
Original: FileInputStream.read()	52,428,784	16214.8 ms	1

Version (50 MB)	Calls to the file stream (= operating-system reads)	Time	Calls compared with the original
BufferedInputStream (8,192-byte buffer), still read() per byte	6,401	1067.7 ms	1 / 8,191
Own byte[65,536] buffer	801	21.4 ms	1 / 65,454

Table A2.1 – Operating-system reads and time (measured, Task B1.3). Calls = ceil(file size / buffer size) + 1.

Explanation. Buffering changes how many times the operating system is asked: instead of one request per byte, one request per buffer. The work of examining each byte remains (O(file size)), but the expensive part, the system call, is divided by the buffer size (8,192 or 65,536). The own byte[] buffer is faster still because the program also avoids 52 million method calls into the BufferedInputStream, each of which goes through the stream's locking and bounds checks.

A2.3 – When buffering does not help (1 mark)

Situation: reading in large blocks. If the program already reads in blocks at least as large as the buffer, a BufferedInputStream passes the request straight through and adds nothing. Measured on a 10 MB file with 64 KB blocks: FileInputStream 0.8 ms (161 calls) and the same stream wrapped in a BufferedInputStream 0.9 ms (161 calls) – the same number of calls and no gain. **A second situation: data already in memory.** Reading a ByteArrayInputStream byte by byte took 255.5 ms and wrapped in a buffer 210.8 ms: there is no operating-system call to save, so the benefit is small (compare with the 16,215 ms of the unbuffered 50 MB file). Buffering also does not help random access, where the next byte needed is not the next byte of the file.

Alternative Concept and Complexity Analysis – Files.readAllLines()

```
ALGORITHM CountLinesReadAll
BEGIN
  1.  SET lines <- Files.readAllLines(path)          // the whole file becomes a List<String> in
memory
  2.  RETURN size of lines
END
```

	Buffered stream / byte[] buffer	Files.readAllLines()
Time	O(file size): every byte is examined once	O(file size): every byte is decoded and copied into Strings
Memory	O(buffer size): 8,192 or 65,536 bytes, independent of the file	O(file size): all lines are held at once
Measured, 50 MB file (52,428,783 bytes, 1,588,751 lines)	byte[64 KB]: 21.4 ms, 0.009 MB allocated for the 8 KB buffer case	291 ms, 115 MB retained (2.3× the file size), 131 MB allocated while reading
Appropriate when	big or unknown file size, one pass, limited heap (logs, imports)	small files, or when the program needs all lines at once (sort, random access to lines, several passes)

Table A2.2 – Buffered streaming against reading everything (measured; programs: BufferingBenchmark, ReadAllLinesMemory)

The memory result is the important one: the 50 MB file needs **115 MB** of heap as a List<String> (each of the 1.59 million lines is a separate String object with its own header and array), so a program with a 100 MB heap would fail with OutOfMemoryError on a 50 MB log, while the buffered version would not even notice the file size. readAllLines was quick here (291 ms) because it reads the file in large blocks internally; its cost is memory, not time.

Task A3 – Discover the Data-Access Design Pattern (5 marks, K4)

A student project has three classes (StudentDB, CourseDB, MarksDB). Each opens a connection, builds an SQL string, executes it, reads the ResultSet into objects and closes the connection; large parts are almost identical.

A3.1 – Identify the common pattern (2 marks)

Step (in every method of all three classes)	Repeated in all three?	Differs between the classes
1. Obtain a Connection (and close it at the end)	identical	–
2. Write the SQL text	same structure	the SQL text: table name, column names
3. Create a Statement / PreparedStatement	identical	–
4. Bind the parameters	same idea	number, types and order of the parameters
5. Execute (executeQuery / executeUpdate)	identical	–
6. Read the ResultSet and copy the columns into an object	same loop	the target class and the column → field mapping
7. Handle SQLException, close ResultSet, Statement and Connection	identical	–

Table A3.1 – Common steps and differing parts. Only steps 2, 4 and 6 differ: the SQL text, the parameters and the row-to-object mapping; steps 1, 3, 5 and 7 (the "plumbing") are identical.

A3.2 – A generic data-access interface (2 marks)

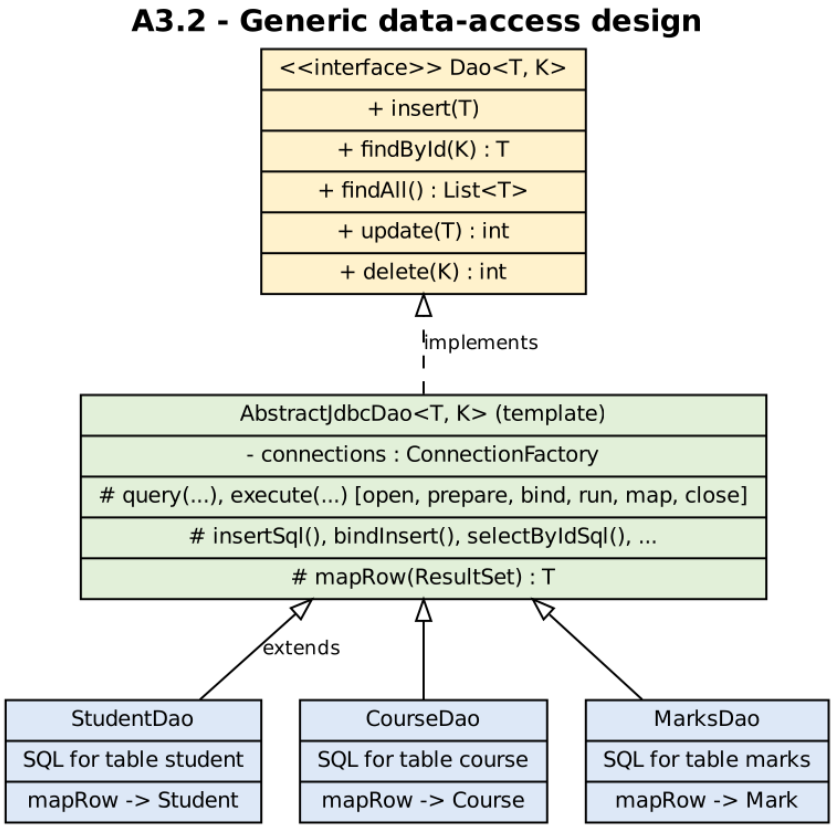


Figure A3.1 – Dao<T, K>, the template class and three DAOs

```
INTERFACE Dao<T, K>                    // T = entity class, K = type of the primary key
  insert(entity)                       // store a new row
  findById(key)   -> T or "none"
  findAll()       -> list of T
  update(entity) -> number of rows changed
  delete(key)    -> number of rows deleted
END INTERFACE
```

```
ALGORITHM Execute(sql, bindParameters, mapRow)        // written ONCE in the shared base class
BEGIN
  1.  OPEN connection;  PREPARE statement from sql                       // try-with-resources:
closed automatically
  2.  CALL bindParameters(statement)
  3.  IF the statement is a query THEN
  4.      FOR EACH row IN the result DO  ADD mapRow(row) to the result list  ENDFOR
  5.  ELSE  SET rows <- statement.executeUpdate()  ENDIF
  6.  CLOSE everything;  RETURN the result list or rows
END

CLASS StudentDao IMPLEMENTS Dao<Student, String>        // supplies only what differs
  insert(s)    : CALL Execute("INSERT INTO student(reg_no, name, email, year_no) VALUES
(?, ?, ?, ?)",
                              bind s.regNo, s.name, s.email, s.year)
  findById(k)  : RETURN first of Execute("SELECT reg_no, name, email, year_no FROM student WHERE
reg_no = ?", bind k, mapStudent)
  findAll()    : RETURN Execute("SELECT reg_no, name, email, year_no FROM student ORDER BY
reg_no", no parameters, mapStudent)
  update(s)    : RETURN Execute("UPDATE student SET name = ?, email = ?, year_no = ? WHERE reg_no
= ?", bind ...)
  delete(k)    : RETURN Execute("DELETE FROM student WHERE reg_no = ?", bind k)
  mapStudent(row) : RETURN new Student(row.reg_no, row.name, row.email, row.year_no)
END CLASS                                              // CourseDao and MarksDao: only the SQL
text and the mapping change
```

The Java implementation (Dao, AbstractJdbcDao, StudentDao, CourseDao, MarksDao) is in Part B / Appendix and was executed: see the output of A3Demo below.

A3.3 – Map objects to tables (1 mark)

Student field (Java)	Column of table student	SQL type	Constraint
String regNo	reg_no	VARCHAR(12)	PRIMARY KEY (identifies the row; K = String in Dao<Student, String>)
String name	name	VARCHAR(60)	NOT NULL
String email	email	VARCHAR(80)	NOT NULL
int year	year_no	INT	NOT NULL (the name “year” is avoided because it is a SQL keyword)

Table A3.2 – Object–table mapping: one class = one table, one object = one row, one field = one column

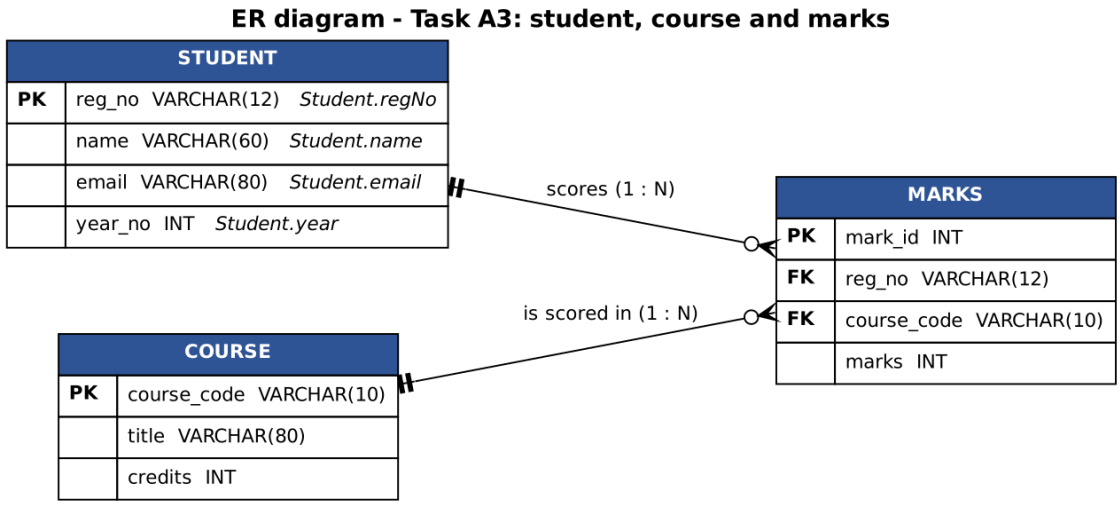


Figure A3.2 – ER diagram of student, course and marks

Alternative Concept and Complexity Analysis – SQL written inside the service / GUI classes

The alternative solution. The service and GUI classes contain their own SQL, each repeating the plumbing (steps 1, 3, 5, 7). The classes ScatteredStudentService, ScatteredReportScreen, ScatteredRegistrationScreen and ScatteredMarksService in Task A3 implement this alternative.

```
// generic DAO: ONE place knows the table                    // scattered: every class that needs
a student writes its own SQL
students.findById("192311001");                              ps = con.prepareStatement("SELECT
reg_no, name, email, year_no FROM student WHERE reg_no = ?");
                                                             ps.setString(1, regNo); rs =
ps.executeQuery(); ... build Student ... close all ...
```

Output of java A3Demo (the DAO design in use, the rename count, and the per-operation cost)

```
=== A3.2 One generic design, three entities ===
findById(192311002)      : 192311002 Divya <divya@example.com> year 3
findById(000000000)      : null   (no such row)
update Arun's year       : rows changed = 1
findAll()                : [192311001 Arun <arun@example.com> year 3, 192311002 Divya <divya@example.com>
year 3]
delete(192311002)        : rows deleted = 1, remaining = 1
CourseDao.findAll()      : [CSA09 Programming in Java (4 credits)]
MarksDao.findAll()       : [#1 192311001 CSA09 = 88, #2 192311001 CSA10 = 76]

=== Alternative: SQL written inside service / GUI classes ===
ScatteredStudentService.find(192311001) : 192311001 Arun <arun@example.com> year 3   (same result)

=== Rename a column of the student table: SQL statements that mention the table, and in how many classes
===
  generic DAO design : StudentDao.java -> 5 statement(s) on the student table
  scattered design   : ScatteredStudentService.java -> 2 statement(s) on the student table
  scattered design   : ScatteredReportScreen.java -> 1 statement(s) on the student table
  scattered design   : ScatteredRegistrationScreen.java -> 2 statement(s) on the student table
  scattered design   : ScatteredMarksService.java -> 1 statement(s) on the student table
  generic DAO : 5 statements, all in 1 class (StudentDao)
  scattered   : 6 statements spread over 4 classes (every service / screen that touches the table)

=== Cost per findById (new connection each time, as in the original StudentDB class) ===
generic Dao<Student,String>.findById : 60.0 microseconds per call, 3,798 bytes allocated per call
SQL inside the service class         : 73.7 microseconds per call, 2,848 bytes allocated per call
(checksum 30600)
```

	Generic Dao<T, K>	SQL in service / GUI classes
Places to change when a column of the student table is renamed	5 SQL statements, all in 1 class (StudentDao)	6 SQL statements spread over 4 classes: every class that touches the table must be found, edited and re-tested
Time per operation (new connection each call)	60.0 µs	73.7 µs
Memory allocated per operation	3,798 bytes	2,848 bytes
Asymptotic cost	O(1) per statement in both designs (the same JDBC calls)	O(1) (the same JDBC calls)

Table A3.3 – Comparison (counted from the source files and measured by A3Demo; the time varies between runs: 60–81 µs in the two runs made)

Analysis. The number of SQL statements is almost the same in both designs (5 and 6); the difference is **where they are**: one class against four. A column rename in the DAO design means editing and re-testing StudentDao only; in the scattered design the programmer has to search every service and screen (and a missed one fails only at run time). The cost per operation is the same within the noise of the measurement because both designs perform the same JDBC calls and the time is dominated by opening the connection (about 60–80 µs); the generic design allocates about 1 KB more per call

(the lambda binders, a list and the mapper), which is irrelevant next to the connection. **Conclusion:** the generic DAO costs nothing measurable and removes duplicated code and the risk of inconsistent changes.

Task A4 – What Should Be Serialized? (4 marks, K4)

A hostel program saves its state by serializing a Hostel object that holds a list of Room objects, each holding a list of Resident objects. A Resident has a name, a register number, a password and a reference to a live database Connection.

A4.1 – Decide what to persist (2 marks)

Field of Resident	transient?	Reason
name	no	real data of the object; must survive
regNo	no	real data of the object; must survive
password	yes	a secret: writing it to a file stores it in clear text where anyone who can read the file can see it (measured: the password text was found in the bytes of a non-transient version). If a password must be kept it should be a salted hash, handled separately.
Connection conn	yes	a live resource that is meaningless after the program ends (the socket and session no longer exist), and java.sql.Connection is not Serializable: with a non-transient connection the whole writeObject call fails with NotSerializableException. After reading, the field is null and the program opens a new connection.

Table A4.1 – transient design

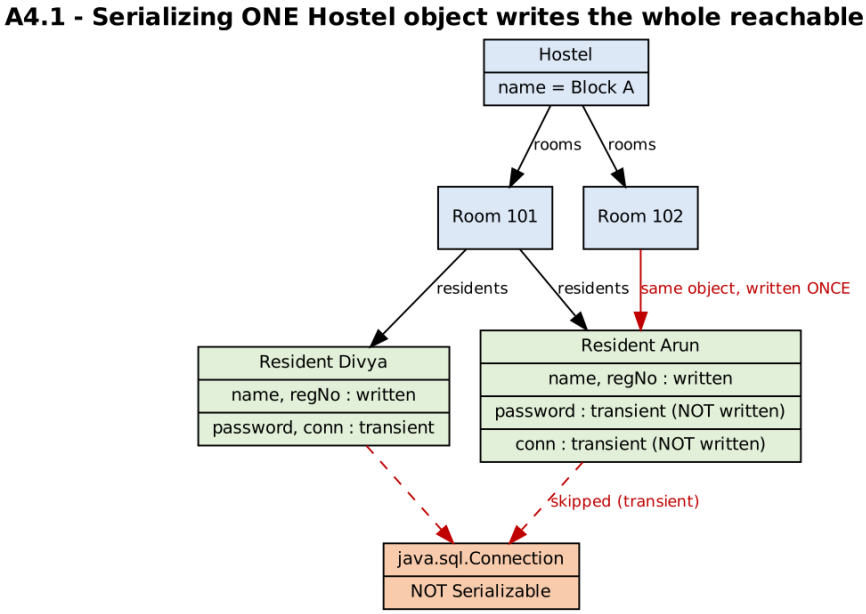


Figure A4.1 – What writeObject(hostel) writes

What happens to the object graph. Serialization follows references: writeObject(hostel) writes the Hostel, then every Room in its list, then every Resident in each room, i.e. **everything reachable from the object**, in one call. A Resident that is referenced from two rooms is written **once** and both rooms refer to the same object again after reading (a handle table records objects already written; cycles are handled in the same way). If any reachable object is not Serializable and is not reached through a transient field, the whole write fails. The executed demonstration below confirms each of these statements.

Output of java SerializationDemo

```
=== 1. Serialize the Hostel (one writeObject call) ===
bytes written: 341
hostel name after reading : Block A, rooms = 2
  room 101 -> [Arun(192311001, pwd=null, conn=null), Divya(192311002, pwd=null, conn=null)]
  room 102 -> [Arun(192311001, pwd=null, conn=null)]
transient fields after reading: password = null, conn = null   (not stored, reset to null)
Arun in room 101 and Arun in room 102 are the same object after reading? true   (a shared reference is
written once and shared again)
```

```
-> writeObject(hostel) also wrote ALL reachable objects: 1 Hostel, 2 Rooms and 2 distinct Residents (3
references).
password visible in the file bytes? false

=== 2. The wrong design: a live Connection that is not transient ===
NotSerializableException: SerializationDemo$$Lambda/0x00007f770c000a08
with conn = null the same class is written, but the password IS in the file: true
```

A4.2 – Plan for change (2 marks)

A phone-number field is added to Resident after files were saved. Two versions of the class were written, compiled separately, and the old files were read with the new class (programs in Task A4/v1 and v2, run by versioning.sh):

Output of ./versioning.sh

```
version 1 (no phone field): wrote fixed_v1.ser and implicit_v1.ser
computed serialVersionUID of ResidentImplicit v1 = 2510292752369857995
version 2 (class now has a phone field) reads the OLD files:
computed serialVersionUID of ResidentImplicit v2 = 1544415746870410737
  explicit serialVersionUID = 1L : OK  -> name=Arun, regNo=192311001, phone=null   (new field gets its
default value)
  no serialVersionUID           : InvalidClassException -> ResidentImplicit; local class incompatible:
stream classdesc serialVersionUID = 2510292752369857995, local class serialVersionUID =
1544415746870410737
```

Class declaration	Old file read by the new class	Why
private static final long serialVersionUID = 1L;	Works: name and regNo are restored, phone = null (the default of the new field)	The stream records the class's serialVersionUID; the reader compares it with the local class. Equal numbers mean "same class, compatible change": fields missing in the stream get default values, extra fields in the stream are ignored.
no serialVersionUID (the compiler computes it)	InvalidClassException: "local class incompatible: stream classdesc serialVersionUID = …, local class serialVersionUID = …" (numbers differ)	The computed number is derived from the class structure (name, fields, methods); adding a field changes it, so the new class is judged incompatible with the old files and all saved data becomes unreadable.

Table A4.2 – Effect of serialVersionUID (executed)

Rule: always declare serialVersionUID explicitly; keep it when the change is compatible (adding a field, adding a method); change it, or write a custom readObject that migrates the data, when the change is incompatible (removing a field, changing a field's type, changing the class hierarchy).

Alternative Concept and Complexity Analysis – CSV or JSON text instead of serialization

The alternative solution. Each resident becomes one text line (CSV: name,regNo,room,phone with a header line; quotes around a field that contains a comma) or one JSON object. The program writes the fields itself and parses them itself, so the format is independent of the class: it can be read by another language, opened in a text editor, and does not depend on serialVersionUID. The experiment FormatBenchmark saves and loads n residents in all three formats (round trip verified by the program).

n	Format	File size (bytes)	Save (ms)	Load (ms)
1,000	serialization	59,044	1.7	5.9
1,000	CSV	44,912	1.2	1.1
1,000	JSON	85,893	2.7	2.3
10,000	serialization	599,044	16.7	24.9
10,000	CSV	458,912	12.8	12.5
10,000	JSON	868,893	16.1	13.8
100,000	serialization	6,089,044	95.7	124.9

n	Format	File size (bytes)	Save (ms)	Load (ms)
100,000	CSV	4,688,912	12.4	35.5
100,000	JSON	8,788,893	19.7	78.7

Table A4.3 – Size and time (best of 5 runs; from TaskA4/format_results.csv)

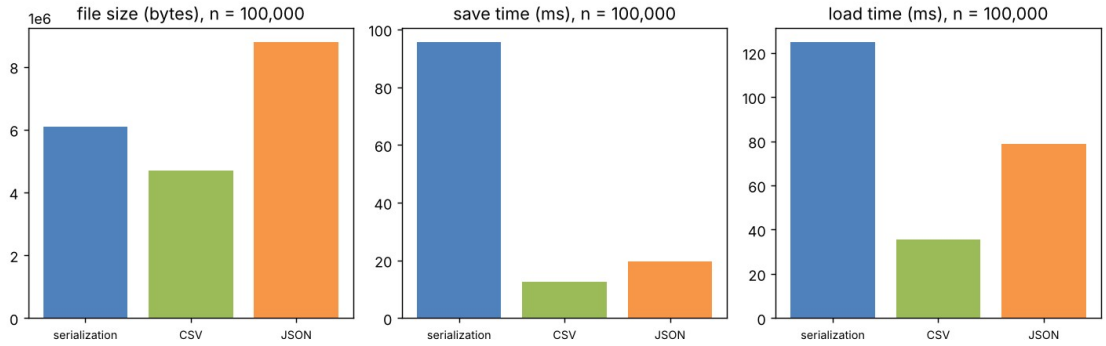


Figure A4.2 – Size and times for 100,000 residents

Output of java FormatBenchmark (includes the new-field experiment)

```
=== Serialization vs CSV vs JSON for n residents (best of 5 runs) ===
        n format             file bytes    save (ms)    load (ms)
    1,000 serialization          59,044          1.7          5.9
    1,000 CSV                    44,912          1.2          1.2
    1,000 JSON                   85,893          2.7          2.3
   10,000 serialization         599,044         16.7         24.9
   10,000 CSV                   458,912         12.8         12.5
   10,000 JSON                  868,893         16.1         13.8
  100,000 serialization       6,089,044         95.7        124.9
  100,000 CSV                 4,688,912         12.4         35.5
  100,000 JSON                8,788,893         19.7         78.7

=== A NEW FIELD (building) is added after files were saved: old file read by the new program ===
CSV, positional reader expecting 5 columns : ArrayIndexOutOfBoundsException (Index 4 out of bounds for
length 4)
CSV, header-aware reader (looks up 'building')      : (missing -> default)  (works, if the reader is
written that way)
JSON, reader looks up the key 'building'            : null  (missing key -> default; extra keys in the
file are ignored)
Java serialization with fixed serialVersionUID      : new field = default value; WITHOUT a fixed UID:
InvalidClassException (see versioning output)
(checksum 0)
```

Analysis. For 100,000 residents: serialization 6,089,044 bytes, CSV 4,688,912 bytes, JSON 8,788,893 bytes. CSV is the smallest (it has no field names) and the fastest to load (35.5 ms); JSON is the largest because every record repeats the field names; serialization is in between in size but slowest to load (124.9 ms) because the stream describes classes and objects and the reader rebuilds every object by reflection. All three are O(n) in time and space. **New field:** serialization copes if serialVersionUID is fixed (new field = default) and fails otherwise; a CSV reader that reads columns by position fails (ArrayIndexOutOfBoundsException in the experiment) unless it reads the header and looks columns up by name; JSON copes best because the reader looks fields up by key and a missing key can take a default. **When to choose:** serialization for a quick save of a complete object graph inside one program version; CSV/JSON when the data must outlive the program, be edited by people or be read by other systems.

Task A5 – Design for Correctness and Safety (4 marks, K4)

A5.1 – Transaction design (2 marks)

```
ALGORITHM TransferMoney
INPUT   : fromAccount (hostel wallet), toAccount (canteen wallet), amount = 500
OUTPUT  : both balances changed, or neither changed
BEGIN
  1.  OPEN a connection;  SET oldMode <- con.getAutoCommit()
  2.  CALL con.setAutoCommit(false)                        // from now on, changes are NOT
permanent until commit()
  3.  TRY
  4.      SET n1 <- executeUpdate("UPDATE account SET balance = balance - ? WHERE acc_id = ?",
amount, fromAccount)
  5.      SET n2 <- executeUpdate("UPDATE account SET balance = balance + ? WHERE acc_id = ?",
amount, toAccount)
  6.      IF n1 <> 1 OR n2 <> 1 THEN  THROW "account not found"  ENDIF
  7.      CALL con.commit()                                 // both updates become permanent
together
  8.  CATCH any exception (SQLException, constraint violation CHECK balance >= 0, program error)
  9.      CALL con.rollback()                               // undo the first update: neither
change is kept
  10.     RETURN failure
  11. FINALLY
  12.     CALL con.setAutoCommit(oldMode);  CLOSE the connection
  13. RETURN success
END
```

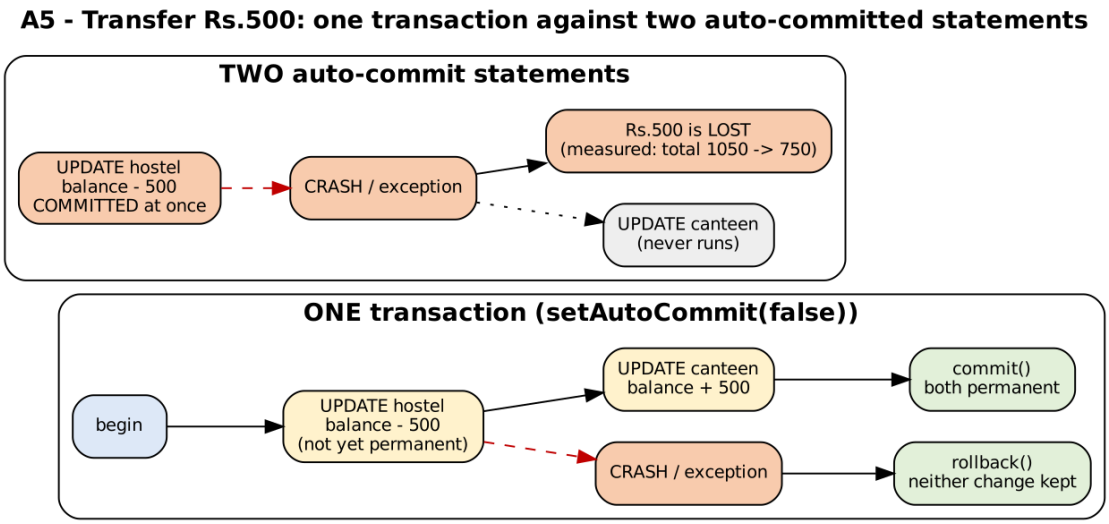


Figure A5.1 – One transaction against two auto-committed updates

Why money is never lost or created. A transaction is atomic: the two updates are one unit. If the program fails between them (exception, crash, power failure), the first update was never committed, so the database discards it (rollback, or recovery after a crash) and both balances are as before. The UPDATE uses balance = balance − ? instead of reading the balance and writing a computed value, so two simultaneous transfers cannot overwrite each other's result; the CHECK (balance ≥ 0) constraint in the table rejects a debit that would make the balance negative, and the rejection rolls the whole transfer back.

A5.2 – Injection analysis (2 marks)

The login check is String sql = "SELECT * FROM users WHERE id='" + id + "' AND pwd='" + pwd + "'";

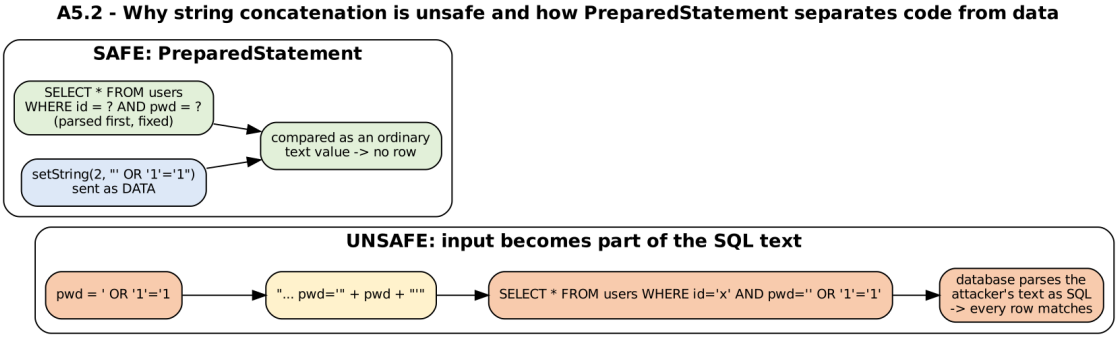


Figure A5.2 – Concatenation lets the input change the SQL; PreparedStatement keeps input as data

Crafted input	SQL that is built	Why the query succeeds without a valid password
id = x, pwd = ' OR '1'='1	SELECT * FROM users WHERE id='x' AND pwd='' OR '1'='1'	AND binds tighter than OR, so the condition is (id='x' AND pwd='') OR '1'='1'; '1'='1' is always true, so every row matches.
id = arun' OR '1'='1, pwd = anything	... WHERE id='arun' OR '1'='1' AND pwd='anything'	id='arun' is true, so the password test is bypassed: the attacker is logged in as arun.
id = admin' -- (with a space after the dashes), pwd = anything	SELECT * FROM users WHERE id='admin' -- ' AND pwd='anything'	-- starts a comment: the password test is removed from the statement; the attacker is logged in as admin.

Table A5.2 – Three crafted inputs (all three were executed in Task B2 and succeeded against the unsafe code)

Root cause. The program builds code (SQL) by pasting data into it, so the database cannot tell where the programmer's SQL ends and the user's text begins. **Redesign:**

```
ALGORITHM CheckLogin
INPUT   : id, pwd (text typed by the user)
OUTPUT  : true if exactly the matching user exists
BEGIN
  1.  PREPARE statement from the FIXED text  "SELECT * FROM users WHERE id = ? AND pwd = ?"   //
the SQL structure is complete before any user data exists
  2.  CALL statement.setString(1, id)                                                          //
id is sent as a value, never parsed as SQL
  3.  CALL statement.setString(2, pwd)
  4.  EXECUTE the query;  SET found <- the result has a row
  5.  CLOSE the result and the statement                                                       //
try-with-resources
  6.  RETURN found
END
```

With the PreparedStatement the three inputs above are compared as ordinary text values: no row has the id “admin' -- ”, so the query returns no row (verified in Task B2). (Further protection, outside this question: store a salted hash of the password instead of the password, give the database user only the rights it needs, and do not show database error messages to the user.)

Alternative Concept and Complexity Analysis – two separate auto-commit updates

The alternative solution. The two UPDATE statements are executed with auto-commit ON, so each is its own transaction and is made permanent immediately (TransferDemo, method transferAuto). The executed outcomes:

Output of java TransferDemo

```
=== A5.3 Transfer Rs.300 from the hostel wallet to the canteen wallet ===
1. transaction, no failure
    before: hostel=1000.00 canteen=50.00 total=1050.00
    error : none
    after : hostel=700.00 canteen=350.00 total=1050.00
2. TRANSACTION, crash between the two updates
    before: hostel=1000.00 canteen=50.00 total=1050.00
    error : IllegalStateException: simulated crash between the two updates
```

```
    after : hostel=1000.00 canteen=50.00 total=1050.00
3. AUTO-COMMIT, crash between the two updates (debit first)
    before: hostel=1000.00 canteen=50.00 total=1050.00
    error : IllegalStateException: simulated crash between the two updates
    after : hostel=700.00 canteen=50.00 total=750.00
4. AUTO-COMMIT, credit first, then the debit fails (balance too low: 2,000)
    before: hostel=1000.00 canteen=50.00 total=1050.00
    error : SQLIntegrityConstraintViolationException: CHECK constraint failed: balance >= 0
    after : hostel=1000.00 canteen=2050.00 total=3050.00
5. TRANSACTION, credit first, then the debit fails (balance too low: 2,000)
    before: hostel=1000.00 canteen=50.00 total=1050.00
    error : SQLIntegrityConstraintViolationException: CHECK constraint failed: balance >= 0
    after : hostel=1000.00 canteen=50.00 total=1050.00

=== Cost: 2,000 transfers of Rs.1 (database on disk, every commit is made durable) ===
one transaction per transfer (1 commit)     : 1599.6 ms in total, 800 us per transfer
two auto-committed updates (2 commits)      : 2767.3 ms in total, 1384 us per transfer
ratio auto-commit / transaction             : 1.73
```

Scenario (Rs.300 transfer; balances hostel / canteen)	Result	Total of both accounts
Transaction, no failure	700 / 350	1050 (unchanged)
Transaction, crash between the two updates	1000 / 50 (rolled back)	1050 (unchanged)
Auto-commit, crash between the two updates (debit first)	700 / 50: the hostel wallet is debited, the canteen wallet never credited	750: Rs.300 LOST
Auto-commit, credit first, then the debit fails (balance too low)	1000 / 2050: the canteen wallet was credited, the debit was rejected	3050: Rs.2000 CREATED
Transaction, credit first, then the debit fails	1000 / 50 (rolled back)	1050 (unchanged)

Table A5.1 – Outcome of a failure between the two updates (executed)

Time cost (2,000 transfers, database on disk, every commit made durable): one transaction per transfer 800 µs; two auto-committed updates 1384 µs; the ratio is 1.73. The reason is the number of commits, not the number of updates: a transaction commits once (one durable write), two auto-commit statements commit twice (about 0.8 ms each on this disk, see Task B3). So the **transaction is also faster**, not slower. The cost of a transaction is something else: it holds its locks until commit, which matters under heavy concurrency, but for two short updates this is negligible. **Risk to data correctness:** auto-commit is correct only when each statement is independently meaningful; for any operation made of several statements that must succeed together (transfers, importing a file, issuing a pass and updating a counter) auto-commit leaves a window in which a failure produces lost or invented money, as the table shows. Space cost: O(1) in both designs (the transaction keeps the uncommitted changes of two rows in the log).

PART B – Implementation & Experimental Investigation (25 marks)

Part B implements the designs of Part A, tests them, measures them and interprets the results. All Java programs listed here were compiled and executed; the console outputs are pasted unchanged. The JDBC programs ran against the real SQLite engine through the test driver described in the front matter and Appendix B (see the limitations there).

Task B1 – Implement and Instrument the File Store (7 marks, K4)

The bus-pass store designed in A1 (text format) and the line counters of A2 were implemented in Java. The classes are: BusPass (record, text line, parsing), DamagedRecordException, BusPassStore (add / find / listByRoute), LineCounter (the counters of A2), FixedRecordStore (the RandomAccessFile alternative), PassStore (the Complete-the-Code class) and the test and benchmark programs. All source code follows in this task.

B1.1 – Validate the implementation (2 marks)

BusPassStoreTest runs eleven tests; each is listed with what it verifies. The damaged lines of tests T7–T10 were created on purpose by appending bad text to the store file.

Output of java BusPassStoreTest

```
=== B1.1 Test table (what each test verifies is in the text of the report) ===
PASS T0   find() on a missing file                          returns null, no exception: a missing file
= empty store
PASS T1   NORMAL: add 4 passes, find P1002                  P1002 192311002 Divya route 7 Main Gate
valid 2027-06-30 Rs.4500.0
PASS T2   NORMAL: name with a comma survives                stored line: P1003,192311003,Kumar\,
Meena,12,Main Gate,2027-06-30,4500.00,6717420c
PASS T3   NORMAL: name with a backslash survives            P1004 192311004 A\B route 9 Main Gate
valid 2027-06-30 Rs.4500.0
PASS T4   NORMAL: list route 12 gives 2 passes              P1001, P1003
PASS T5   NOT FOUND: find("P9999") returns null             no exception
PASS T6   duplicate pass id is rejected                     IOException: pass id already exists

=== damaged data: five bad lines are put between good ones ===
PASS T7   DAMAGED lines do not stop the search              P2000 found after 5 bad lines
     damaged lines reported by the scan:
       line 5: expected 8 fields but found 3
       line 6: not a number: For input string: "abc"
       line 7: empty line
       line 8: checksum mismatch (line changed or cut off)
     (find() stops at P2000, so the cut-off last line was not reached yet; now a FULL scan:)
       line 5: expected 8 fields but found 3
       line 6: not a number: For input string: "abc"
       line 7: empty line
       line 8: checksum mismatch (line changed or cut off)
       line 10: checksum mismatch (line changed or cut off)
PASS T8   full scan reports exactly the 5 damaged lines     5 reported, the last one is the cut-off
line 10
PASS T9   good records before the damage are still found    P1001 found
PASS T10  add after a cut-off last line does not glue records  newline repaired before appending

Result: 11 passed, 0 failed
```

Test	Case	What it verifies
T0	find() on a missing file	A missing file is an empty store, not an exception (A1.3).
T1	add four passes, find P1002	Normal operation: record written and read back with all fields equal.
T2	name “Kumar, Meena”	Delimiter inside a field is escaped on writing and restored on reading (A1.1); the stored line shows Kumar\, Meena.
T3	name with a backslash	The escape character itself is escaped correctly.
T4	list route 12	listByRoute returns exactly the two passes of that route.
T5	find(“P9999”)	An id that does not exist returns null, with no exception.
T6	duplicate pass id	add() rejects a second pass with an existing id.
T7	five bad lines, then a good one	Damaged lines do not stop the search: P2000 is still found after them.
T8	full scan after the damage	Exactly five damaged lines are reported with line number and reason: wrong field count, bad number (checksum correct but amount not numeric), empty line, changed text (checksum mismatch), cut-off last line.

Test	Case	What it verifies
T9	good record before the damage	Earlier records are not affected by later damage.
T10	add after a cut-off last line	The missing line break is repaired first, so the new record is not glued to the broken one (A1.3).

Table B1.1 – What each test verifies. Result: 11 passed, 0 failed.

One test of my own first failed because of a flaw in the test, not in the store: find() stops at the first match, so the cut-off last line after P2000 had not been scanned when I counted the damaged lines. T8 now uses a full scan (listByRoute) to count all damaged lines; this is a property of the design worth stating: the damaged-line report covers only the part of the file that was scanned.

B1.2 – Compare with the Part A pseudocode (1 mark)

Part A pseudocode	What I had to change in the implementation	Why
Parse: “split at unescaped commas”	Wrote a small tokenizer (BusPass.split) instead of String.split(",")	String.split cannot tell an escaped comma from a delimiter; with split a name such as “Kumar\, Meena” would give nine fields.
Order of checks in Parse()	Field count → CRC → number / date conversion	The CRC is computed on the text before the last unescaped comma, so the field count has to be known first. A consequence: a hand-edited bad amount is reported as “checksum mismatch” unless the checksum was recomputed (test T8 uses a line with a correct checksum to reach the “not a number” message).
AddPass step 2: duplicate check	Added a second method append() without the check	The check is an O(n) scan, so loading n records with add() would cost $O(n^2)$; benchmarks and the migration load with append().
AddPass step 5: repair the missing line break	Implemented with RandomAccessFile (seek to the last byte)	readLine() of BufferedReader cannot tell whether the final line ended with a line break; checking the last byte is O(1).
FindPass: report damaged lines	The report is a list in the store object, cleared at the start of every scan	Because find() returns early, only the scanned part of the file is reported (see B1.1).
“PassStore” of the assignment (5 fields, String.split, no checksum)	Kept exactly as given for the Complete-the-Code task; the full 8-field design is BusPassStore	The Complete-the-Code class is a simplified version of the store: it has no escaping and no checksum, so a name with a comma would be lost (shown in Task B2).

Table B1.2 – Differences between design and implementation

B1.3 – Measure the effect of buffering (2 marks)

BufferingBenchmark counts the lines of attendance logs of 1, 10 and 50 MB (33-byte lines) with five methods. “Read calls” counts the calls that reach the FileInputStream, which for a FileInputStream is one operating-system read each. Times are the best of 5 runs (best of 3 for the unbuffered 1 and 10 MB, a single run for the unbuffered 50 MB).

Size (MB)	Method	Time (ms)	Read calls	File bytes
1	unbuffered read()	286.6	1,048,576	1,048,575
1	BufferedInputStream	20.3	129	1,048,575
1	byte[64 KB] buffer	0.9	17	1,048,575
1	BufferedReader	2.5	(not counted)	1,048,575
1	Files.readAllLines	2.4	(not counted)	1,048,575
10	unbuffered read()	2885.8	10,485,751	10,485,750
10	BufferedInputStream	221.1	1,281	10,485,750
10	byte[64 KB] buffer	3.6	161	10,485,750
10	BufferedReader	20.5	(not counted)	10,485,750
10	Files.readAllLines	25.4	(not counted)	10,485,750
50	unbuffered read()	16214.8	52,428,784	52,428,783
50	BufferedInputStream	1067.7	6,401	52,428,783
50	byte[64 KB] buffer	21.4	801	52,428,783

Size (MB)	Method	Time (ms)	Read calls	File bytes
50	BufferedReader	64.5	(not counted)	52,428,783
50	Files.readAllLines	123.7	(not counted)	52,428,783

Table B1.3 – Line counting with and without buffering (from TaskB1/buffering_results.csv)

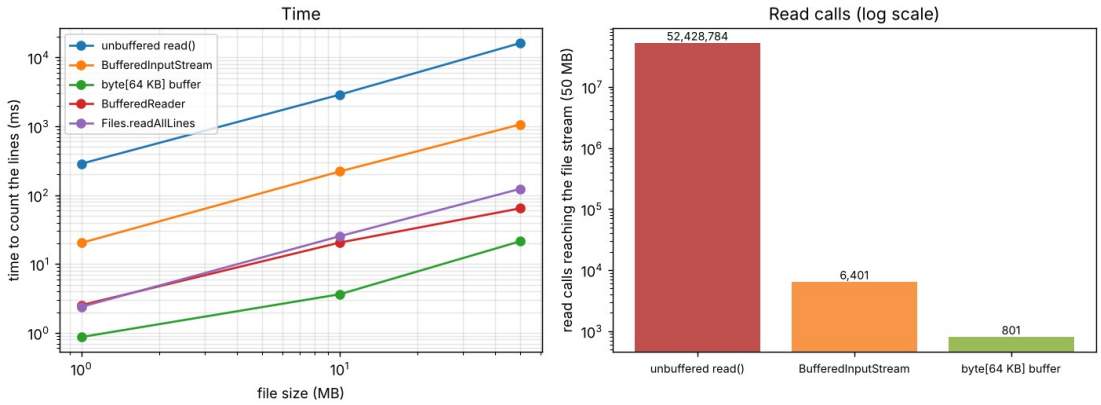


Figure B1.1 – Time against file size (left) and read calls for the 50 MB file (right)

Output of java BufferingBenchmark

```
=== B1.3 Count the lines of an attendance log (time = best of the runs, in milliseconds) ===
   size        bytes method                    time (ms)       read calls      lines
    1 MB    1,048,575 unbuffered read()             286.6        1,048,576     31,775
    1 MB    1,048,575 BufferedInputStream            20.3              129     31,775
    1 MB    1,048,575 byte[64 KB] buffer              0.9               17     31,775
    1 MB    1,048,575 BufferedReader                  2.5    (not counted)     31,775
    1 MB    1,048,575 Files.readAllLines              2.4    (not counted)     31,775
   10 MB   10,485,750 unbuffered read()           2885.8       10,485,751    317,750
   10 MB   10,485,750 BufferedInputStream          221.1            1,281    317,750
   10 MB   10,485,750 byte[64 KB] buffer             3.6              161    317,750
   10 MB   10,485,750 BufferedReader                20.5    (not counted)    317,750
   10 MB   10,485,750 Files.readAllLines            25.4    (not counted)    317,750
   50 MB   52,428,783 unbuffered read()          16214.8       52,428,784  1,588,751
   50 MB   52,428,783 BufferedInputStream         1067.7            6,401  1,588,751
   50 MB   52,428,783 byte[64 KB] buffer            21.4              801  1,588,751
   50 MB   52,428,783 BufferedReader                64.5    (not counted)  1,588,751
   50 MB   52,428,783 Files.readAllLines           123.7    (not counted)  1,588,751

=== A2.3 When buffering gives little or no benefit (10 MB file, best of 5) ===
(a) reads of 64 KB blocks: FileInputStream 0.8 ms (161 calls)  |  wrapped in BufferedInputStream 0.9 ms
(161 calls)
(b) data already in memory (ByteArrayInputStream, byte by byte): plain 255.5 ms  |  buffered 210.8 ms
(checksum 0)
```

Comparison with the prediction of A2.1. (1) Number of calls: predicted file size + 1; measured for the unbuffered counter: 1 MB: 1,048,576 = 1,048,575 + 1; 10 MB: 10,485,751 = 10,485,750 + 1; 50 MB: 52,428,784 = 52,428,783 + 1 – the prediction was exact. (2) Time: predicted about 10 s for 50 MB; measured 16.2 s (309 ns per call): the right order of magnitude. (3) Effect of buffering: with a BufferedInputStream the calls fell by a factor of 8,191 and the time from 16.2 s to 1.07 s (15 times faster); with a 64 KB array the calls fell by 65,454 and the time to 21.4 ms (757 times faster). The time grows linearly with the file size in every method (O(file size)); buffering changes the constant, not the complexity. What surprised me: the BufferedInputStream with a one-byte read() is still 50 times slower than the own byte[] buffer, because 52 million method calls remain.

B1.4 – Measure how the cost grows (2 marks)

GrowthBenchmark creates stores of 1,000 to 100,000 passes and measures one find() for the last pass (worst case that exists), for a pass in the middle, and for an id that is not in the file (the scan

must read everything). It also measures the fixed-length RandomAccessFile store and a HashMap loaded once. Averages of 10–30 runs after a warm-up.

n	Text file bytes	Fixed file bytes	scan: last (ms)	scan: middle (ms)	scan: missing (ms)	fixed seek (µs)
1,000	71,205	92,000	1.83	1.21	2.71	3.75
5,000	356,004	460,000	6.71	3.03	6.04	3.02
10,000	712,984	920,000	14.03	7.07	14.32	1.27
50,000	3,601,000	4,600,000	61.88	34.30	66.02	3.22
100,000	7,211,928	9,200,000	103.07	54.84	111.31	1.52

Table B1.4 – Time of one find() as the file grows (from TaskB1/growth_results.csv)

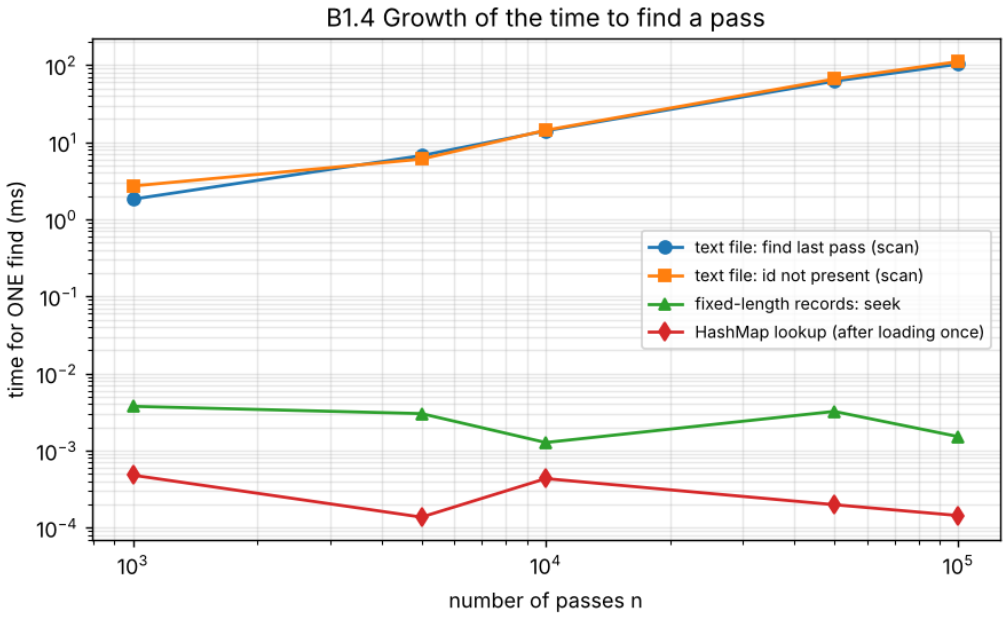


Figure B1.2 – Growth of the time to find one pass (logarithmic axes)

Interpretation. The scan time grows with n: from n = 10,000 to 100,000 (×10) the time to find the last pass grew from 14.03 ms to 103.07 ms (×7.3), and the middle pass is found in about half the time (the scan stops halfway) – this is O(n), as predicted in A1. Between 1,000 and 100,000 the time grew ×56 for ×100 records: less than 100 because the small runs include fixed costs (opening the file, JIT warm-up) that do not grow with n. The "missing" id costs about as much as the last pass because both read the whole file. The fixed-length store needs 3.7–3.7 µs for every n (O(1)). For the stated need (8,000 passes a year) a scan takes about 14.0 ms per search at 10,000 passes, which a clerk does not notice; the trend shows when that stops being true.

Complete the Code – PassStore.find (Task B1)

TODO 1 reads the file line by line with a BufferedReader inside try-with-resources, skips damaged lines (not exactly five fields) and returns the fields of the matching pass. Everything else of the program is as given.

TaskB1/PassStore.java - TODO 1 completed

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.Arrays;

public class PassStore {
```

```
    private final Path file;

    PassStore(Path file) { this.file = file; }

    public void add(String passId, String regNo, String name, int route, double amount) throws
IOException {
        try (BufferedWriter w = Files.newBufferedWriter(file, StandardOpenOption.CREATE,
StandardOpenOption.APPEND)) {
            w.write(String.join(",", passId, regNo, name, String.valueOf(route),
String.valueOf(amount)));
            w.newLine();
        }
    }

    /** Returns the five fields of the pass with the given ID, or null if not found. */
    public String[] find(String passId) throws IOException {
        // TODO 1 (completed): read the file line by line with a BufferedReader inside try-with-
resources;
        // skip damaged lines (not exactly 5 fields); return the fields of the matching pass
        try (BufferedReader r = Files.newBufferedReader(file)) {
            String line;
            while ((line = r.readLine()) != null) {
                String[] f = line.split(",", -1);          // -1 keeps empty trailing fields, so
"a,b," has 3 fields
                if (f.length != 5) continue;               // damaged line: skip it, do not stop
the search
                if (f[0].equals(passId)) return f;
            }
        }
        return null;
    }

    public static void main(String[] args) throws IOException {
        Path f = Files.createTempFile("passes", ".csv");
        PassStore store = new PassStore(f);
        store.add("P1001", "192311001", "Arun", 12, 4500);
        Files.writeString(f, "P1002,DAMAGED LINE\n", StandardOpenOption.APPEND);
        store.add("P1003", "192311003", "Divya", 7, 3800);
        System.out.println(Arrays.toString(store.find("P1003")));
        System.out.println(Arrays.toString(store.find("P1002")));
        Files.delete(f);
    }
}
```

Output of java PassStore (expected: the five fields of P1003, then null)

```
[P1003, 192311003, Divya, 7, 3800.0]
null
```

The output equals the expected output. P1003 is found although a damaged line (P1002,DAMAGED LINE: two fields) comes before it, because a damaged line is skipped with continue and the loop goes on. find("P1002") returns null: the line with that id is damaged and therefore not a valid pass; the program does not guess. split(",", -1) is used so that trailing empty fields are kept ("a,b," has three fields, not two), otherwise a line with a missing last field could be mistaken for a valid shorter line.

Alternative Concept and Complexity Analysis – HashMap loaded once

```
ALGORITHM SearchManyWithHashMap
INPUT   : the pass file, q pass ids to look up
BEGIN
  1.  SET map <- empty HashMap from passId to pass
  2.  Scan the file ONCE and put every valid pass into the map                  // O(n) time, O(n)
memory
  3.  FOR EACH id IN the q ids DO  SET pass <- map.get(id)  ENDFOR             // O(1) each
END
// the scanning version calls FindPass for every id: q scans of n lines = O(q * n)
```

q = 100 searches	n	Scan the file for every search (ms)	HashMap: load once + 100 lookups (ms)	Speed-up
100	1,000	118.0	3.6	32.9×
100	10,000	722.5	13.6	53.2×
100	100,000	4819.8	227.4	21.2×

Table B1.5 – q searches on n passes (from TaskB1/search_many_results.csv)

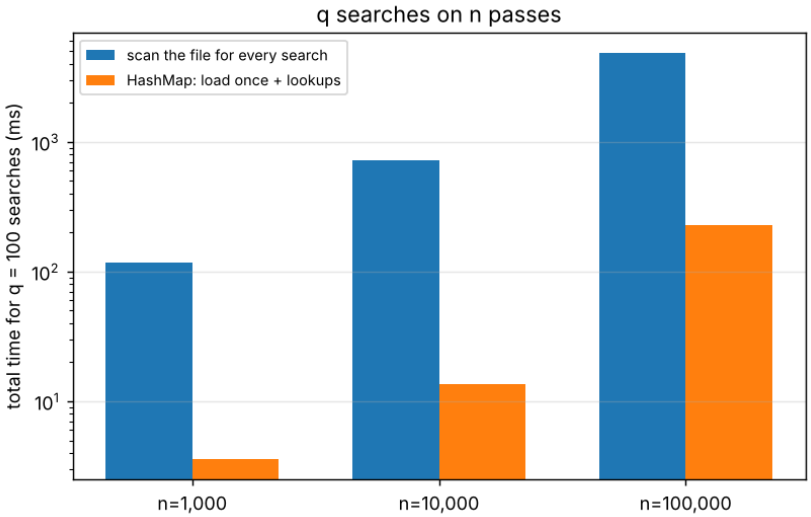


Figure B1.3 – Total time of 100 searches

n	HashMap load (ms)	HashMap lookup (ns)	Heap used by the map (MB)	Heap used by the scan
5,000	8.8	137.0	1.5	O(1): one 8 KB buffer
10,000	8.7	433.1	1.5	O(1): one 8 KB buffer
50,000	143.1	198.6	15.4	O(1): one 8 KB buffer
100,000	203.6	143.5	30.8	O(1): one 8 KB buffer

Table B1.6 – Cost of the HashMap alternative (the heap figure for n = 1,000 is lost in garbage-collection noise and is omitted)

Analysis. Scanning for every search costs O(q·n); the HashMap costs O(n + q): one scan plus q constant-time lookups. For n = 100,000 and q = 100 the total fell from 4819.8 ms to 227.4 ms (21× faster), and the gain grows with q. The price is memory: O(n) – about 30.8 MB for 100,000 passes against a single 8 KB buffer for the scan – and the map is a snapshot: passes added later by another program are not seen until the file is read again. For one search the scan is cheaper (the map needs the whole file to be read first); the break-even is at about q = 1 here. **Evaluation:** a long-running program that answers many searches should load the file into a map (or use a database); a program that does one search and exits should scan.

Source code of Task B1

TaskB1/DamagedRecordException.java

```
/** A line of the pass file could not be accepted (wrong field count, bad number, bad date,
checksum mismatch ...). */
public class DamagedRecordException extends Exception {
    public DamagedRecordException(String message) { super(message); }
}
```

TaskB1/BusPass.java - record, escaping, CRC, parsing

```
import java.io.Serializable;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.util.zip.CRC32;
```

```
/**
 * One bus-pass record (Task A1).  Text line layout (8 comma-separated fields):
 *   passId , regNo , name , route , stop , validity , amount , crc32
 * A comma, a backslash or a line break INSIDE a field is written as  \,  \\  or  \n  (backslash
escape).
 * The last field is the CRC-32 of the first seven fields, so a damaged or half-written line is
detected.
 */
public class BusPass implements Serializable {
    private static final long serialVersionUID = 1L;
    public final String passId, regNo, name, stop;
    public final int route;
    public final LocalDate validity;
    public final double amount;

    public BusPass(String passId, String regNo, String name, int route, String stop, LocalDate
validity, double amount) {
        this.passId = passId; this.regNo = regNo; this.name = name; this.route = route;
        this.stop = stop; this.validity = validity; this.amount = amount;
    }

    // ---------- writing ----------
    static String escape(String s) { return s.replace("\\", "\\\\").replace(",", "\\,").replace("\
n", "\\n").replace("\r", ""); }

    private String body() {
        return String.join(",", escape(passId), escape(regNo), escape(name),
String.valueOf(route), escape(stop), validity.toString(), String.format("%.2f", amount));
    }

    public String toLine() {
        String b = body();
        CRC32 crc = new CRC32(); crc.update(b.getBytes(java.nio.charset.StandardCharsets.UTF_8));
        return b + "," + Long.toHexString(crc.getValue());
    }

    // ---------- reading ----------
    /** Splits a line at the commas that are NOT escaped, and removes the escapes. */
    static String[] split(String line) {
        java.util.List<String> parts = new java.util.ArrayList<>();
        StringBuilder cur = new StringBuilder();
        for (int i = 0; i < line.length(); i++) {
            char ch = line.charAt(i);
            if (ch == '\\' && i + 1 < line.length()) {
                char next = line.charAt(++i);
                cur.append(next == 'n' ? '\n' : next);
            } else if (ch == ',') { parts.add(cur.toString()); cur.setLength(0); }
            else cur.append(ch);
        }
        parts.add(cur.toString());
        return parts.toArray(new String[0]);
    }

    public static BusPass fromLine(String line) throws DamagedRecordException {
        if (line == null || line.isBlank()) throw new DamagedRecordException("empty line");
        String[] f = split(line);
        if (f.length != 8) throw new DamagedRecordException("expected 8 fields but found " +
f.length);
        try {
            int lastComma = lastUnescapedComma(line);
            CRC32 crc = new CRC32(); crc.update(line.substring(0,
lastComma).getBytes(java.nio.charset.StandardCharsets.UTF_8));
            if (!Long.toHexString(crc.getValue()).equals(f[7])) throw new
DamagedRecordException("checksum mismatch (line changed or cut off)");
```

```
            if (f[0].isBlank()) throw new DamagedRecordException("pass id is empty");
            return new BusPass(f[0], f[1], f[2], Integer.parseInt(f[3]), f[4],
LocalDate.parse(f[5]), parseAmount(f[6]));
        } catch (NumberFormatException e) { throw new DamagedRecordException("not a number: " +
e.getMessage()); }
        catch (DateTimeParseException e) { throw new DamagedRecordException("bad validity date '"
+ f[5] + "'"); }
    }

    private static double parseAmount(String s) throws DamagedRecordException {
        double a = Double.parseDouble(s);
        if (Double.isNaN(a) || a <= 0) throw new DamagedRecordException("amount must be positive:
" + s);
        return a;
    }

    private static int lastUnescapedComma(String line) {
        int last = -1;
        for (int i = 0; i < line.length(); i++) { char ch = line.charAt(i); if (ch == '\\') i++;
else if (ch == ',') last = i; }
        return last;
    }

    @Override public String toString() { return passId + " " + regNo + " " + name + " route " +
route + " " + stop + " valid " + validity + " Rs." + amount; }
}
```

TaskB1/BusPassStore.java - add / find / listByRoute

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.ArrayList;
import java.util.List;

/**
 * The text-file store designed in Task A1.  Operations: add a pass, find a pass by ID, list the
passes of a route.
 * Streams: BufferedWriter (append, one write per record) and BufferedReader (line by line).
Damaged lines are skipped and reported.
 */
public class BusPassStore {
    private final Path file;
    /** One entry per damaged line met during the LAST scan: "line 7: expected 8 fields but found
2". */
    public final List<String> damaged = new ArrayList<>();

    public BusPassStore(Path file) { this.file = file; }

    /** Adds a pass at the end of the file (rejects a duplicate pass id). */
    public void add(BusPass p) throws IOException {
        if (find(p.passId) != null) throw new IOException("pass id already exists: " + p.passId);
        append(p);
    }

    /** Append without the duplicate scan (used for bulk loading, O(1) per record). */
    public void append(BusPass p) throws IOException {
        repairMissingNewline();
        try (BufferedWriter w = Files.newBufferedWriter(file, StandardCharsets.UTF_8,
StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
            w.write(p.toLine() + "\n");                         // the whole record in ONE write:
a power cut can only cut the last line
        }
    }
```

```
    /** If the last write was cut off (no line break at the end), start a new line so that the
next record is not glued to the broken one. */
    private void repairMissingNewline() throws IOException {
        if (!Files.exists(file) || Files.size(file) == 0) return;
        try (RandomAccessFile raf = new RandomAccessFile(file.toFile(), "rw")) {
            raf.seek(raf.length() - 1);
            if (raf.read() != '\n') { raf.seek(raf.length()); raf.write('\n'); }
        }
    }

    /** Linear scan, O(n). Returns null when the id is not found OR the file does not exist. */
    public BusPass find(String passId) throws IOException {
        BusPass[] found = new BusPass[1];
        scan(p -> { if (p.passId.equals(passId)) { found[0] = p; return false; } return true; });
        return found[0];
    }

    public List<BusPass> listByRoute(int route) throws IOException {
        List<BusPass> out = new ArrayList<>();
        scan(p -> { if (p.route == route) out.add(p); return true; });
        return out;
    }

    public interface Visitor { boolean visit(BusPass p); }       // return false to stop the scan

    public void scan(Visitor v) throws IOException {
        damaged.clear();
        if (!Files.exists(file)) return;                          // missing file = empty store
(not a crash)
        try (BufferedReader r = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
            String line; int n = 0;
            while ((line = r.readLine()) != null) {
                n++;
                try { if (!v.visit(BusPass.fromLine(line))) return; }
                catch (DamagedRecordException e) { damaged.add("line " + n + ": " +
e.getMessage()); }   // skip and remember
            }
        }
    }
}
```

TaskB1/LineCounter.java - the counters of A2

```
import java.io.*;
import java.nio.file.*;

/** The line counters of Task A2: from one-byte-at-a-time to buffered, plus the "read everything"
alternative. */
public class LineCounter {

    /** Counts the read calls that reach the stream below it (for FileInputStream every call is
one operating-system read). */
    public static class CountingInputStream extends FilterInputStream {
        public long calls = 0;
        public CountingInputStream(InputStream in) { super(in); }
        @Override public int read() throws IOException { calls++; return super.read(); }
        @Override public int read(byte[] b, int off, int len) throws IOException { calls++; return
in.read(b, off, len); }
    }

    /** (1) ORIGINAL: FileInputStream.read() one byte at a time: one operating-system call per
byte. */
    public static long unbuffered(Path file, long[] readCalls) throws IOException {
        long lines = 0;
        try (CountingInputStream in = new CountingInputStream(new FileInputStream(file.toFile())))
{
```

```
            int b;
            while ((b = in.read()) != -1) if (b == '\n') lines++;
            readCalls[0] = in.calls;
        }
        return lines;
    }

    /** (2) BufferedInputStream: the program still calls read() per byte, but the stream below is
read 8,192 bytes at a time. */
    public static long bufferedStream(Path file, long[] readCalls) throws IOException {
        long lines = 0;
        try (CountingInputStream raw = new CountingInputStream(new
FileInputStream(file.toFile()));
             BufferedInputStream in = new BufferedInputStream(raw)) {
            int b;
            while ((b = in.read()) != -1) if (b == '\n') lines++;
            readCalls[0] = raw.calls;
        }
        return lines;
    }

    /** (3) Own byte[] buffer: read(byte[]) fills 64 KB at a time. */
    public static long byteArray(Path file, int bufSize, long[] readCalls) throws IOException {
        long lines = 0;
        byte[] buf = new byte[bufSize];
        try (CountingInputStream in = new CountingInputStream(new FileInputStream(file.toFile())))
{
            int n;
            while ((n = in.read(buf, 0, buf.length)) != -1) for (int i = 0; i < n; i++) if (buf[i]
== '\n') lines++;
            readCalls[0] = in.calls;
        }
        return lines;
    }

    /** (4) BufferedReader.readLine(): reads characters, builds a String per line. */
    public static long bufferedReader(Path file) throws IOException {
        long lines = 0;
        try (BufferedReader r = Files.newBufferedReader(file)) { while (r.readLine() != null)
lines++; }
        return lines;
    }

    /** (5) ALTERNATIVE: Files.readAllLines(): the whole file as a List<String> in memory, O(file
size). */
    public static long readAllLines(Path file) throws IOException { return
Files.readAllLines(file).size(); }
}
```

TaskB1/FixedRecordStore.java - the RandomAccessFile alternative

```
import java.io.IOException;
import java.io.RandomAccessFile;
import java.nio.charset.StandardCharsets;
import java.nio.file.Path;
import java.time.LocalDate;

/**
 * ALTERNATIVE to the text file (Task A1): fixed-length records in a RandomAccessFile.
 * Record i starts at byte i * RECORD_SIZE, so a pass is found with ONE seek: O(1).  Pass ids must
be dense: P1001 -> index 0, P1002 -> index 1 ...
 * Record layout (92 bytes): passId 8 | regNo 12 | name 30 | route int 4 | stop 20 | validity 10 |
amount double 8
 */
public class FixedRecordStore implements AutoCloseable {
    public static final int RECORD_SIZE = 8 + 12 + 30 + 4 + 20 + 10 + 8;
```

```
    public static final int FIRST_ID = 1001;
    private final RandomAccessFile raf;

    public FixedRecordStore(Path file) throws IOException { raf = new
RandomAccessFile(file.toFile(), "rw"); }

    static int indexOf(String passId) { return Integer.parseInt(passId.substring(1)) - FIRST_ID; }

    private static void putText(byte[] rec, int off, int width, String s) {
        byte[] b = s.getBytes(StandardCharsets.UTF_8);
        int n = Math.min(b.length, width);
        System.arraycopy(b, 0, rec, off, n);
        for (int i = n; i < width; i++) rec[off + i] = ' ';                // padding = the wasted
space
    }
    private static String getText(byte[] rec, int off, int width) { return new String(rec, off,
width, StandardCharsets.UTF_8).stripTrailing(); }

    public void put(BusPass p) throws IOException {
        byte[] rec = new byte[RECORD_SIZE];
        putText(rec, 0, 8, p.passId); putText(rec, 8, 12, p.regNo); putText(rec, 20, 30, p.name);
        java.nio.ByteBuffer bb = java.nio.ByteBuffer.wrap(rec);
        bb.position(50); bb.putInt(p.route);
        putText(rec, 54, 20, p.stop); putText(rec, 74, 10, p.validity.toString());
        bb.position(84); bb.putDouble(p.amount);
        raf.seek((long) indexOf(p.passId) * RECORD_SIZE);
        raf.write(rec);
    }

    /** O(1): one seek and one read of RECORD_SIZE bytes. */
    public BusPass find(String passId) throws IOException {
        long pos = (long) indexOf(passId) * RECORD_SIZE;
        if (pos < 0 || pos + RECORD_SIZE > raf.length()) return null;
        byte[] rec = new byte[RECORD_SIZE];
        raf.seek(pos);
        raf.readFully(rec);
        if (!getText(rec, 0, 8).equals(passId)) return null;               // empty slot or other
record
        java.nio.ByteBuffer bb = java.nio.ByteBuffer.wrap(rec);
        return new BusPass(getText(rec, 0, 8), getText(rec, 8, 12), getText(rec, 20, 30),
bb.getInt(50), getText(rec, 54, 20),
                LocalDate.parse(getText(rec, 74, 10)), bb.getDouble(84));
    }

    @Override public void close() throws IOException { raf.close(); }
}
```

TaskB1/BusPassStoreTest.java

```
import java.io.IOException;
import java.nio.file.*;
import java.time.LocalDate;
import java.util.List;

/** Task B1.1 - tests of the store: normal cases, an id that does not exist, and damaged data. */
public class BusPassStoreTest {
    static int pass = 0, fail = 0;
    static void check(String id, String what, boolean ok, String detail) {
        if (ok) pass++; else fail++;
        System.out.printf("%-4s %-4s %-52s %s%n", ok ? "PASS" : "FAIL", id, what, detail);
    }
    static BusPass p(String id, String reg, String name, int route) { return new BusPass(id, reg,
name, route, "Main Gate", LocalDate.of(2027, 6, 30), 4500); }

    public static void main(String[] args) throws Exception {
        Path f = Files.createTempFile("buspass", ".txt");
```

```
        Files.delete(f);                                                    // start with a
MISSING file
        BusPassStore store = new BusPassStore(f);

        System.out.println("=== B1.1 Test table (what each test verifies is in the text of the
report) ===");
        check("T0", "find() on a missing file", store.find("P1001") == null, "returns null, no
exception: a missing file = empty store");
        store.add(p("P1001", "192311001", "Arun", 12));
        store.add(p("P1002", "192311002", "Divya", 7));
        store.add(p("P1003", "192311003", "Kumar, Meena", 12));                // name contains
the delimiter
        store.add(p("P1004", "192311004", "A\\B", 9));                         // name contains
the escape character
        BusPass x = store.find("P1002");
        check("T1", "NORMAL: add 4 passes, find P1002", x != null && x.name.equals("Divya") &&
x.route == 7, String.valueOf(x));
        x = store.find("P1003");
        check("T2", "NORMAL: name with a comma survives", x != null && x.name.equals("Kumar,
Meena"), "stored line: " + Files.readAllLines(f).get(2));
        x = store.find("P1004");
        check("T3", "NORMAL: name with a backslash survives", x != null && x.name.equals("A\\B"),
String.valueOf(x));
        List<BusPass> r = store.listByRoute(12);
        check("T4", "NORMAL: list route 12 gives 2 passes", r.size() == 2, r.get(0).passId + ", "
+ r.get(1).passId);
        check("T5", "NOT FOUND: find(\"P9999\") returns null", store.find("P9999") == null, "no
exception");
        boolean dup = false; try { store.add(p("P1001", "192311009", "Other", 1)); } catch
(IOException e) { dup = true; }
        check("T6", "duplicate pass id is rejected", dup, "IOException: pass id already exists");

        System.out.println();
        System.out.println("=== damaged data: five bad lines are put between good ones ===");
        String good = p("P2000", "192312000", "Good", 5).toLine();
        String badAmountBody = "P1501,192311501,Ravi,3,Gate,2027-06-30,abc";
        java.util.zip.CRC32 crc = new java.util.zip.CRC32(); crc.update(badAmountBody.getBytes());
        Files.writeString(f, "P1500,192311500,Short\n"                                  // wrong
number of fields
                + badAmountBody + "," + Long.toHexString(crc.getValue()) + "\n"         // non-
numeric amount (checksum is correct)
                + "\n"                                                                  // empty
line
                + good.replace("Good", "Goop") + "\n"                                   //
changed text: checksum no longer matches
                + good + "\n"                                                           // a
good line after the damage
                + good.substring(0, good.length() - 7),                                 // last
line cut off (no line break): power failure
                StandardOpenOption.APPEND);
        x = store.find("P2000");
        check("T7", "DAMAGED lines do not stop the search", x != null && x.name.equals("Good"),
"P2000 found after 5 bad lines");
        System.out.println("     damaged lines reported by the scan:");
        for (String d : store.damaged) System.out.println("       " + d);
        System.out.println("     (find() stops at P2000, so the cut-off last line was not reached
yet; now a FULL scan:)");
        store.listByRoute(5);
        for (String d : store.damaged) System.out.println("       " + d);
        check("T8", "full scan reports exactly the 5 damaged lines", store.damaged.size() == 5 &&
store.damaged.get(4).startsWith("line 10"), store.damaged.size() + " reported, the last one is the
cut-off line 10");
        check("T9", "good records before the damage are still found", store.find("P1001") != null,
"P1001 found");
```

```
        store.add(p("P3000", "192313000", "After", 1));                                  // add
after a cut-off last line
        check("T10", "add after a cut-off last line does not glue records", store.find("P3000") !=
null, "newline repaired before appending");

        System.out.println();
        System.out.println("Result: " + pass + " passed, " + fail + " failed");
        Files.delete(f);
    }
}
```

TaskB1/BufferingBenchmark.java

```
import java.io.*;
import java.nio.file.*;

/** Tasks A2 / B1.3: how many read calls and how much time does each line counter need for 1, 10
and 50 MB? */
public class BufferingBenchmark {
    static final int LINE = 33;                                                  // bytes per
generated log line
    static Path makeLog(long megabytes) throws IOException {
        Path f = Files.createTempFile("attendance" + megabytes + "MB", ".log");
        long lines = megabytes * 1024 * 1024 / LINE;
        try (BufferedWriter w = Files.newBufferedWriter(f)) {
            for (long i = 0; i < lines; i++) w.write(String.format("2026-11-09,%09d,IN,08:45:12\
n", 192311000L + (i % 9000)));
        }
        return f;
    }

    interface Counter { long run(Path f, long[] calls) throws IOException; }

    public static void main(String[] args) throws Exception {
        long[] sizes = {1, 10, 50};
        String[] names = {"unbuffered read()", "BufferedInputStream", "byte[64 KB] buffer",
"BufferedReader", "Files.readAllLines"};
        Counter[] cs = {
            LineCounter::unbuffered, LineCounter::bufferedStream, (f, c) ->
LineCounter.byteArray(f, 65536, c),
            (f, c) -> { c[0] = -1; return LineCounter.bufferedReader(f); }, (f, c) -> { c[0] = -1;
return LineCounter.readAllLines(f); }};
        PrintWriter csv = new PrintWriter("buffering_results.csv");
        csv.println("size_MB,file_bytes,method,time_ms,read_calls,lines");
        System.out.println("=== B1.3 Count the lines of an attendance log (time = best of the
runs, in milliseconds) ===");
        System.out.printf("%7s %12s %-22s %12s %16s %10s%n", "size", "bytes", "method", "time
(ms)", "read calls", "lines");
        for (int w = 0; w < 2; w++) { Path t = makeLog(1); for (Counter c : cs) c.run(t, new
long[1]); Files.delete(t); }     // warm-up
        for (long mb : sizes) {
            Path f = makeLog(mb);
            long bytes = Files.size(f);
            for (int m = 0; m < cs.length; m++) {
                int runs = (m == 0) ? (mb >= 50 ? 1 : 3) : 5;
                double best = Double.MAX_VALUE; long lines = 0; long[] calls = new long[1];
                for (int r = 0; r < runs; r++) {
                    long t0 = System.nanoTime();
                    lines = cs[m].run(f, calls);
                    best = Math.min(best, (System.nanoTime() - t0) / 1e6);
                }
                System.out.printf("%5d MB %,12d %-22s %12.1f %16s %,10d%n", mb, bytes, names[m],
best, calls[0] < 0 ? "(not counted)" : String.format("%,d", calls[0]), lines);
                csv.printf("%d,%d,%s,%.2f,%d,%d%n", mb, bytes, names[m], best, calls[0], lines);
            }
            Files.delete(f);
```

```
        }
        csv.close();

        // ---------------- A2.3: when does a buffer not help? ----------------
        System.out.println();
        System.out.println("=== A2.3 When buffering gives little or no benefit (10 MB file, best
of 5) ===");
        Path f = makeLog(10);
        double direct = Double.MAX_VALUE, wrapped = Double.MAX_VALUE;
        long[] c1 = new long[1], c2 = new long[1];
        byte[] big = new byte[65536];
        for (int r = 0; r < 5; r++) {
            long t0 = System.nanoTime();
            try (LineCounter.CountingInputStream in = new LineCounter.CountingInputStream(new
FileInputStream(f.toFile()))) { while (in.read(big, 0, big.length) != -1) { } c1[0] = in.calls; }
            direct = Math.min(direct, (System.nanoTime() - t0) / 1e6);
            t0 = System.nanoTime();
            try (LineCounter.CountingInputStream raw = new LineCounter.CountingInputStream(new
FileInputStream(f.toFile())); BufferedInputStream in = new BufferedInputStream(raw)) { while
(in.read(big, 0, big.length) != -1) { } c2[0] = raw.calls; }
            wrapped = Math.min(wrapped, (System.nanoTime() - t0) / 1e6);
        }
        System.out.printf("(a) reads of 64 KB blocks: FileInputStream %.1f ms (%,d calls)  |
wrapped in BufferedInputStream %.1f ms (%,d calls)%n", direct, c1[0], wrapped, c2[0]);
        byte[] mem = new byte[10 * 1024 * 1024];
        double plain = Double.MAX_VALUE, buf = Double.MAX_VALUE; long sink = 0;
        for (int r = 0; r < 5; r++) {
            long t0 = System.nanoTime();
            try (InputStream in = new ByteArrayInputStream(mem)) { int b; while ((b = in.read()) !
= -1) sink += b; }
            plain = Math.min(plain, (System.nanoTime() - t0) / 1e6);
            t0 = System.nanoTime();
            try (InputStream in = new BufferedInputStream(new ByteArrayInputStream(mem))) { int b;
while ((b = in.read()) != -1) sink += b; }
            buf = Math.min(buf, (System.nanoTime() - t0) / 1e6);
        }
        System.out.printf("(b) data already in memory (ByteArrayInputStream, byte by byte): plain
%.1f ms  |  buffered %.1f ms%n", plain, buf);
        Files.delete(f);
        System.out.println("(checksum " + sink + ")");
    }
}
```

TaskB1/GrowthBenchmark.java

```
import java.io.*;
import java.nio.file.*;
import java.time.LocalDate;
import java.util.*;

/** Task B1.4 and the alternatives of A1 / B1: find a pass as the file grows; HashMap loaded once;
fixed-length records. */
public class GrowthBenchmark {
    static long sink = 0;
    static BusPass make(int i) { return new BusPass("P" + (1001 + i), String.format("19%07d", i),
"Student Name " + (i % 1000), 1 + i % 40, "Stop " + (i % 25), LocalDate.of(2027, 6, 30), 3000 + i
% 2000); }
    static long used() { for (int i = 0; i < 3; i++) System.gc(); Runtime r =
Runtime.getRuntime(); return r.totalMemory() - r.freeMemory(); }

    public static void main(String[] args) throws Exception {
        int[] ns = {1000, 5000, 10000, 50000, 100000};
        PrintWriter csv = new PrintWriter("growth_results.csv");

csv.println("n,text_bytes,fixed_bytes,scan_last_ms,scan_middle_ms,scan_missing_ms,fixed_seek_us,ma
p_load_ms,map_lookup_ns,map_heap_MB");
```

```
        System.out.println("=== B1.4 Time to find ONE pass as the file grows (average of repeated
runs) ===");
        System.out.printf("%8s %12s %12s %14s %16s %14s %14s%n", "n", "text bytes", "fixed bytes",
"scan: last (ms)", "scan: middle (ms)", "scan: missing", "fixed seek (us)");
        // warm-up
        { Path w = Files.createTempFile("w", ".txt"); try (BufferedWriter bw =
Files.newBufferedWriter(w)) { for (int i = 0; i < 5000; i++) bw.write(make(i).toLine() + "\n"); }
          BusPassStore s = new BusPassStore(w); for (int i = 0; i < 30; i++) sink +=
s.find("P6000") == null ? 1 : 0; Files.delete(w); }
        for (int n : ns) {
            Path text = Files.createTempFile("passes" + n, ".txt");
            Path fixed = Files.createTempFile("fixed" + n, ".dat");
            try (BufferedWriter bw = Files.newBufferedWriter(text)) { for (int i = 0; i < n; i++)
bw.write(make(i).toLine() + "\n"); }
            try (FixedRecordStore fs = new FixedRecordStore(fixed)) { for (int i = 0; i < n; i++)
fs.put(make(i)); }
            BusPassStore store = new BusPassStore(text);
            int reps = n >= 50000 ? 10 : 30;
            double last = 0, mid = 0, miss = 0;
            for (int r = 0; r < reps; r++) {
                long t = System.nanoTime(); sink += store.find("P" + (1001 + n - 1)) != null ? 1 :
0; last += (System.nanoTime() - t) / 1e6;
                t = System.nanoTime(); sink += store.find("P" + (1001 + n / 2)) != null ? 1 : 0;
mid += (System.nanoTime() - t) / 1e6;
                t = System.nanoTime(); sink += store.find("P0") != null ? 1 : 0; miss +=
(System.nanoTime() - t) / 1e6;
            }
            last /= reps; mid /= reps; miss /= reps;

            double seekUs;
            try (FixedRecordStore fs = new FixedRecordStore(fixed)) {
                for (int i = 0; i < 2000; i++) sink += fs.find("P" + (1001 + (i * 7919) %
n)).route;       // warm-up
                int q = 20000; long t = System.nanoTime();
                for (int i = 0; i < q; i++) sink += fs.find("P" + (1001 + (i * 7919) % n)).route;
                seekUs = (System.nanoTime() - t) / 1e3 / q;
            }

            // HashMap alternative: read the file ONCE, then every search is a hash lookup
            long before = used();
            long t0 = System.nanoTime();
            Map<String, BusPass> map = new HashMap<>();
            store.scan(p -> { map.put(p.passId, p); return true; });
            double loadMs = (System.nanoTime() - t0) / 1e6;
            double heapMB = (used() - before) / 1048576.0;
            int q = 200000; t0 = System.nanoTime();
            for (int i = 0; i < q; i++) sink += map.get("P" + (1001 + (i * 7919) % n)).route;
            double lookupNs = (System.nanoTime() - t0) / (double) q;
            sink += map.size();

            long tb = Files.size(text), fb = Files.size(fixed);
            System.out.printf("%,8d %,12d %,12d %14.2f %16.2f %14.2f %14.2f%n", n, tb, fb, last,
mid, miss, seekUs);
            csv.printf("%d,%d,%d,%.3f,%.3f,%.3f,%.3f,%.3f,%.1f,%.2f%n", n, tb, fb, last, mid,
miss, seekUs, loadMs, lookupNs, heapMB);
            Files.delete(text); Files.delete(fixed);
        }
        csv.close();

        // ---------------- q searches on n passes: scan every time  vs  HashMap loaded once
----------------
        System.out.println();
        System.out.println("=== q searches on n passes: O(q * n) scan  versus  O(n + q) HashMap
===");
```

```
        System.out.printf("%8s %6s %18s %26s %18s%n", "n", "q", "scan every time (ms)", "HashMap:
load + q lookups (ms)", "speed-up");
        PrintWriter c2 = new PrintWriter("search_many_results.csv");
        c2.println("n,q,scan_total_ms,hashmap_total_ms");
        for (int n : new int[]{1000, 10000, 100000}) {
            int q = 100;
            Path text = Files.createTempFile("many" + n, ".txt");
            try (BufferedWriter bw = Files.newBufferedWriter(text)) { for (int i = 0; i < n; i++)
bw.write(make(i).toLine() + "\n"); }
            BusPassStore store = new BusPassStore(text);
            long t0 = System.nanoTime();
            for (int i = 0; i < q; i++) sink += store.find("P" + (1001 + (i * 7919) % n)) !=
null ? 1 : 0;
            double scan = (System.nanoTime() - t0) / 1e6;
            t0 = System.nanoTime();
            Map<String, BusPass> map = new HashMap<>();
            store.scan(p -> { map.put(p.passId, p); return true; });
            for (int i = 0; i < q; i++) sink += map.get("P" + (1001 + (i * 7919) % n)) != null ? 1
: 0;
            double hash = (System.nanoTime() - t0) / 1e6;
            System.out.printf("%,8d %6d %18.1f %26.1f %17.1fx%n", n, q, scan, hash, scan / hash);
            c2.printf("%d,%d,%.2f,%.2f%n", n, q, scan, hash);
            Files.delete(text);
        }
        c2.close();
        System.out.println("(checksum " + sink + ")");
    }
}
```

TaskB1/ReadAllLinesMemory.java

```
import java.io.*;
import java.lang.management.ManagementFactory;
import java.nio.file.*;
import java.util.List;

/** A2 alternative: memory and time of Files.readAllLines() against a buffered stream for the 50
MB attendance log. */
public class ReadAllLinesMemory {
    static long allocated() { com.sun.management.ThreadMXBean b =
(com.sun.management.ThreadMXBean) ManagementFactory.getThreadMXBean(); return
b.getThreadAllocatedBytes(Thread.currentThread().getId()); }
    static long used() { for (int i = 0; i < 3; i++) System.gc(); Runtime r =
Runtime.getRuntime(); return r.totalMemory() - r.freeMemory(); }
    public static void main(String[] args) throws Exception {
        Path f = Files.createTempFile("log50", ".log");
        long lines = 50L * 1024 * 1024 / 33;
        try (BufferedWriter w = Files.newBufferedWriter(f)) { for (long i = 0; i < lines; i++)
w.write(String.format("2026-11-09,%09d,IN,08:45:12\n", 192311000L + (i % 9000))); }
        System.out.printf("file: %,d bytes, %,d lines%n", Files.size(f), lines);
        long[] calls = new long[1];
        for (int w = 0; w < 2; w++) { LineCounter.readAllLines(f); LineCounter.bufferedStream(f,
calls); }
        long base = used(), a0 = allocated(); long t = System.nanoTime();
        List<String> all = Files.readAllLines(f);
        double ms = (System.nanoTime() - t) / 1e6; long alloc = allocated() - a0; long retained =
used() - base;
        System.out.printf("Files.readAllLines : %.0f ms, %,d lines in the list, memory retained by
the list = %.0f MB, allocated while reading = %.0f MB%n", ms, all.size(), retained / 1048576.0,
alloc / 1048576.0);
        all = null;
        base = used(); a0 = allocated(); t = System.nanoTime();
        long n = LineCounter.bufferedStream(f, calls);
        System.out.printf("BufferedInputStream: %.0f ms, %,d lines counted, allocated while
reading = %.3f MB (the 8,192-byte buffer), retained = about 0%n", (System.nanoTime() - t) / 1e6,
n, (allocated() - a0) / 1048576.0);
```

```
        Files.delete(f);
    }
}
```

Task B2 – Engineer an Adversarial Input (5 marks, K5)

B2 asks for deliberately constructed inputs that make the designs of Part A fail, and for the response of the repaired designs. The experiments use a small **local test database** (table users, created only for this assignment; the passwords are test values).

TaskB2/users.sql

```
-- Task B2: LOCAL TEST DATABASE created only for this assignment (never use real passwords here)
CREATE TABLE users (
    user_no INT PRIMARY KEY,
    id      VARCHAR(20) NOT NULL UNIQUE,
    pwd     VARCHAR(40) NOT NULL,
    role    VARCHAR(10) NOT NULL
);
INSERT INTO users(user_no, id, pwd, role) VALUES (1, 'admin', 'Adm!n#2026', 'ADMIN');
INSERT INTO users(user_no, id, pwd, role) VALUES (2, 'arun',  'secret123',  'STUDENT');
INSERT INTO users(user_no, id, pwd, role) VALUES (3, 'divya', 'pass456',    'STUDENT');
INSERT INTO users(user_no, id, pwd, role) VALUES (4, 'obrien', 'it''s-me',  'STUDENT');
```

TaskB2/LoginCheck.java - unsafe(), safe() and the two weaker fixes

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.regex.Pattern;

/** The unsafe login of Task A5.2, the PreparedStatement redesign, and the two weaker "fixes"
analysed in B2. */
public class LoginCheck {
    /** Result: the SQL text that was run (for the report) and the ids that were returned. */
    public static class Result { public String sql; public List<String> ids = new ArrayList<>();
public boolean accepted() { return !ids.isEmpty(); } }

    // ---------- (1) UNSAFE: the original code ----------
    public static Result unsafe(Connection c, String id, String pwd) throws SQLException {
        Result r = new Result();
        r.sql = "SELECT * FROM users WHERE id='" + id + "' AND pwd='" + pwd + "'";
        try (Statement st = c.createStatement(); ResultSet rs = st.executeQuery(r.sql)) { while
(rs.next()) r.ids.add(rs.getString("id")); }
        return r;
    }

    // ---------- (2) SAFE: PreparedStatement ----------
    public static Result safe(Connection c, String id, String pwd) throws SQLException {
        Result r = new Result();
        r.sql = "SELECT * FROM users WHERE id = ? AND pwd = ?     [id and pwd sent as DATA]";
        try (PreparedStatement ps = c.prepareStatement("SELECT * FROM users WHERE id = ? AND pwd =
?")) {
            ps.setString(1, id); ps.setString(2, pwd);
            try (ResultSet rs = ps.executeQuery()) { while (rs.next())
r.ids.add(rs.getString("id")); }
        }
        return r;
    }

    // ---------- (3) WEAKER: escape the quote characters, still build the SQL string ----------
    public static String escapeQuotes(String s) { return s.replace("'", "''"); }
    public static Result escaped(Connection c, String id, String pwd) throws SQLException { return
unsafe(c, escapeQuotes(id), escapeQuotes(pwd)); }

    // ---------- (4) WEAKER: validate with a regular expression, still build the SQL string
----------
    static final Pattern SAFE_TEXT = Pattern.compile("[A-Za-z0-9_]{1,20}");
    public static Result regex(Connection c, String id, String pwd) throws SQLException {
```

```
        if (!SAFE_TEXT.matcher(id).matches() || !SAFE_TEXT.matcher(pwd).matches()) { Result r =
new Result(); r.sql = "(rejected by the validator, no SQL sent)"; return r; }
        return unsafe(c, id, pwd);
    }

    // ---------- a numeric context: no quotes at all, so quote-escaping cannot help ----------
    public static Result findByNumber(Connection c, String userNoText, boolean useEscaping) throws
SQLException {
        Result r = new Result();
        String text = useEscaping ? escapeQuotes(userNoText) : userNoText;
        r.sql = "SELECT * FROM users WHERE user_no = " + text;
        try (Statement st = c.createStatement(); ResultSet rs = st.executeQuery(r.sql)) { while
(rs.next()) r.ids.add(rs.getString("id")); }
        return r;
    }
}
```

B2.1 – Inputs that fool the unsafe login check

AdversarialDemo runs each input against the original string-built query and against the PreparedStatement version. **None of the three attacks contains a valid password.**

Output of java AdversarialDemo

```
Test database: users = admin, arun, divya, obrien (passwords are test values only)

=== 1. Normal behaviour (both versions must agree) ===
 arun / secret123 :
   unsafe                       accepted=true  ids=[arun]
   PreparedStatement            accepted=true  ids=[arun]
 arun / wrongpass :
   unsafe                       accepted=false ids=[]
   PreparedStatement            accepted=false ids=[]

=== 2. Adversarial inputs (NO valid password is known in any of them) ===
 A1  password = ' OR '1'='1     [id="x", pwd="' OR '1'='1"]
   SQL built by the unsafe code: SELECT * FROM users WHERE id='x' AND pwd='' OR '1'='1'
   unsafe                       accepted=true  ids=[admin, arun, divya, obrien]
   PreparedStatement            accepted=false ids=[]
 A2  id = arun' OR '1'='1     [id="arun' OR '1'='1", pwd="anything"]
   SQL built by the unsafe code: SELECT * FROM users WHERE id='arun' OR '1'='1' AND pwd='anything'
   unsafe                       accepted=true  ids=[arun]
   PreparedStatement            accepted=false ids=[]
 A3  id = admin' --  (comment)     [id="admin' -- ", pwd="anything"]
   SQL built by the unsafe code: SELECT * FROM users WHERE id='admin' -- ' AND pwd='anything'
   unsafe                       accepted=true  ids=[admin]
   PreparedStatement            accepted=false ids=[]

=== 3. The weaker fixes: escaping quotes and regular-expression validation ===
 Attacks A1-A3 against the two weaker fixes:
   escaped  A1                  accepted=false ids=[]
   regex    A1                  accepted=false ids=[]
   escaped  A2                  accepted=false ids=[]
   regex    A2                  accepted=false ids=[]
   escaped  A3                  accepted=false ids=[]
   regex    A3                  accepted=false ids=[]
 Numeric context (no quotes in the SQL, user_no = <input>), input "0 OR 1=1":
   escaped quotes               accepted=true  ids=[admin, arun, divya, obrien]
   SQL: SELECT * FROM users WHERE user_no = 0 OR 1=1
 A legitimate user whose password contains an apostrophe (obrien / it's-me):
   unsafe code raised: near "s": syntax error  [SELECT * FROM users WHERE id='obrien' AND pwd='it's-me']
   unsafe (no protection)       accepted=false ids=[]
   escaped quotes               accepted=true  ids=[obrien]
   regex validation             accepted=false ids=[]
   PreparedStatement            accepted=true  ids=[obrien]
 Admin with a normal password containing # and !   (admin / Adm!n#2026):
```

```
   regex validation            accepted=false ids=[]
   PreparedStatement           accepted=true  ids=[admin]

=== 4. Cost per login check (20,000 logins, microseconds per login, best of 5) ===
   string-built Statement (unsafe)              11.24 us
   escape quotes + Statement                     9.50 us
   regex validate + Statement                    9.84 us
   PreparedStatement, prepared every time       10.67 us
   PreparedStatement, prepared ONCE              3.87 us
   validation work alone: regex match 194 ns, quote escaping 38 ns
(checksum 20507500)
```

Input (id / pwd)	SQL built by the unsafe code	Unsafe check	PreparedStatement
A1: x / ' OR '1'='1	SELECT * FROM users WHERE id='x' AND pwd='' OR '1'='1'	accepted, returns all four users	rejected
A2: arun' OR '1'='1 / anything	… WHERE id='arun' OR '1'='1' AND pwd='anything'	accepted as arun	rejected
A3: admin' -- / anything	… WHERE id='admin' -- ' AND pwd='anything'	accepted as admin	rejected
normal: arun / secret123	a correct query	accepted	accepted
normal: arun / wrongpass	a correct query	rejected	rejected

Table B2.1 – Adversarial and normal inputs (executed): the two versions agree on normal input and differ on every attack

B2.2 – A damaged bus-pass file

DamagedFileDemo builds a bus-pass file containing the damage named in the assignment (wrong number of fields, non-numeric amount, empty line) and six more kinds, between good records, and lists the response of BusPassStore:

Output of java DamagedFileDemo

```
=== The damaged file (11 lines) ===
 1 | P1001,192311001,Arun,12,Main Gate,2027-06-30,4500.00,1a06a4e3
 2 | P1100,192311100,Short
 3 | P1101,192311101,Kumar, Meena,12,Main Gate,2027-06-30,4500.00,f8866aaf
 4 | P1102,192311102,Ravi,3,Main Gate,2027-06-30,forty,29541469
 5 | P1103,192311103,Ravi,abc,Main Gate,2027-06-30,4500.00,265efdab
 6 | P1104,192311104,Ravi,3,Main Gate,2027-13-45,4500.00,a591a562
 7 | P1105,192311105,Ravi,3,Main Gate,2027-06-30,-50.00,5e905741
 8 |
 9 | P1002,192311002,Divyaa,7,Bus Stand,2027-06-30,3800.00,cdc554fb
10 | P1002,192311002,Divya,7,Bus Stand,2027-06-30,3800
11 | P1003,192311003,Kavin,12,Main Gate,2027-06-30,4500.00,36146e7b

=== Response of the store: list all passes of route 12 (a full scan) ===
passes found : [P1001 192311001 Arun route 12 Main Gate valid 2027-06-30 Rs.4500.0, P1003 192311003 Kavin
route 12 Main Gate valid 2027-06-30 Rs.4500.0]
damaged lines reported (9):
   line 2: expected 8 fields but found 3
   line 3: expected 8 fields but found 9
   line 4: not a number: For input string: "forty"
   line 5: not a number: For input string: "abc"
   line 6: bad validity date '2027-13-45'
   line 7: amount must be positive: -50.00
   line 8: empty line
   line 9: checksum mismatch (line changed or cut off)
   line 10: expected 8 fields but found 7

what the damage reveals about the ORIGINAL design (5 fields, no checksum, String.split):
   the original store would split  "P1101,192311101,Kumar, Meena,12,4500.0"  into 6 fields -> silently
ignored, the pass is lost
   with a non-numeric amount the original (no validation) would store it and fail later in
Double.parseDouble
```

Damage	Reported as	Good records
wrong number of fields (3 instead of 8)	expected 8 fields but found 3	unaffected

Damage	Reported as	Good records
name with an unescaped comma (nine fields)	expected 8 fields but found 9	unaffected
non-numeric amount ("forty"), non-numeric route ("abc")	not a number: …	unaffected
impossible date 2027-13-45	bad validity date	unaffected
negative amount	amount must be positive	unaffected
empty line	empty line	unaffected
line edited by hand / line cut off by a power failure	checksum mismatch / wrong field count	unaffected

Table B2.3 – Response of the store to nine damaged lines: all nine are reported with a line number, and the three good records are still found

B2.3 – What each input reveals about the original designs

Input	What it reveals
A1 (pwd = ' OR '1'='1)	The query returns every row: the program has no check that exactly one user matched, and the attacker also learns that the database content is reachable. It reveals that data and code are mixed and that operator precedence (AND before OR) can be exploited.
A2 (id = arun' OR …)	The password test can be skipped for a chosen account: injection is not only a data leak, it is an authentication bypass.
A3 (id = admin' --)	A comment sequence removes the rest of the query; the attacker does not even need to know the structure of the password test.
A legitimate password with an apostrophe (obrien / it's-me)	The unsafe code fails with an SQL syntax error for an honest user: the design is also a correctness bug, not only a security hole.
Wrong number of fields in the file	The original 5-field store using String.split would silently ignore (or mis-read) the line: it reveals that nothing in the original design detects damage.
Non-numeric amount ("forty")	The original would store it without validation and fail later in Double.parseDouble: it reveals missing input validation.
Empty line / cut-off last line	The original has no rule for blank or half-written lines; a power failure during add could silently corrupt the next record.
Name containing the delimiter	Without escaping the record splits into too many fields: it reveals the weakness of a plain delimiter format (A1.1).

Table B2.4 – Interpretation of the inputs

Alternative Concept and Complexity Analysis – escaping the quotes, or validating with a regular expression

The alternative solutions. (a) Keep the string-built SQL but double every apostrophe (replace("'", "''")). (b) Keep the string-built SQL but accept only ids and passwords that match [A-Za-z0-9_]{1,20}. Both are implemented in LoginCheck (escaped(), regex()).

Input	Escaping quotes	Regex validation	PreparedStatement
A1, A2, A3 (quote-based attacks)	rejected	rejected	rejected
Numeric context: WHERE user_no = <input>, input “0 OR 1=1” (no quotes needed)	accepted: returns all four users (SQL: … WHERE user_no = 0 OR 1=1)	would reject (letters/spaces not allowed)	rejected (the value is a number or an error)
Honest password with an apostrophe: it's-me	accepted (correct)	rejected: an honest user cannot log in	accepted (correct)
Honest password with # and !: Adm!n#2026	accepted	rejected	accepted

Table B2.5 – Weaknesses of the alternatives (executed, except the cell “would reject”, which follows from the pattern). Escaping fails whenever the input is not inside quotes; regex validation blocks honest input and has to be remembered in every query.

Why they are weaker. Escaping depends on the database dialect and on the character set: MySQL also treats a backslash as an escape character in strings, so a naive “double the apostrophe” rule can be defeated by an input ending in a backslash (this MySQL-specific bypass could not be demonstrated here because SQLite does not treat the backslash specially; it is a documented limitation, not an executed result). It must be applied to every value in every query, and one

forgotten value reopens the hole. A regular expression is a **whitelist of characters**, which blocks injection but also blocks honest users (apostrophes in names, # and ! in passwords) and does not protect numeric parameters unless the pattern is exactly right. PreparedStatement is different in kind: the SQL structure is fixed before the data arrives, so there is nothing to forget.

Version (20,000 logins, best of 5)	Time per login (µs)
string-built Statement (unsafe)	11.24
escape quotes + Statement	9.50
regex validate + Statement	9.84
PreparedStatement, prepared every time	10.67
PreparedStatement, prepared ONCE	3.87

Table B2.6 – Cost per login check (executed; database on disk, read-only queries)

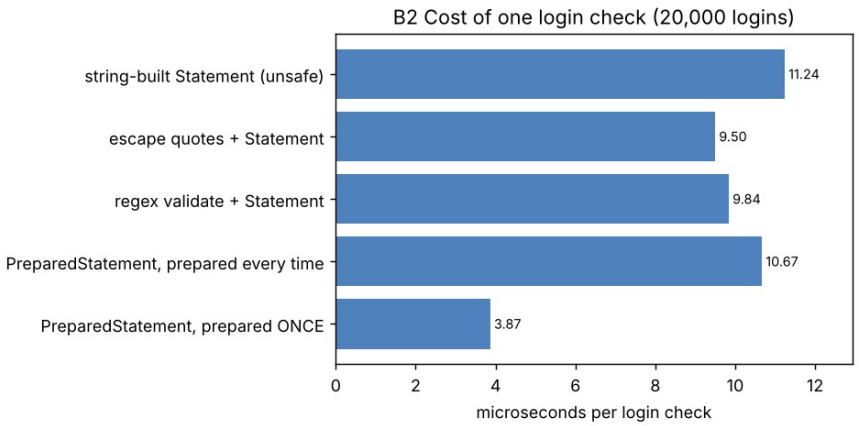


Figure B2.1 – Cost of one login check

Analysis. The validation work itself is negligible: a regular-expression match takes about 194 ns and quote escaping 38 ns per input, i.e. 0.19 µs and 0.04 µs against about 11.24 µs for the query. The real difference is how the statement reaches the database: a string-built Statement must be parsed again for every login (11.24 µs), a PreparedStatement that is prepared once and reused is the fastest (3.87 µs, about 3× faster), and a PreparedStatement that is re-prepared for every call (10.67 µs) costs roughly the same as the string-built one because preparing is the parse. All five are O(1) per login in time and space. **Evaluation:** security does not cost speed here: the safe version is not slower than the unsafe one, and when the statement is reused it is much faster. There is no performance argument for the weaker fixes.

Source code of Task B2

TaskB2/AdversarialDemo.java

```
import java.io.File;
import java.nio.file.*;
import java.sql.*;
import java.util.*;

/** Task B2: deliberately constructed adversarial inputs against the unsafe and the safe login. */
public class AdversarialDemo {
    static long sink = 0;
    static void line(String title, LoginCheck.Result r) { System.out.printf("   %-28s accepted=%-
5s ids=%-26s%n", title, r.accepted(), r.ids); }

    public static void main(String[] args) throws Exception {
        minisqlite.MiniSqlite.install();
        File db = File.createTempFile("b2demo", ".db"); db.delete();
        try (Connection c = DriverManager.getConnection("jdbc:minisqlite:" +
db.getAbsolutePath())) {
```

```
            try (Statement st = c.createStatement()) { for (String s :
Files.readString(Path.of("users.sql")).split(";\\s*\\n")) if (!s.isBlank()) st.executeUpdate(s); }
            System.out.println("Test database: users = admin, arun, divya, obrien (passwords are
test values only)");

            System.out.println();
            System.out.println("=== 1. Normal behaviour (both versions must agree) ===");
            System.out.println(" arun / secret123 :"); line("unsafe", LoginCheck.unsafe(c, "arun",
"secret123")); line("PreparedStatement", LoginCheck.safe(c, "arun", "secret123"));
            System.out.println(" arun / wrongpass :"); line("unsafe", LoginCheck.unsafe(c, "arun",
"wrongpass")); line("PreparedStatement", LoginCheck.safe(c, "arun", "wrongpass"));

            System.out.println();
            System.out.println("=== 2. Adversarial inputs (NO valid password is known in any of
them) ===");
            String[][] attacks = {
                {"A1  password = ' OR '1'='1", "x", "' OR '1'='1"},
                {"A2  id = arun' OR '1'='1", "arun' OR '1'='1", "anything"},
                {"A3  id = admin' --  (comment)", "admin' -- ", "anything"},
            };
            for (String[] a : attacks) {
                System.out.println(" " + a[0] + "     [id=\"" + a[1] + "\", pwd=\"" + a[2] +
"\"]");
                LoginCheck.Result u = LoginCheck.unsafe(c, a[1], a[2]);
                System.out.println("   SQL built by the unsafe code: " + u.sql);
                line("unsafe", u);
                line("PreparedStatement", LoginCheck.safe(c, a[1], a[2]));
            }

            System.out.println();
            System.out.println("=== 3. The weaker fixes: escaping quotes and regular-expression
validation ===");
            System.out.println(" Attacks A1-A3 against the two weaker fixes:");
            for (String[] a : attacks) { line("escaped  " + a[0].substring(0, 2),
LoginCheck.escaped(c, a[1], a[2])); line("regex    " + a[0].substring(0, 2), LoginCheck.regex(c,
a[1], a[2])); }
            System.out.println(" Numeric context (no quotes in the SQL, user_no = <input>),
input \"0 OR 1=1\":");
            line("escaped quotes", LoginCheck.findByNumber(c, "0 OR 1=1", true));
            System.out.println("   SQL: " + LoginCheck.findByNumber(c, "0 OR 1=1", true).sql);
            System.out.println(" A legitimate user whose password contains an apostrophe (obrien /
it's-me):");
            line("unsafe (no protection)", safeCatch(c, "obrien", "it's-me"));
            line("escaped quotes", LoginCheck.escaped(c, "obrien", "it's-me"));
            line("regex validation", LoginCheck.regex(c, "obrien", "it's-me"));
            line("PreparedStatement", LoginCheck.safe(c, "obrien", "it's-me"));
            System.out.println(" Admin with a normal password containing # and !   (admin / Adm!
n#2026):");
            line("regex validation", LoginCheck.regex(c, "admin", "Adm!n#2026"));
            line("PreparedStatement", LoginCheck.safe(c, "admin", "Adm!n#2026"));

            System.out.println();
            System.out.println("=== 4. Cost per login check (20,000 logins, microseconds per
login, best of 5) ===");
            int n = 20000;
            String[] names = {"string-built Statement (unsafe)", "escape quotes + Statement",
"regex validate + Statement", "PreparedStatement, prepared every time", "PreparedStatement,
prepared ONCE"};
            for (int w = 0; w < 3; w++) for (int k = 0; k < 5; k++) run(c, k, 500);
            for (int k = 0; k < 5; k++) {
                double best = 1e18;
                for (int r = 0; r < 5; r++) { long t = System.nanoTime(); run(c, k, n); best =
Math.min(best, (System.nanoTime() - t) / 1e3 / n); }
                System.out.printf("   %-42s %8.2f us%n", names[k], best);
            }
```

```
            long t = System.nanoTime();
            for (int i = 0; i < 2_000_000; i++) { sink += LoginCheck.SAFE_TEXT.matcher("arun" + (i
& 7)).matches() ? 1 : 0; }
            double regexNs = (System.nanoTime() - t) / 2e6;
            t = System.nanoTime();
            for (int i = 0; i < 2_000_000; i++) { sink += LoginCheck.escapeQuotes("secret12" + (i
& 7)).length(); }
            double escNs = (System.nanoTime() - t) / 2e6;
            System.out.printf("   validation work alone: regex match %.0f ns, quote escaping %.0f
ns%n", regexNs, escNs);
            System.out.println("(checksum " + sink + ")");
        }
        db.delete();
    }

    static LoginCheck.Result safeCatch(Connection c, String id, String pwd) {
        try { return LoginCheck.unsafe(c, id, pwd); }
        catch (SQLException e) { LoginCheck.Result r = new LoginCheck.Result(); r.sql = "(SQL
error: " + e.getMessage() + ")"; System.out.println("   unsafe code raised: " + e.getMessage());
return r; }
    }

    static PreparedStatement reused;
    static void run(Connection c, int kind, int n) throws SQLException {
        if (kind == 4 && reused == null) reused = c.prepareStatement("SELECT * FROM users WHERE id
= ? AND pwd = ?");
        for (int i = 0; i < n; i++) {
            String id = (i % 2 == 0) ? "arun" : "divya", pwd = (i % 2 == 0) ? "secret123" :
"pass456";
            switch (kind) {
                case 0: sink += LoginCheck.unsafe(c, id, pwd).ids.size(); break;
                case 1: sink += LoginCheck.escaped(c, id, pwd).ids.size(); break;
                case 2: sink += LoginCheck.regex(c, id, pwd).ids.size(); break;
                case 3: sink += LoginCheck.safe(c, id, pwd).ids.size(); break;
                default: reused.setString(1, id); reused.setString(2, pwd); try (ResultSet rs =
reused.executeQuery()) { while (rs.next()) sink++; }
            }
        }
    }
}
```

TaskB2/DamagedFileDemo.java

```
import java.nio.file.*;
import java.time.LocalDate;
import java.util.zip.CRC32;

/** Task B2 (second bullet): a damaged bus-pass file and the response of the file store
(BusPassStore from Task B1). */
public class DamagedFileDemo {
    static String withCrc(String body) { CRC32 c = new CRC32(); c.update(body.getBytes()); return
body + "," + Long.toHexString(c.getValue()); }
    public static void main(String[] args) throws Exception {
        Path f = Files.createTempFile("damagedpasses", ".txt");
        BusPassStore store = new BusPassStore(f);
        store.append(new BusPass("P1001", "192311001", "Arun", 12, "Main Gate", LocalDate.of(2027,
6, 30), 4500));
        String good2 = new BusPass("P1002", "192311002", "Divya", 7, "Bus Stand",
LocalDate.of(2027, 6, 30), 3800).toLine();
        String[][] bad = {
            {"wrong number of fields (3 instead of 8)", "P1100,192311100,Short"},
            {"too many fields (name with an unescaped comma)", withCrc("P1101,192311101,Kumar,
Meena,12,Main Gate,2027-06-30,4500.00")},
            {"non-numeric amount (checksum correct)", withCrc("P1102,192311102,Ravi,3,Main
Gate,2027-06-30,forty")},
```

```
            {"non-numeric route number", withCrc("P1103,192311103,Ravi,abc,Main Gate,2027-06-
30,4500.00")},
            {"impossible validity date", withCrc("P1104,192311104,Ravi,3,Main Gate,2027-13-
45,4500.00")},
            {"negative amount", withCrc("P1105,192311105,Ravi,3,Main Gate,2027-06-30,-50.00")},
            {"empty line", ""},
            {"line edited by hand (checksum mismatch)", good2.replace("Divya", "Divyaa")},
            {"line cut off by a power failure", good2.substring(0, good2.length() - 12)},
        };
        StringBuilder file = new StringBuilder();
        for (String[] b : bad) file.append(b[1]).append('\n');
        Files.writeString(f, file, StandardOpenOption.APPEND);
        store.append(new BusPass("P1003", "192311003", "Kavin", 12, "Main Gate",
LocalDate.of(2027, 6, 30), 4500));

        System.out.println("=== The damaged file (" + Files.readAllLines(f).size() + " lines)
===");
        int i = 1; for (String l : Files.readAllLines(f)) System.out.printf("%2d | %s%n", i++,
l.length() > 100 ? l.substring(0, 100) + "..." : l);
        System.out.println();
        System.out.println("=== Response of the store: list all passes of route 12 (a full scan)
===");
        System.out.println("passes found : " + store.listByRoute(12));
        System.out.println("damaged lines reported (" + store.damaged.size() + "):");
        for (String d : store.damaged) System.out.println("   " + d);
        System.out.println();
        System.out.println("what the damage reveals about the ORIGINAL design (5 fields, no
checksum, String.split):");
        String original = "P1101,192311101,Kumar, Meena,12,4500.0";          // a name containing
the delimiter, as the original PassStore would see it
        String[] parts = original.split(",", -1);
        System.out.println("   the original store would split  \"" + original + "\"  into " +
parts.length + " fields -> silently ignored, the pass is lost");
        System.out.println("   with a non-numeric amount the original (no validation) would store
it and fail later in Double.parseDouble");
        Files.delete(f);
    }
}
```

Task B3 – Statement vs PreparedStatement vs Batch (8 marks, K5)

B3 compares three ways to insert passes into the table bus_pass, with auto-commit and with transactions, and completes the batch loader PassLoader. The table (MySQL script; the same text also runs in SQLite):

TaskB3/bus_pass.sql

```
-- Task B3: test table (MySQL).  The same statement also runs in SQLite, which the test driver
uses.
DROP TABLE IF EXISTS bus_pass;
CREATE TABLE bus_pass (
    pass_id  VARCHAR(10)   PRIMARY KEY,
    reg_no   VARCHAR(12)   NOT NULL,
    name     VARCHAR(60)   NOT NULL,
    route_no INT           NOT NULL,
    amount   DECIMAL(8,2)  NOT NULL
);
```

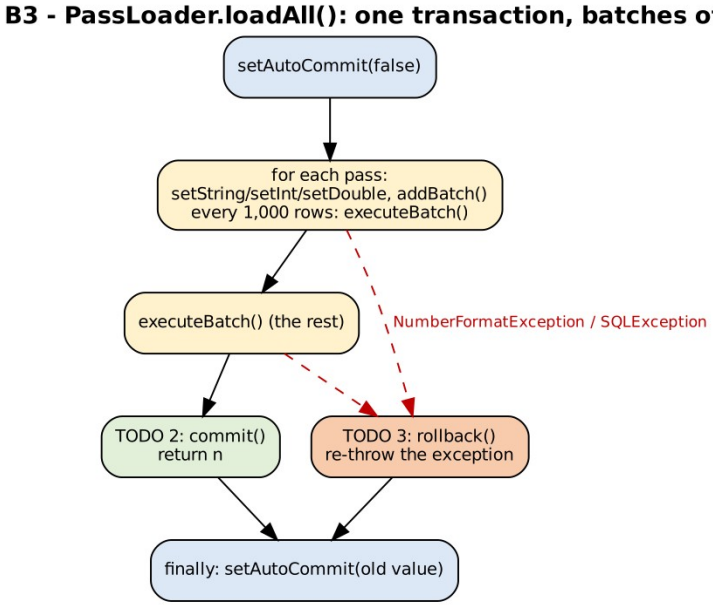


Figure B3.1 – Control flow of PassLoader.loadAll() (TODO 2 and TODO 3 are the commit and rollback paths)

B3.1 – Correctness on small data (2 marks)

B3Correctness inserts the same 10 records with (a) a string-built Statement, (b) a PreparedStatement and (c) PassLoader (batch + transaction), and for each reads the table back with SELECT, compares every field with the source value, and prints the column names and types from ResultSetMetaData.

Output of java B3Correctness

```
=== Statement (string-built SQL) ===
  columns: pass_id VARCHAR(10) (java.sql.Types.VARCHAR), reg_no VARCHAR(12) (java.sql.Types.VARCHAR),
name VARCHAR(60) (java.sql.Types.VARCHAR), route_no INT (java.sql.Types.INTEGER), amount DECIMAL(8,2)
(java.sql.Types.DECIMAL)
  row 1: P100000, 190000000, Student Name 0, 1, 3000.5
  row 2: P100001, 190000001, Student Name 1, 2, 3001.5
  row 3: P100002, 190000002, Student Name 2, 3, 3002.5
  PASS: 10 rows found, every field equals the source value = true
=== PreparedStatement ===
  columns: pass_id VARCHAR(10) (java.sql.Types.VARCHAR), reg_no VARCHAR(12) (java.sql.Types.VARCHAR),
name VARCHAR(60) (java.sql.Types.VARCHAR), route_no INT (java.sql.Types.INTEGER), amount DECIMAL(8,2)
(java.sql.Types.DECIMAL)
  row 1: P100000, 190000000, Student Name 0, 1, 3000.5
  row 2: P100001, 190000001, Student Name 1, 2, 3001.5
```

```
  row 3: P100002, 190000002, Student Name 2, 3, 3002.5
  PASS: 10 rows found, every field equals the source value = true
   PassLoader.loadAll returned 10 and auto-commit is back to true
=== PassLoader (batch + transaction, completed TODOs) ===
  columns: pass_id VARCHAR(10) (java.sql.Types.VARCHAR), reg_no VARCHAR(12) (java.sql.Types.VARCHAR),
name VARCHAR(60) (java.sql.Types.VARCHAR), route_no INT (java.sql.Types.INTEGER), amount DECIMAL(8,2)
(java.sql.Types.DECIMAL)
  row 1: P100000, 190000000, Student Name 0, 1, 3000.5
  row 2: P100001, 190000001, Student Name 1, 2, 3001.5
  row 3: P100002, 190000002, Student Name 2, 3, 3002.5
  PASS: 10 rows found, every field equals the source value = true

Result: 3 passed, 0 failed
```

Result: 3 of 3 pass. Every version stored exactly 10 rows with all fields equal to the source. ResultSetMetaData confirms the table definition (pass_id VARCHAR(10), reg_no VARCHAR(12), name VARCHAR(60), route_no INT, amount DECIMAL(8,2)) and the matching java.sql.Types, so the program can check what it is reading instead of assuming. The completed PassLoader returned 10 and restored auto-commit to true.

B3.2 – Increase the data size (2 marks)

LoaderBenchmark inserts 1,000, 10,000 and 100,000 generated passes into a fresh database file for every run, with each approach, in two commit modes. Table B3.1 is for a database on a real disk, where every commit is made durable (the realistic case); Table B3.2 repeats the runs with the database file in memory (/dev/shm) to show the cost of the work without disk waits.

Rows	Approach	Auto-commit (ms)	One transaction (ms)	Auto-commit ÷ transaction	Transaction: µs per row
1,000	Statement	867.5	22.4	38.8×	22.4
1,000	PreparedStatement	1007.6	12.3	81.9×	12.3
1,000	Prepared+Batch(1000)	1080.8	15.3	70.6×	15.3
10,000	Statement	8409.3	67.2	125.2×	6.7
10,000	PreparedStatement	8725.0	59.5	146.5×	6.0
10,000	Prepared+Batch(1000)	8166.1	75.5	108.2×	7.5
100,000	Statement	80075.7	1107.7	72.3×	11.1
100,000	PreparedStatement	76779.8	646.4	118.8×	6.5
100,000	Prepared+Batch(1000)	82087.4	311.6	263.4×	3.1

Table B3.1 – Insert time, database on disk (from TaskB3/loader_results.csv). Single runs for the large cases, best of 3 for the 1,000-row transaction cases.

Rows	Approach	Auto-commit (ms)	One transaction (ms)	Auto-commit ÷ transaction	Transaction: µs per row
1,000	Statement	75.8	10.6	7.1×	10.6
1,000	PreparedStatement	61.2	10.5	5.8×	10.5
1,000	Prepared+Batch(1000)	62.5	7.1	8.8×	7.1
10,000	Statement	430.7	75.7	5.7×	7.6
10,000	PreparedStatement	649.8	69.6	9.3×	7.0
10,000	Prepared+Batch(1000)	395.1	38.5	10.3×	3.8
100,000	Statement	3685.2	806.5	4.6×	8.1
100,000	PreparedStatement	2933.4	272.4	10.8×	2.7
100,000	Prepared+Batch(1000)	3134.1	279.5	11.2×	2.8

Table B3.2 – The same runs with the database file in memory (no disk waits)

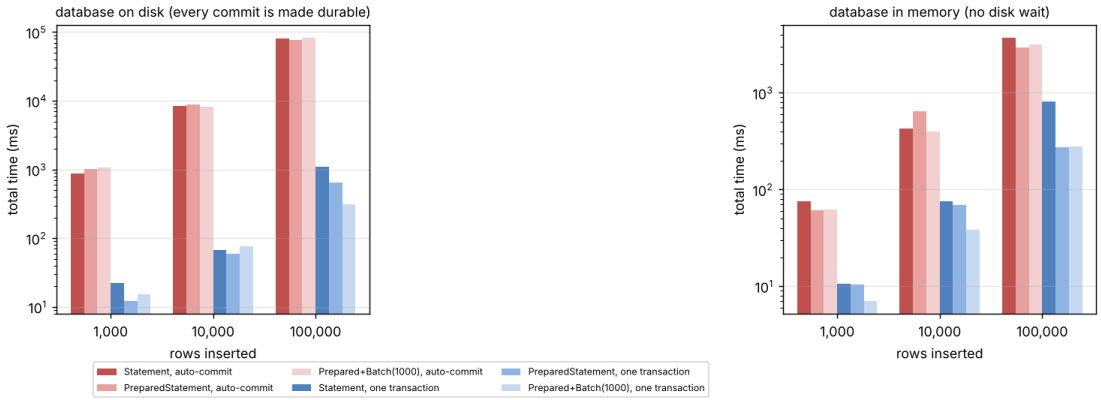


Figure B3.2 – Insert time for the three approaches in both commit modes (logarithmic scale)

Excerpt of the benchmark output: 100,000 rows, one transaction, disk

```
=== Insert n passes (database file in /tmp), time in milliseconds ===
     rows approach               commit mode        total (ms)     us per row
  100,000 Statement              one transaction         1107.7          11.08
  100,000 PreparedStatement      one transaction          646.4           6.46
  100,000 Prepared+Batch(1000)   one transaction          311.6           3.12
```

Interpretation – within the transaction (parse cost and call overhead). For 100,000 rows in one transaction on disk: Statement 1107.7 ms, PreparedStatement 646.4 ms, Prepared+Batch 311.6 ms. The string-built Statement is slowest because the database parses 100,000 different SQL texts; the PreparedStatement parses once and only binds values (1.7× faster); the batch is faster again (2.1×) because it also reduces the per-statement call overhead. All three grow linearly with the number of rows: O(n). **How reliable is the last comparison?** Not very: in the separate run of MultiRowBenchmark (Table B3.5) the plain PreparedStatement loop took 231 ms and the batch 311 ms for 100,000 rows, the opposite order. Differences of about 30% between PreparedStatement and batch are therefore within the run-to-run variation of this machine (JIT compilation, garbage collection, file-system state); the robust results are the two large effects: string-built Statement against PreparedStatement (the parse cost) and auto-commit against one transaction (two orders of magnitude, B3.3). **A caution about the batch:** with this test driver a batch is executed as a loop on the same machine, so no network round trip is saved; with a real MySQL server over a network, batching also removes one network round trip per row (and with rewriteBatchedStatements=true the driver rewrites the batch into multi-row inserts), so the advantage of the batch there is expected to be larger than measured here. I could not measure that.

B3.3 – Auto-commit against one transaction (2 marks)

Speed. The auto-commit rows in Table B3.1 show the dominant effect: with auto-commit every INSERT is its own transaction and must be made durable before the next one starts, which costs about 0.8 ms on this disk, **whichever approach is used** (100,000 rows: Statement 80 s, PreparedStatement 77 s, batch 82 s; all about 0.8 ms per row). Putting the same batch in one transaction made the load **263 times faster** (82.1 s → 0.31 s), because one commit replaces 100,000. With the database in memory the difference shrinks to 11× (Table B3.2): the rest is the per-transaction overhead, which shows that most of the on-disk cost is waiting for the disk, not computing. This also means that the choice between Statement, PreparedStatement and batch **hardly matters when auto-commit is on**: the commit hides it.

Failure half-way. FailureDemo makes the load fail at row 500 of 1,000, in two ways: (1) the amount of row 500 is the text “abc” (a program-level failure); (2) row 500 repeats the pass id of row 10 (the database rejects it). It then counts the rows that remain in the table.

Output of java FailureDemo

```
=== Failure 1: row 500 of 1,000 has a non-numeric amount ("abc") ===
   PassLoader: one transaction, batches of 1,000        -> NumberFormatException: For input string: "abc"
```

```
      rows left in the table: 0   (auto-commit mode afterwards: true)
   auto-commit, one PreparedStatement per row          -> NumberFormatException: For input string: "abc"
      rows left in the table: 500   (auto-commit mode afterwards: true)
   auto-commit, batches of 100                         -> NumberFormatException: For input string: "abc"
      rows left in the table: 500   (auto-commit mode afterwards: true)

=== Failure 2: row 500 of 1,000 repeats the pass id of row 10 (the DATABASE rejects it) ===
   PassLoader: one transaction, batches of 1,000       -> SQLIntegrityConstraintViolationException:
UNIQUE constraint failed: bus_pass.pass_id
      rows left in the table: 0   (auto-commit mode afterwards: true)
   auto-commit, one PreparedStatement per row          -> SQLIntegrityConstraintViolationException:
UNIQUE constraint failed: bus_pass.pass_id
      rows left in the table: 500   (auto-commit mode afterwards: true)
   auto-commit, batches of 100                         -> SQLIntegrityConstraintViolationException:
UNIQUE constraint failed: bus_pass.pass_id
      rows left in the table: 500   (auto-commit mode afterwards: true)

=== Control: no failure ===
      loaded 1000 rows
   PassLoader with 1,000 valid rows                    -> none
      rows left in the table: 1000   (auto-commit mode afterwards: true)
```

Failure	PassLoader (one transaction)	Auto-commit, one PreparedStatement per row	Auto-commit, batches of 100
Row 500 has amount “abc” (NumberFormatException)	0 rows (rolled back)	500 rows left behind	500 rows left behind
Row 500 repeats a pass id (SQLIntegrityConstraintViolationException)	0 rows (rolled back)	500 rows left behind	500 rows left behind
No failure (control)	1000 rows	–	–

Table B3.3 – Rows remaining in bus_pass after a failure half-way

Meaning. With the transaction the table is either completely loaded or untouched, so the import can simply be corrected and run again. Without it, the table holds a **partial load of unknown extent** (500 rows here); running the import again would fail with duplicate keys, and finding out which rows were loaded requires extra work. The completed PassLoader also restores the old auto-commit setting in a finally block, so a failure does not leave the connection in a different mode (printed in the output: “auto-commit mode afterwards: true”).

B3.4 – Engineering trade-off (2 marks)

Situation	Choice	Reason (evidence)
(a) A nightly import of 100,000 passes from a file	PreparedStatement with batches (1,000 rows) inside ONE transaction, as in PassLoader	Measured: 0.3 s against 82 s for the same load with auto-commit (Table B3.1); all-or-nothing (Table B3.3), so a bad file cannot leave a half-loaded table; PreparedStatement also keeps the data out of the SQL text. If the MySQL server and the policy allow it, LOAD DATA INFILE is the usual fastest path (not measured here), but it needs file access on the server and its own error handling.
(b) One pass issued at a counter	A single PreparedStatement INSERT with auto-commit on	One statement is already one atomic transaction, so a transaction adds nothing; the ~0.8 ms durable commit is invisible to a clerk; a batch has nothing to batch. A PreparedStatement is still used for safety (B2). If issuing a pass involves several statements that must succeed together (pass row + payment record + counter), those statements need one transaction (A5).

Table B3.4 – Choice for each situation

Complete the Code – PassLoader.loadAll (Task B3)

The template was completed as follows: TODO 1 sets the five parameters (route as int, amount as double), adds the row to the batch and executes the batch after every 1,000 rows; TODO 2 commits

and returns the number of rows; TODO 3 rolls back every insert of the transaction and re-throws the exception. (The template's SQL text was written on two lines; the missing + between the two string literals was restored.)

TaskB3/PassLoader.java - TODO 1, TODO 2 and TODO 3 completed

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.List;

public class PassLoader {
    /**
     * Loads all passes in one transaction, in batches of 1,000 rows.
     * Returns the number of rows loaded.
     */
    public static int loadAll(Connection con, List<String[]> passes) throws SQLException {
        String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount) "
                   + "VALUES (?, ?, ?, ?, ?)";
        boolean oldAutoCommit = con.getAutoCommit();
        con.setAutoCommit(false);
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            int n = 0;
            for (String[] p : passes) {
                // TODO 1 (completed): set the five parameters from p (route as int, amount as
double),
                // add the row to the batch, and execute the batch after every 1,000 rows
                ps.setString(1, p[0]);
                ps.setString(2, p[1]);
                ps.setString(3, p[2]);
                ps.setInt(4, Integer.parseInt(p[3]));
                ps.setDouble(5, Double.parseDouble(p[4]));
                ps.addBatch();
                n++;
                if (n % 1000 == 0) ps.executeBatch();
            }
            ps.executeBatch();
            // TODO 2 (completed): make all inserted rows permanent and return the number of rows
loaded
            con.commit();
            return n;
        } catch (SQLException | RuntimeException e) {
            // TODO 3 (completed): undo every insert made in this transaction, then re-throw the
exception
            con.rollback();
            throw e;
        } finally {
            con.setAutoCommit(oldAutoCommit);
        }
    }
}
```

Execution. loadAll was run with 1,000, 10,000 and 100,000 passes in LoaderBenchmark (every run checks that the table holds exactly n rows afterwards): 1,000 rows in 15.3 ms; 10,000 rows in 75.5 ms; 100,000 rows in 311.6 ms (database on disk). The invalid-amount case of FailureDemo (above) is the required test with an invalid amount in the middle of the list: the NumberFormatException is a RuntimeException, so it is caught by the second alternative of the catch clause, the rollback leaves **0 rows**, and the exception is re-thrown to the caller.

Alternative Concept and Complexity Analysis – one multi-row INSERT

```
ALGORITHM InsertMultiRow
INPUT   : passes (n rows), b = rows per statement
BEGIN
```

```
  1.  SET sql <- "INSERT INTO bus_pass(...) VALUES " + b copies of "(?, ?, ?, ?, ?)" separated by
commas
  2.  PREPARE statement from sql
  3.  FOR EACH group of b passes DO
  4.      bind 5 * b parameters;  EXECUTE the statement                 // one statement inserts
b rows
  5.  ENDFOR
  6.  insert the remaining rows (fewer than b) with a statement prepared for that size
END
// LOAD DATA (MySQL):  LOAD DATA LOCAL INFILE 'passes.csv' INTO TABLE bus_pass FIELDS TERMINATED
BY ',' LINES TERMINATED BY '\n';
// the server reads the whole file itself: no per-row JDBC call at all
```

Output of java MultiRowBenchmark (all runs in one transaction, database on disk)

```
=== Multi-row INSERT against addBatch (one transaction each, database in /tmp) ===
     rows approach                          time (ms)     us per row client bytes / row
   10,000 addBatch, batches of 1,000             69.6           6.96                653
   10,000 multi-row INSERT, b = 100              53.4           5.34                669
   10,000 multi-row INSERT, b = 500              69.6           6.96                689
   10,000 PreparedStatement per row              84.2           8.42                608
  100,000 addBatch, batches of 1,000            311.2           3.11                652
  100,000 multi-row INSERT, b = 100             230.1           2.30                667
  100,000 multi-row INSERT, b = 500             224.2           2.24                684
  100,000 PreparedStatement per row             231.2           2.31                608

SQL text of ONE multi-row statement with b = 500 rows: 8567 characters (17134 bytes as a Java String),
2,500 parameters
LOAD DATA (MySQL only, not available in this test engine):  LOAD DATA LOCAL INFILE 'passes.csv' INTO
TABLE bus_pass FIELDS TERMINATED BY ',' LINES TERMINATED BY '\n';
```

Rows	Approach	Time (ms)	µs per row	Client bytes allocated per row
10,000	addBatch, batches of 1,000	69.6	6.96	653
10,000	multi-row INSERT, b = 100	53.4	5.34	669
10,000	multi-row INSERT, b = 500	69.6	6.96	689
10,000	PreparedStatement per row	84.2	8.42	608
100,000	addBatch, batches of 1,000	311.2	3.11	652
100,000	multi-row INSERT, b = 100	230.1	2.30	667
100,000	multi-row INSERT, b = 500	224.2	2.24	684
100,000	PreparedStatement per row	231.2	2.31	608

Table B3.5 – Multi-row INSERT against addBatch (from the output above)

Analysis. Time: all variants are O(n); a multi-row statement with b rows needs n/b statement executions instead of n, which saves the per-statement overhead. In this test the four variants differ by less than the run-to-run variation (at 100,000 rows: 231 ms for one PreparedStatement per row, 224–230 ms for the multi-row statements, 311 ms for addBatch; at 10,000 rows the order is different), so **no local speed advantage can be claimed** for batch or multi-row INSERT over a plain prepared loop, because the database and the client share one machine; across a network, n/b round trips against n/1,000 batched round trips would decide, and a larger b reduces them further. **Memory:** a statement with b rows is one SQL text of 8,567 characters for b = 500 (about 17,134 bytes as a Java String) and 5·b = 2,500 parameter slots, i.e. O(b) memory for the statement itself, whereas addBatch keeps b parameter sets (O(b)) in the client until executeBatch; the "client bytes per row" column shows that both cost a few hundred bytes of garbage per row. The database also limits the size of one statement (MySQL: max_allowed_packet; SQLite: at most 32,766 parameters), so b must stay below it. **LOAD DATA** was not measured (not available in the test engine); it avoids all per-row work and is normally the fastest bulk path in MySQL, but it requires file access rights on the server or the client (security setting local_infile) and reports bad rows as warnings rather than exceptions, so the validation and failure report of B4 would have to be done separately. **Evaluation:**

use batches in a transaction as the default; use multi-row INSERT or LOAD DATA only when the measured import time is not acceptable.

Source code of Task B3

TaskB3/PassData.java

```
import java.util.ArrayList;
import java.util.List;

public class PassData {
    /** n generated passes: {passId, regNo, name, route, amount}. */
    public static List<String[]> generate(int n) {
        List<String[]> l = new ArrayList<>(n);
        for (int i = 0; i < n; i++)
            l.add(new String[]{"P" + (100000 + i), String.format("19%07d", i), "Student Name " +
(i % 1000), String.valueOf(1 + i % 40), String.valueOf(3000 + (i % 2000) + 0.5)});
        return l;
    }
}
```

TaskB3/PassInserters.java - Statement, PreparedStatement, batch, multi-row

```
import java.sql.*;
import java.util.List;

/** The insert strategies compared in Task B3. The caller decides the commit mode (autocommit
on/off) before calling. */
public class PassInserters {
    /** (a) Statement: a NEW SQL string for every row (the database parses every statement again).
*/
    public static int statement(Connection c, List<String[]> passes) throws SQLException {
        try (Statement st = c.createStatement()) {
            for (String[] p : passes)
                st.executeUpdate("INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount)
VALUES ('" + p[0] + "', '" + p[1] + "', '" + p[2] + "', " + p[3] + ", " + p[4] + ")");
        }
        return passes.size();
    }

    /** (b) PreparedStatement: parsed once, then bind + execute for every row. */
    public static int prepared(Connection c, List<String[]> passes) throws SQLException {
        try (PreparedStatement ps = c.prepareStatement("INSERT INTO bus_pass(pass_id, reg_no,
name, route_no, amount) VALUES (?, ?, ?, ?, ?)")) {
            for (String[] p : passes) { bind(ps, p); ps.executeUpdate(); }
        }
        return passes.size();
    }

    /** (c) PreparedStatement + addBatch()/executeBatch() every batchSize rows (commit mode
decided by the caller). */
    public static int batch(Connection c, List<String[]> passes, int batchSize) throws
SQLException {
        try (PreparedStatement ps = c.prepareStatement("INSERT INTO bus_pass(pass_id, reg_no,
name, route_no, amount) VALUES (?, ?, ?, ?, ?)")) {
            int n = 0;
            for (String[] p : passes) { bind(ps, p); ps.addBatch(); if (++n % batchSize == 0)
ps.executeBatch(); }
            ps.executeBatch();
            return n;
        }
    }

    /** Alternative: ONE statement inserts b rows:  INSERT ... VALUES (?,?,?,?,?),(?,?,?,?,?),...
*/
    public static int multiRow(Connection c, List<String[]> passes, int b) throws SQLException {
        String one = "(?, ?, ?, ?, ?)";
```

```
        String head = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount) VALUES ";
        int n = passes.size(), full = n / b;
        if (full > 0) {
            String sql = head + String.join(", ", java.util.Collections.nCopies(b, one));
            try (PreparedStatement ps = c.prepareStatement(sql)) {
                for (int k = 0; k < full; k++) { for (int j = 0; j < b; j++) bindAt(ps,
passes.get(k * b + j), j * 5); ps.executeUpdate(); }
            }
        }
        int rest = n - full * b;
        if (rest > 0) {
            String sql = head + String.join(", ", java.util.Collections.nCopies(rest, one));
            try (PreparedStatement ps = c.prepareStatement(sql)) { for (int j = 0; j < rest; j++)
bindAt(ps, passes.get(full * b + j), j * 5); ps.executeUpdate(); }
        }
        return n;
    }

    static void bind(PreparedStatement ps, String[] p) throws SQLException { bindAt(ps, p, 0); }
    static void bindAt(PreparedStatement ps, String[] p, int off) throws SQLException {
        ps.setString(off + 1, p[0]); ps.setString(off + 2, p[1]); ps.setString(off + 3, p[2]);
ps.setInt(off + 4, Integer.parseInt(p[3])); ps.setDouble(off + 5, Double.parseDouble(p[4]));
    }
}
```

TaskB3/Db.java

```
import java.io.File;
import java.nio.file.*;
import java.sql.*;

/** Creates a fresh test database (a new file) with the bus_pass table, using the SQL script. */
public class Db {
    public static String dir = System.getProperty("db.dir", "/tmp");
    public static File newFile(String tag) { File f = new File(dir, "b3_" + tag + "_" +
System.nanoTime() + ".db"); f.deleteOnExit(); return f; }
    public static Connection fresh(File f, String scriptPath) throws Exception {
        minisqlite.MiniSqlite.install();
        Connection c = DriverManager.getConnection("jdbc:minisqlite:" + f.getAbsolutePath());
        try (Statement st = c.createStatement()) { for (String s :
Files.readString(Path.of(scriptPath)).split(";\\s*\\n")) if (!s.isBlank() && !
s.trim().startsWith("--")) st.executeUpdate(s); }
        return c;
    }
    public static int count(Connection c) throws SQLException { try (Statement st =
c.createStatement(); ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM bus_pass")) { rs.next();
return rs.getInt(1); } }
}
```

TaskB3/B3Correctness.java

```
import java.io.File;
import java.sql.*;
import java.util.*;

/** Task B3.1: insert 10 records with each approach, then verify the table and the column types
with a SELECT and ResultSetMetaData. */
public class B3Correctness {
    static String typeName(int t) { for (java.lang.reflect.Field f : Types.class.getFields()) try
{ if (f.getInt(null) == t) return f.getName(); } catch (Exception e) { } return "?"; }

    public static void main(String[] args) throws Exception {
        List<String[]> ten = PassData.generate(10);
        String[] names = {"Statement (string-built SQL)", "PreparedStatement", "PassLoader (batch
+ transaction, completed TODOs)"};
        int pass = 0, fail = 0;
        for (int a = 0; a < 3; a++) {
```

```
            File f = Db.newFile("correct");
            try (Connection c = Db.fresh(f, "bus_pass.sql")) {
                if (a == 0) PassInserters.statement(c, ten); else if (a == 1)
PassInserters.prepared(c, ten); else System.out.println("   PassLoader.loadAll returned " +
PassLoader.loadAll(c, ten) + " and auto-commit is back to " + c.getAutoCommit());
                System.out.println("=== " + names[a] + " ===");
                try (Statement st = c.createStatement(); ResultSet rs = st.executeQuery("SELECT *
FROM bus_pass ORDER BY pass_id")) {
                    ResultSetMetaData md = rs.getMetaData();
                    StringBuilder cols = new StringBuilder("  columns: ");
                    for (int i = 1; i <= md.getColumnCount(); i++)
cols.append(md.getColumnName(i)).append(" ").append(md.getColumnTypeName(i)).append("
(java.sql.Types.").append(typeName(md.getColumnType(i))).append(")").append(i <
md.getColumnCount() ? ", " : "");
                    System.out.println(cols);
                    int row = 0; boolean allEqual = true;
                    while (rs.next()) {
                        String[] e = ten.get(row++);
                        boolean ok = rs.getString("pass_id").equals(e[0]) &&
rs.getString("reg_no").equals(e[1]) && rs.getString("name").equals(e[2])
                                && rs.getInt("route_no") == Integer.parseInt(e[3]) &&
Math.abs(rs.getDouble("amount") - Double.parseDouble(e[4])) < 0.001;
                        if (!ok) allEqual = false;
                        if (row <= 3) System.out.println("  row " + row + ": " + rs.getString(1) +
", " + rs.getString(2) + ", " + rs.getString(3) + ", " + rs.getInt(4) + ", " + rs.getDouble(5));
                    }
                    boolean good = row == 10 && allEqual;
                    if (good) pass++; else fail++;
                    System.out.println("  " + (good ? "PASS" : "FAIL") + ": " + row + " rows
found, every field equals the source value = " + allEqual);
                }
            } finally { f.delete(); }
        }
        System.out.println();
        System.out.println("Result: " + pass + " passed, " + fail + " failed");
    }
}
```

TaskB3/LoaderBenchmark.java

```
import java.io.File;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.sql.*;
import java.util.List;

/**
 * Tasks B3.2 / B3.3: time to insert n passes with Statement, PreparedStatement and
PreparedStatement batches,
 * each with auto-commit ON (every statement is committed separately) and inside ONE transaction.
 * usage: LoaderBenchmark <db directory> <sizes comma separated> <auto|tx|both> <output csv>
 */
public class LoaderBenchmark {
    static final String[] APPROACH = {"Statement", "PreparedStatement", "Prepared+Batch(1000)"};

    static double run(String dir, int n, int approach, boolean transaction) throws Exception {
        Db.dir = dir;
        List<String[]> passes = PassData.generate(n);
        File f = Db.newFile("bench");
        try (Connection c = Db.fresh(f, "bus_pass.sql")) {
            long t = System.nanoTime();
            if (transaction) {
                if (approach == 2) PassLoader.loadAll(c, passes);                // the completed
template: one transaction, batches of 1,000
                else {
                    c.setAutoCommit(false);
```

```
                    if (approach == 0) PassInserters.statement(c, passes); else
PassInserters.prepared(c, passes);
                    c.commit();
                }
            } else {
                c.setAutoCommit(true);
                if (approach == 0) PassInserters.statement(c, passes); else if (approach == 1)
PassInserters.prepared(c, passes); else PassInserters.batch(c, passes, 1000);
            }
            double ms = (System.nanoTime() - t) / 1e6;
            if (Db.count(c) != n) throw new IllegalStateException("row count wrong: " +
Db.count(c) + " instead of " + n);
            return ms;
        } finally { f.delete(); }
    }

    public static void main(String[] args) throws Exception {
        String dir = args[0]; String mode = args[2];
        PrintWriter csv = new PrintWriter(new FileWriter(args[3], true), true);
        System.out.println("=== Insert n passes (database file in " + dir + "), time in
milliseconds ===");
        System.out.printf("%9s %-22s %-14s %14s %14s%n", "rows", "approach", "commit mode", "total
(ms)", "us per row");
        run(dir, 500, 0, true); run(dir, 500, 1, true); run(dir, 500, 2, true);            //
warm-up
        for (String sz : args[1].split(",")) {
            int n = Integer.parseInt(sz.trim());
            for (int a = 0; a < 3; a++) {
                if (args.length > 4 && Integer.parseInt(args[4]) != a) continue;      // optional:
run only one approach
                for (boolean tx : new boolean[]{false, true}) {
                    if (mode.equals("auto") && tx) continue;
                    if (mode.equals("tx") && !tx) continue;
                    int reps = (n <= 1000 && tx) ? 3 : 1;
                    double best = 1e18;
                    for (int r = 0; r < reps; r++) best = Math.min(best, run(dir, n, a, tx));
                    System.out.printf("%,9d %-22s %-14s %14.1f %14.2f%n", n, APPROACH[a], tx ?
"one transaction" : "auto-commit", best, best * 1000 / n);
                    csv.printf("%s,%d,%s,%s,%.2f%n", dir, n, APPROACH[a], tx ? "transaction" :
"autocommit", best);
                }
            }
        }
        csv.close();
    }
}
```

TaskB3/FailureDemo.java

```
import java.io.File;
import java.sql.*;
import java.util.*;

/** Task B3.3: a failure in the middle of the load: how many rows stay in the table with and
without a transaction? */
public class FailureDemo {
    interface Loader { void load(Connection c, List<String[]> l) throws Exception; }
    static int attempt(String title, List<String[]> data, Loader loader) throws Exception {
        File f = Db.newFile("fail");
        try (Connection c = Db.fresh(f, "bus_pass.sql")) {
            String error = "none";
            try { loader.load(c, data); } catch (Exception e) { error =
e.getClass().getSimpleName() + ": " + e.getMessage().replaceAll("\\s+\\[.*", ""); }
            int rows = Db.count(c);
            System.out.printf("   %-52s -> %s%n      rows left in the table: %d   (auto-commit
mode afterwards: %s)%n", title, error, rows, c.getAutoCommit());
```

```
            return rows;
        } finally { f.delete(); }
    }

    public static void main(String[] args) throws Exception {
        System.out.println("=== Failure 1: row 500 of 1,000 has a non-numeric amount (\"abc\")
===");
        List<String[]> d1 = PassData.generate(1000); d1.get(500)[4] = "abc";
        attempt("PassLoader: one transaction, batches of 1,000", d1, (c, l) ->
PassLoader.loadAll(c, l));
        attempt("auto-commit, one PreparedStatement per row", d1, (c, l) ->
PassInserters.prepared(c, l));
        attempt("auto-commit, batches of 100", d1, (c, l) -> PassInserters.batch(c, l, 100));

        System.out.println();
        System.out.println("=== Failure 2: row 500 of 1,000 repeats the pass id of row 10 (the
DATABASE rejects it) ===");
        List<String[]> d2 = PassData.generate(1000); d2.get(500)[0] = d2.get(10)[0];
        attempt("PassLoader: one transaction, batches of 1,000", d2, (c, l) ->
PassLoader.loadAll(c, l));
        attempt("auto-commit, one PreparedStatement per row", d2, (c, l) ->
PassInserters.prepared(c, l));
        attempt("auto-commit, batches of 100", d2, (c, l) -> PassInserters.batch(c, l, 100));

        System.out.println();
        System.out.println("=== Control: no failure ===");
        attempt("PassLoader with 1,000 valid rows", PassData.generate(1000), (c, l) ->
System.out.println("      loaded " + PassLoader.loadAll(c, l) + " rows"));
    }
}
```

TaskB3/MultiRowBenchmark.java

```
import java.io.File;
import java.lang.management.ManagementFactory;
import java.sql.*;
import java.util.List;

/** Alternative of B3: ONE INSERT statement with b rows, compared with addBatch()/executeBatch()
(both in one transaction). */
public class MultiRowBenchmark {
    static long allocated() { com.sun.management.ThreadMXBean b =
(com.sun.management.ThreadMXBean) ManagementFactory.getThreadMXBean(); return
b.getThreadAllocatedBytes(Thread.currentThread().getId()); }
    public static void main(String[] args) throws Exception {
        Db.dir = args.length > 0 ? args[0] : "/tmp";
        System.out.println("=== Multi-row INSERT against addBatch (one transaction each, database
in " + Db.dir + ") ===");
        System.out.printf("%9s %-30s %12s %14s %18s%n", "rows", "approach", "time (ms)", "us per
row", "client bytes / row");
        for (int w = 0; w < 2; w++) for (int a = 0; a < 3; a++) one(2000, a, false);
// warm-up
        for (int n : new int[]{10000, 100000}) {
            for (int a = 0; a < 4; a++) one(n, a, true);
        }
        int b = 500;
        String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount) VALUES " +
String.join(", ", java.util.Collections.nCopies(b, "(?, ?, ?, ?, ?)"));
        System.out.println();
        System.out.println("SQL text of ONE multi-row statement with b = 500 rows: " +
sql.length() + " characters (" + sql.length() * 2 + " bytes as a Java String), 2,500 parameters");
        System.out.println("LOAD DATA (MySQL only, not available in this test engine):  LOAD DATA
LOCAL INFILE 'passes.csv' INTO TABLE bus_pass FIELDS TERMINATED BY ',' LINES TERMINATED BY '\\
n';");
    }
```

```
    static String[] NAMES = {"addBatch, batches of 1,000", "multi-row INSERT, b = 100", "multi-row
INSERT, b = 500", "PreparedStatement per row"};
    static void one(int n, int approach, boolean print) throws Exception {
        List<String[]> passes = PassData.generate(n);
        File f = Db.newFile("multi");
        try (Connection c = Db.fresh(f, "bus_pass.sql")) {
            c.setAutoCommit(false);
            long a0 = allocated(), t = System.nanoTime();
            switch (approach) {
                case 0: PassInserters.batch(c, passes, 1000); break;
                case 1: PassInserters.multiRow(c, passes, 100); break;
                case 2: PassInserters.multiRow(c, passes, 500); break;
                default: PassInserters.prepared(c, passes);
            }
            c.commit();
            double ms = (System.nanoTime() - t) / 1e6; long bytes = allocated() - a0;
            if (Db.count(c) != n) throw new IllegalStateException("wrong count");
            if (print) System.out.printf("%,9d %-30s %12.1f %14.2f %,18d%n", n, NAMES[approach],
ms, ms * 1000 / n, bytes / n);
        } finally { f.delete(); }
    }
}
```

Task B4 – Final Engineering Decision (5 marks, K5)

The transport office will serve **12 campuses, about 100 times more passes (800,000), and several counters issuing passes at the same time.** B4 migrates the existing file, inspects the database, and decides what the production system should use.

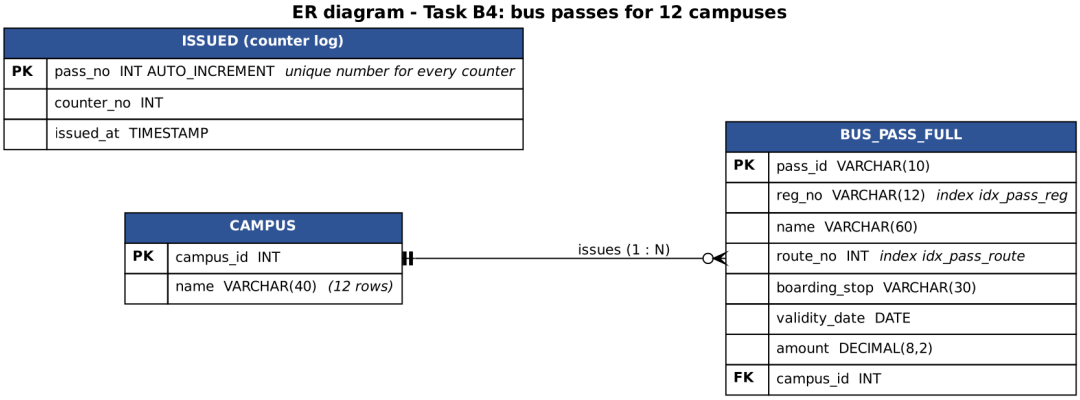


Figure B4.1 – ER diagram of the production database (bus passes, campuses and the counter log)

TaskB4/bus_pass_full.sql

```
-- Task B4: the production table for 12 campuses (MySQL; the same text also runs in SQLite)
DROP TABLE IF EXISTS bus_pass_full;
CREATE TABLE bus_pass_full (
    pass_id       VARCHAR(10)   PRIMARY KEY,
    reg_no        VARCHAR(12)   NOT NULL,
    name          VARCHAR(60)   NOT NULL,
    route_no      INT           NOT NULL,
    boarding_stop VARCHAR(30)   NOT NULL,
    validity_date DATE          NOT NULL,
    amount        DECIMAL(8,2)  NOT NULL,
    campus_id     INT           NOT NULL DEFAULT 1
);
```

TaskB4/bus_pass_indexes.sql

```
CREATE INDEX idx_pass_reg   ON bus_pass_full(reg_no);       -- "find the passes of a student"
CREATE INDEX idx_pass_route ON bus_pass_full(route_no);     -- "list the passes of a route"
```

TaskB4/issued_mysql.sql

```
-- MySQL: the database hands out unique pass numbers to all counters
CREATE TABLE issued (pass_no INT AUTO_INCREMENT PRIMARY KEY, counter_no INT NOT NULL, issued_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

B4.1 – A practical migration strategy (2 marks)

Strategy. The migration reads the existing file line by line (BufferedReader), validates each line with the same rules as the file store (field count, CRC, numbers, date), rejects a line that is too long for its column or repeats a pass id already loaded (a duplicate would violate the PRIMARY KEY and abort the whole load), and inserts the good records with a PreparedStatement in batches of 1,000 **inside one transaction**: any unexpected failure rolls everything back, so the database is never half-loaded (B3.3). Rejected lines are collected with their line number and reason in a **failure report**, and the program finishes by verifying that the database holds exactly the accepted records (row count, number of distinct ids and the sum of the amounts).

MigrationDemo builds an “existing” file of 8,000 good passes with 14 deliberately bad lines (wrong field count, non-numeric amount and route, impossible date, empty line, hand-edited line, negative amount, name too long for the column, an unescaped comma, a cut-off line, and four repeated pass ids) and migrates it:

Output of java MigrationDemo

```
Existing file: 8014 lines (8,000 good, 14 deliberately bad: 4 of them repeat an earlier pass id)

=== B4.2 DatabaseMetaData report (before the migration) ===
Database product  : SQLite (via MiniSqlite test driver)  version 3.45.1
Driver            : MiniSqlite JDBC test driver (java.lang.foreign)  version 0.1
Tables present    :
   bus_pass_full  (TABLE)

=== B4.1 Migration: batches of 1,000 inside one transaction ===
lines read       : 8014
loaded           : 8000
failed validation: 14  (of which duplicate pass ids: 4)
time             : 632 ms
failure report, grouped by reason:
     1 x amount must be positive: #
     1 x bad validity date '..'
     1 x checksum mismatch (line changed or cut off)
     4 x duplicate pass id P#
     1 x empty line
     3 x expected # fields but found #
     1 x name longer than # characters
     2 x not a number: For input string: ".."
failure report, line by line:
   line 501: expected 8 fields but found 3   | P1500,192311500,Short
   line 1202: not a number: For input string: "forty"   | P9001,192319001,Ravi,3,Main Gate,2027-06-
30,forty,f55ad2a8
   line 2003: not a number: For input string: "abc"   | P9002,192319002,Ravi,abc,Main Gate,2027-06-
30,4500.00,34b8627e
   line 3004: bad validity date '2027-13-45'   | P9003,192319003,Ravi,3,Main Gate,2027-13-
45,4500.00,15b1858c
   line 3505: empty line   |
   line 4006: checksum mismatch (line changed or cut off)   | P1011,190000010,Studant 10,11,Stop 10,2027-
06-30,3010.00,b111102b
   line 4507: amount must be positive: -50.00   | P9004,192319004,Ravi,3,Main Gate,2027-06-30,-
50.00,bc2c33ec
   line 5008: name longer than 60 characters   |
P9005,192319005,NNNN
```

B4.2 – Inspect the database (1 mark)

DbInspector uses DatabaseMetaData to print the database product and version, the driver name and version, and the tables that exist (see the two reports at the beginning and at the end of the migration output above: the table bus_pass_full appears after it has been created).

Why this is useful. (1) Portability and diagnostics: the program can log exactly which database and driver version it is talking to; most production problems ("works on my machine") are version problems, and a support engineer needs this line first. (2) Self-checks: the program can verify at start-up that the expected tables exist before it tries to migrate or issue passes, and refuse to run (or create them) with a clear message instead of failing in the middle of a transaction. (3) Adapting to the database: behaviour that differs between products (limits, supported features) can be selected from the reported product instead of hard-coded. (4) Generic tools: a report generator or an export function can discover tables and columns (with getColumns and ResultSetMetaData) without knowing the schema. With MySQL the first two lines would read "MySQL" and its version, and the driver line "MySQL Connector/J"; here they read SQLite and the test driver (see the limitations).

B4.3 – The production decision (2 marks)

ScaleBenchmark generates **800,000 passes** (100 × 8,000) and measures the three candidate designs: (A) one text file, (B) one serialized file per campus (12 files), (C) one database table with indexes. The concurrency question is investigated by ConcurrencyDemo.

Output of java ScaleBenchmark

```
=== 12 campuses, N = 800,000 passes ===

(A) ONE TEXT FILE
    file size 54,558,528 bytes | write 3244 ms | find(last pass) 781 ms | list one route (20000 passes)
895 ms

(B) ONE SERIALIZED FILE PER CAMPUS (12 files, each file read by one campus program)
    total size 63,594,628 bytes | write all 2814 ms | LOAD all 12 files 2684 ms | heap needed 216 MB
    search ALL passes for one id (scan the 12 lists in memory) 51.1 ms | one route 10.8 ms (20000 passes)
    with an extra HashMap index: build 112 ms, +25 MB heap, then 1409 ns per lookup

(C) ONE DATABASE (table bus_pass_full, campus_id column)
    load all rows (batches of 1,000, one transaction) 3390 ms | database size 65,073,152 bytes
(87,486,464 with indexes, index build 638 ms)
    find one pass by pass_id (PRIMARY KEY index) 0.025 ms | by reg_no WITHOUT index 51.9 ms | by reg_no
WITH index 0.006 ms
    list one route (20,000 rows) 163 ms (with index on route_no) | count of one campus (66,667) 58 ms
(checksum 2256024)
```

Design (800,000 passes)	Find ONE pass	List ONE route (20,000 passes)	Size on disk	Setup cost
(A) one text file	781.4 ms (scan)	895.4 ms (scan)	54,558,528 bytes	none; but every search reads the whole file
(B) 12 serialized files (all loaded into memory)	51.1 ms (scan of all lists in memory); 1409 ns with an extra HashMap	10.8 ms	63,594,628 bytes	load all files 2,684 ms; 216 MB of heap (plus 25 MB for the map)
(C) database with indexes	0.03 ms (primary key); by student number 51.9 ms without an index, 0.006 ms with one	163.2 ms	65,073,152 bytes (before the indexes)	load 3,390 ms once; nothing to load afterwards

Table B4.1 – Measured behaviour at the new scale (from TaskB4/scale_results.csv and the output above)

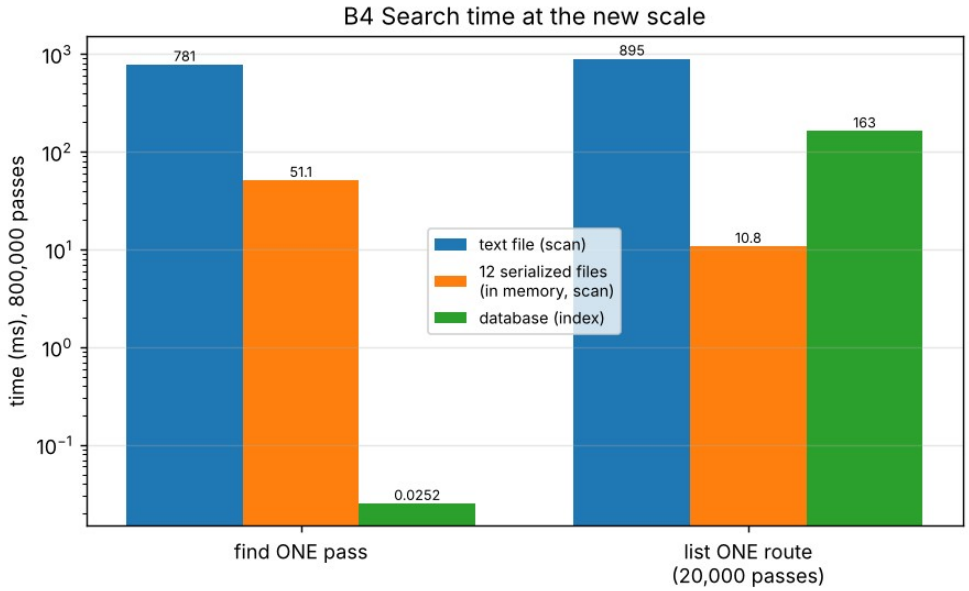


Figure B4.2 – Search time at 800,000 passes (logarithmic axis)

Concurrency. Four counters issue 500 passes each at the same moment; every pass must get a unique number and no record may be damaged.

Output of java ConcurrencyDemo

```
=== 4 counters issue 500 passes each at the same moment (expected: 2000 passes, all numbers different)
===
(1) text file, no coordination (run 1)
      lines: 2000 | distinct pass numbers: 1981 | duplicate numbers: 19 | damaged lines: 0
(1) text file, no coordination (run 2)
      lines: 2000 | distinct pass numbers: 1990 | duplicate numbers: 10 | damaged lines: 0
(1) text file, no coordination (run 3)
      lines: 2000 | distinct pass numbers: 1999 | duplicate numbers: 1 | damaged lines: 0
(2) text file + synchronized (ONE program only)
      lines: 2000 | distinct pass numbers: 2000 | duplicate numbers: 0 | damaged lines: 0
(3) database, each counter has its own connection and the DATABASE assigns the number
      passes issued: 2000 | rows in the table: 2000 | distinct numbers: 2000 | duplicates: 0 | time 2092
ms
```

How the number is created	Passes issued	Distinct numbers	Duplicate numbers
text file, no coordination – run 1	2,000	1,981	19
text file, no coordination – run 2	2,000	1,990	10
text file, no coordination – run 3	2,000	1,999	1
text file + synchronized (ONE program only)	2,000	2,000	0
database, each counter has its own connection and the DATABASE assigns the number	2,000	2,000	0

Table B4.2 – Four counters, 2,000 passes (executed)

Decision: a relational database (MySQL) with an index on the primary key and on reg_no and route_no, accessed through PreparedStatement, with one transaction per issue and the bulk loads of B3/B4.1. The evidence:

- Search at scale: finding one pass in the text file takes 781.4 ms and grows with every pass (O(n)); the database needs 0.03 ms by primary key (O(log n)), i.e. 31,008 times less, and a lookup by student number fell from 51.9 ms to 0.006 ms when the index was added. A clerk can tolerate 0.8 s once, but twelve campuses searching all day cannot.
- Memory: design (B) must load the data of all campuses into the program (216 MB here, growing with the passes); the database keeps the data on disk and returns only the rows asked for.

- Concurrency: without coordination the text file gave **duplicate pass numbers** (the counters read the same line count and wrote the same number; the count differs from run to run, which is exactly why such a defect passes testing). A lock inside one program (synchronized) removed the duplicates, but it protects only counters that are threads of the same program; 12 campuses are separate machines and programs. The database assigns the number itself (AUTO_INCREMENT) and gave 2,000 distinct numbers and no damaged rows, with each counter on its own connection.
- Correctness and safety: transactions (B3) make an issue all-or-nothing, constraints (PRIMARY KEY, NOT NULL, CHECK) reject bad data that the file store had to detect with a checksum, and PreparedStatement (B2) removes the injection risk.
- Evolution: a new column is an ALTER TABLE and not a change of file format for twelve programs (A4).

Costs of this decision (and when it would be wrong): a database server has to be installed, backed up and secured; every counter needs network access to it; the team needs SQL knowledge; a power cut of the server stops all campuses. For a single counter with fewer than about 10,000 passes (the original situation, 1.5 ms per search in Table B1.4) the text file is still entirely adequate and simpler. The text export remains useful as a backup and as the migration source.

Alternative Concept and Complexity Analysis – one serialized file per campus

The alternative solution. Each campus saves its List<BusPass> with ObjectOutputStream into its own file and loads it at start-up (design B above). Cross-campus questions load all twelve files.

```
ALGORITHM SearchAllCampuses
INPUT   : passId
BEGIN
  1.  FOR campus FROM 1 TO 12 DO
  2.      IF the list of that campus is not in memory THEN  READ the campus file with
ObjectInputStream  ENDIF      // O(n_c) time and memory
  3.      FOR EACH pass IN the list DO  IF pass.passId = passId THEN RETURN pass  ENDIF  ENDFOR
// O(n_c)
  4.  ENDFOR
  5.  RETURN "not found"
// O(N) in total, N = 800,000
END
ALGORITHM SavePass:  append the pass to the list in memory, then WRITE THE WHOLE LIST again with
writeObject      // O(n_c) per change
```

	(B) Serialized file per campus	(C) Database
Search for one pass (N = 800,000)	O(N) scan of the lists: 51.1 ms in memory; O(1) only with an extra HashMap (1409 ns, but +25 MB and a build time of 112 ms)	O(log N): 0.03 ms by primary key
Start-up	load 12 files: 2,684 ms (O(N))	none (connect only)
Memory	O(N): 216 MB heap for 800,000 passes	O(1) per query (the server holds a cache)
Save one new pass	rewrite the whole campus file: O(n_c) (writing all 12 files took 2,814 ms)	one INSERT: O(log N)
Several counters of one campus at the same time	each counter has its own copy in memory; two counters writing the file overwrite each other's passes (lost updates) unless one program serializes all access	handled by the database (row locks, transactions)
Counters of different campuses	independent files, so no conflicts between campuses, but no shared view: a pass issued at campus 3 is invisible to campus 5 until files are exchanged	one shared, consistent view
A damaged file	a damaged serialized file is unreadable as a whole (one bad byte can lose all of that campus's passes)	the database recovers committed transactions after a crash
Evolving the class	old files break unless serialVersionUID is managed (A4.2)	ALTER TABLE

Table B4.3 – Serialized files per campus against the database (measured values from Table B4.1 and ConcurrencyDemo)

Conclusion. Per-campus serialization works only if each campus has **one** program that owns its file and the questions never cross campuses. For the stated requirements (12 campuses, several counters at once, searches by pass id, student and route) the database is the only one of the three designs that satisfies all of them at the measured scale. The measurements also give the limits of the other designs: the single text file stops being adequate at roughly 100,000 passes (about 100 ms per search); per-campus serialization costs a heap proportional to the whole organisation and loses updates as soon as two counters share a campus.

Source code of Task B4

TaskB4/MigrationReport.java

```
import java.util.*;

public class MigrationReport {
    public int linesRead, loaded, rejected, duplicates;
    public long millis;
    public final Map<String, Integer> reasons = new TreeMap<>();
    public final List<String> rejectLines = new ArrayList<>();
    public void reject(int lineNo, String reason, String raw) {
        rejected++;
        String key = reason.replaceAll("[0-9.\\-]+", "#").replaceAll("'[^']*'",
"'..'").replaceAll("\"[^\"]*\"", "\"..\"");
        reasons.merge(key, 1, Integer::sum);
        rejectLines.add("line " + lineNo + ": " + reason + "   | " + (raw.length() > 70 ?
raw.substring(0, 70) + "..." : raw));
    }
}
```

TaskB4/Migration.java - validation + batches + one transaction + failure report

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.*;
import java.util.*;

/**
 * Task B4.1: reads the existing pass file, validates every line, and loads the good records into
bus_pass_full
 * in batches of 1,000 inside ONE transaction. Lines that fail validation are collected in a
failure report.
 */
public class Migration {
    static final String INSERT = "INSERT INTO bus_pass_full(pass_id, reg_no, name, route_no,
boarding_stop, validity_date, amount, campus_id) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";

    public static MigrationReport run(Path passFile, Connection con, int campusId) throws
IOException, SQLException {
        MigrationReport rep = new MigrationReport();
        long t0 = System.nanoTime();
        Set<String> seen = new HashSet<>();                                    // duplicate pass
ids would violate the PRIMARY KEY
        boolean old = con.getAutoCommit();
        con.setAutoCommit(false);
        try (BufferedReader r = Files.newBufferedReader(passFile, StandardCharsets.UTF_8);
PreparedStatement ps = con.prepareStatement(INSERT)) {
            String line; int lineNo = 0, inBatch = 0;
            while ((line = r.readLine()) != null) {
                lineNo++; rep.linesRead++;
                BusPass p;
                try { p = BusPass.fromLine(line); } catch (DamagedRecordException e)
{ rep.reject(lineNo, e.getMessage(), line); continue; }
```

```
                String tooLong = tooLongForColumns(p);
                if (tooLong != null) { rep.reject(lineNo, tooLong, line); continue; }
                if (!seen.add(p.passId)) { rep.duplicates++; rep.reject(lineNo, "duplicate pass id
" + p.passId, line); continue; }
                ps.setString(1, p.passId); ps.setString(2, p.regNo); ps.setString(3, p.name);
ps.setInt(4, p.route);
                ps.setString(5, p.stop); ps.setString(6, p.validity.toString()); ps.setDouble(7,
p.amount); ps.setInt(8, campusId);
                ps.addBatch(); rep.loaded++;
                if (++inBatch == 1000) { ps.executeBatch(); inBatch = 0; }
            }
            ps.executeBatch();
            con.commit();                                                   // all good records
become permanent together
        } catch (SQLException | IOException | RuntimeException e) {
            con.rollback();                                                 // nothing is half-
loaded
            throw e;
        } finally { con.setAutoCommit(old); }
        rep.millis = (System.nanoTime() - t0) / 1_000_000;
        return rep;
    }

    /** the table columns are VARCHAR(n): a longer value would be cut or rejected by the database,
so it is reported here */
    static String tooLongForColumns(BusPass p) {
        if (p.passId.length() > 10) return "pass id longer than 10 characters";
        if (p.regNo.length() > 12) return "register number longer than 12 characters";
        if (p.name.length() > 60) return "name longer than 60 characters";
        if (p.stop.length() > 30) return "boarding stop longer than 30 characters";
        return null;
    }
}
```

TaskB4/DbInspector.java

```
import java.sql.*;

/** Task B4.2: report the database product, the driver version and the tables, using
DatabaseMetaData. */
public class DbInspector {
    public static void report(Connection c) throws SQLException {
        DatabaseMetaData md = c.getMetaData();
        System.out.println("Database product  : " + md.getDatabaseProductName() + "  version " +
md.getDatabaseProductVersion());
        System.out.println("Driver           : " + md.getDriverName() + "  version " +
md.getDriverVersion());
        System.out.println("Tables present    :");
        try (ResultSet rs = md.getTables(null, null, "%", new String[]{"TABLE"})) {
            while (rs.next()) System.out.println("   " + rs.getString("TABLE_NAME") + "  (" +
rs.getString("TABLE_TYPE") + ")");
        }
    }
}
```

TaskB4/MigrationDemo.java

```
import java.io.File;
import java.nio.file.*;
import java.sql.*;
import java.time.LocalDate;
import java.util.*;
import java.util.zip.CRC32;

/** Task B4.1 / B4.2: migrate an existing pass file that contains some bad lines and print the
failure report. */
public class MigrationDemo {
```

```
    static String withCrc(String body) { CRC32 c = new CRC32(); c.update(body.getBytes()); return
body + "," + Long.toHexString(c.getValue()); }

    public static void main(String[] args) throws Exception {
        minisqlite.MiniSqlite.install();
        // ----- the existing file: 8,000 good passes + 14 bad lines of different kinds -----
        Path file = Files.createTempFile("existing_passes", ".txt");
        List<String> lines = new ArrayList<>();
        for (int i = 0; i < 8000; i++) lines.add(new BusPass("P" + (1001 + i),
String.format("19%07d", i), "Student " + (i % 500), 1 + i % 40, "Stop " + (i % 25),
LocalDate.of(2027, 6, 30), 3000 + i % 2000).toLine());
        String[][] bad = {{"500", "P1500,192311500,Short"}, {"1200",
withCrc("P9001,192319001,Ravi,3,Main Gate,2027-06-30,forty")}, {"2000",
withCrc("P9002,192319002,Ravi,abc,Main Gate,2027-06-30,4500.00")},
            {"3000", withCrc("P9003,192319003,Ravi,3,Main Gate,2027-13-45,4500.00")}, {"3500",
""}, {"4000", lines.get(10).replace("Student", "Studant")},
            {"4500", withCrc("P9004,192319004,Ravi,3,Main Gate,2027-06-30,-50.00")}, {"5000",
withCrc("P9005,192319005," + "N".repeat(70) + ",3,Main Gate,2027-06-30,4500.00")},
            {"5500", lines.get(7)}, {"6000", lines.get(8)}, {"6500", lines.get(9)}, {"7000",
lines.get(10)}, {"7500", withCrc("P9006,192319006,Kumar, Meena,3,Main Gate,2027-06-30,4500.00")},
{"7900", lines.get(11).substring(0, 40)}};
        for (int k = bad.length - 1; k >= 0; k--) lines.add(Integer.parseInt(bad[k][0]), bad[k]
[1]);       // insert from the back so positions stay valid
        Files.write(file, lines);
        System.out.println("Existing file: " + lines.size() + " lines (8,000 good, 14 deliberately
bad: 4 of them repeat an earlier pass id)");

        File dbFile = File.createTempFile("campus", ".db"); dbFile.delete();
        try (Connection c = DriverManager.getConnection("jdbc:minisqlite:" +
dbFile.getAbsolutePath())) {
            try (Statement st = c.createStatement()) { for (String s :
Files.readString(Path.of("bus_pass_full.sql")).split(";\\s*\\n")) if (!s.isBlank() && !
s.trim().startsWith("--")) st.executeUpdate(s); }
            System.out.println();
            System.out.println("=== B4.2 DatabaseMetaData report (before the migration) ===");
            DbInspector.report(c);

            System.out.println();
            System.out.println("=== B4.1 Migration: batches of 1,000 inside one transaction ===");
            MigrationReport rep = Migration.run(file, c, 1);
            System.out.println("lines read       : " + rep.linesRead);
            System.out.println("loaded           : " + rep.loaded);
            System.out.println("failed validation: " + rep.rejected + "  (of which duplicate pass
ids: " + rep.duplicates + ")");
            System.out.println("time             : " + rep.millis + " ms");
            System.out.println("failure report, grouped by reason:");
            for (Map.Entry<String, Integer> e : rep.reasons.entrySet()) System.out.printf("   %3d
x %s%n", e.getValue(), e.getKey());
            System.out.println("failure report, line by line:");
            for (String s : rep.rejectLines) System.out.println("   " + s);

            System.out.println();
            System.out.println("=== Verification: the database must contain exactly what was
accepted ===");
            try (Statement st = c.createStatement(); ResultSet rs = st.executeQuery("SELECT
COUNT(*), SUM(amount), COUNT(DISTINCT pass_id) FROM bus_pass_full")) {
                rs.next();
                double expectedSum = 0; for (int i = 0; i < 8000; i++) expectedSum += 3000 + i %
2000;
                System.out.println("rows in database = " + rs.getInt(1) + " (expected " +
rep.loaded + "), distinct pass ids = " + rs.getInt(3) + ", sum of amounts = " + rs.getDouble(2) +
" (expected " + expectedSum + ")");
                System.out.println((rs.getInt(1) == rep.loaded && rs.getInt(1) == 8000 &&
Math.abs(rs.getDouble(2) - expectedSum) < 0.01) ? "PASS: file and database agree" : "FAIL");
            }
```

```
            try (Statement st = c.createStatement(); ResultSet rs = st.executeQuery("SELECT
route_no, COUNT(*) FROM bus_pass_full WHERE route_no <= 3 GROUP BY route_no ORDER BY route_no")) {
                StringBuilder sb = new StringBuilder("passes per route (routes 1-3): ");
                while (rs.next()) sb.append("route ").append(rs.getInt(1)).append(" =
").append(rs.getInt(2)).append("   ");
                System.out.println(sb);
            }
            System.out.println();
            System.out.println("=== The same report AFTER the migration (tables present) ===");
            DbInspector.report(c);
        }
        dbFile.delete(); Files.delete(file);
    }
}
```

TaskB4/ScaleBenchmark.java

```
import java.io.*;
import java.nio.file.*;
import java.sql.*;
import java.time.LocalDate;
import java.util.*;

/**
 * Tasks B4.3 and the alternative: 12 campuses, N = 800,000 passes (100 x 8,000).
 *   (A) one text file            (B) one serialized file per campus            (C) one database
(indexed)
 */
public class ScaleBenchmark {
    static long sink = 0;
    static final int N = Integer.getInteger("n", 800_000), CAMPUSES = 12;
    static BusPass make(int i) { return new BusPass("P" + (100001 + i), String.format("19%07d",
i), "Student " + (i % 1000), 1 + i % 40, "Stop " + (i % 25), LocalDate.of(2027, 6, 30), 3000 + i %
2000); }
    static long used() { for (int i = 0; i < 3; i++) System.gc(); Runtime r =
Runtime.getRuntime(); return r.totalMemory() - r.freeMemory(); }
    static double ms(long t0) { return (System.nanoTime() - t0) / 1e6; }

    public static void main(String[] args) throws Exception {
        minisqlite.MiniSqlite.install();
        PrintWriter csv = new PrintWriter("scale_results.csv");
        csv.println("design,measure,value,unit");
        System.out.println("=== 12 campuses, N = " + String.format("%,d", N) + " passes ===");
        Path dir = Files.createTempDirectory("scale");
        String lastId = "P" + (100001 + N - 1), midReg = String.format("19%07d", N / 2);

        // ---------------- (A) one text file ----------------
        Path text = dir.resolve("passes.txt");
        long t = System.nanoTime();
        try (BufferedWriter w = Files.newBufferedWriter(text)) { for (int i = 0; i < N; i++)
w.write(make(i).toLine() + "\n"); }
        double writeA = ms(t);
        BusPassStore store = new BusPassStore(text);
        for (int i = 0; i < 2; i++) sink += store.find(lastId) != null ? 1 : 0;
        t = System.nanoTime(); for (int i = 0; i < 3; i++) sink += store.find(lastId) != null ?
1 : 0; double findA = ms(t) / 3;
        t = System.nanoTime(); for (int i = 0; i < 3; i++) sink += store.listByRoute(7).size();
double routeA = ms(t) / 3;
        long bytesA = Files.size(text);
        System.out.println();
        System.out.println("(A) ONE TEXT FILE");
        System.out.printf("    file size %,d bytes | write %.0f ms | find(last pass) %.0f ms |
list one route (%d passes) %.0f ms%n", bytesA, writeA, findA, store.listByRoute(7).size(),
routeA);
        csv.printf("text file,size,%d,bytes%ntext file,find one pass,%.1f,ms%ntext file,list one
route,%.1f,ms%n", bytesA, findA, routeA);
```

```
        // ---------------- (B) one serialized file per campus ----------------
        List<List<BusPass>> perCampus = new ArrayList<>();
        for (int c = 0; c < CAMPUSES; c++) perCampus.add(new ArrayList<>());
        for (int i = 0; i < N; i++) perCampus.get(i % CAMPUSES).add(make(i));
        t = System.nanoTime(); long bytesB = 0;
        for (int c = 0; c < CAMPUSES; c++) {
            Path f = dir.resolve("campus" + (c + 1) + ".ser");
            try (ObjectOutputStream o = new ObjectOutputStream(new
BufferedOutputStream(Files.newOutputStream(f)))) { o.writeObject(perCampus.get(c)); }
            bytesB += Files.size(f);
        }
        double writeB = ms(t);
        perCampus = null;
        long before = used();
        t = System.nanoTime();
        List<List<BusPass>> loaded = new ArrayList<>();
        for (int c = 0; c < CAMPUSES; c++) try (ObjectInputStream in = new ObjectInputStream(new
BufferedInputStream(Files.newInputStream(dir.resolve("campus" + (c + 1) + ".ser"))))
{ @SuppressWarnings("unchecked") List<BusPass> l = (List<BusPass>) in.readObject(); loaded.add(l);
}
        double loadB = ms(t);
        double heapB = (used() - before) / 1048576.0;
        t = System.nanoTime(); for (int r = 0; r < 5; r++) for (List<BusPass> l : loaded) for
(BusPass p : l) if (p.passId.equals(lastId)) sink++; double searchB = ms(t) / 5;
        t = System.nanoTime(); int cnt = 0; for (List<BusPass> l : loaded) for (BusPass p : l) if
(p.route == 7) cnt++; double routeB = ms(t);
        t = System.nanoTime(); Map<String, BusPass> index = new HashMap<>(2 * N); for
(List<BusPass> l : loaded) for (BusPass p : l) index.put(p.passId, p); double buildIdx = ms(t);
        double heapIdx = (used() - before) / 1048576.0 - heapB;
        t = System.nanoTime(); for (int i = 0; i < 100000; i++) sink += index.get("P" + (100001 +
(i * 7919) % N)).route; double lookupB = (System.nanoTime() - t) / 1e5;
        System.out.println();
        System.out.println("(B) ONE SERIALIZED FILE PER CAMPUS (12 files, each file read by one
campus program)");
        System.out.printf("    total size %,d bytes | write all %.0f ms | LOAD all 12 files %.0f
ms | heap needed %.0f MB%n", bytesB, writeB, loadB, heapB);
        System.out.printf("    search ALL passes for one id (scan the 12 lists in memory) %.1f ms
| one route %.1f ms (%d passes)%n", searchB, routeB, cnt);
        System.out.printf("    with an extra HashMap index: build %.0f ms, +%.0f MB heap, then
%.0f ns per lookup%n", buildIdx, heapIdx, lookupB);
        csv.printf("per-campus serialization,size,%d,bytes%nper-campus serialization,load all,
%.1f,ms%nper-campus serialization,heap,%.1f,MB%nper-campus serialization,search all for one id,
%.1f,ms%nper-campus serialization,list one route,%.1f,ms%n", bytesB, loadB, heapB, searchB,
routeB);
        loaded = null; index = null;

        // ---------------- (C) one database ----------------
        File dbFile = new File(dir.toFile(), "campus.db");
        try (Connection c = DriverManager.getConnection("jdbc:minisqlite:" +
dbFile.getAbsolutePath())) {
            try (Statement st = c.createStatement()) { for (String s :
Files.readString(Path.of("bus_pass_full.sql")).split(";\\s*\\n")) if (!s.isBlank() && !
s.trim().startsWith("--")) st.executeUpdate(s); }
            t = System.nanoTime();
            c.setAutoCommit(false);
            try (PreparedStatement ps = c.prepareStatement(Migration.INSERT)) {
                for (int i = 0; i < N; i++) {
                    BusPass p = make(i);
                    ps.setString(1, p.passId); ps.setString(2, p.regNo); ps.setString(3, p.name);
ps.setInt(4, p.route); ps.setString(5, p.stop); ps.setString(6, p.validity.toString());
ps.setDouble(7, p.amount); ps.setInt(8, 1 + i % CAMPUSES);
                    ps.addBatch(); if ((i + 1) % 1000 == 0) ps.executeBatch();
                }
                ps.executeBatch();
```

```
            }
            c.commit(); c.setAutoCommit(true);
            double load = ms(t);
            long bytesC = dbFile.length();
            double pk = timeQuery(c, "SELECT * FROM bus_pass_full WHERE pass_id = ?", lastId,
3000);
            double regNoIdx = timeQuery(c, "SELECT * FROM bus_pass_full WHERE reg_no = ?", midReg,
5);
            t = System.nanoTime(); try (Statement st = c.createStatement()) { for (String s :
Files.readString(Path.of("bus_pass_indexes.sql")).split(";\\s*\\n")) if (!s.isBlank())
st.executeUpdate(s); } double idxBuild = ms(t);
            long bytesIdx = dbFile.length();
            double regIdx = timeQuery(c, "SELECT * FROM bus_pass_full WHERE reg_no = ?", midReg,
3000);
            t = System.nanoTime(); int rows = 0; try (PreparedStatement ps =
c.prepareStatement("SELECT pass_id, name FROM bus_pass_full WHERE route_no = ?")) { ps.setInt(1,
7); try (ResultSet rs = ps.executeQuery()) { while (rs.next()) { rows++; sink +=
rs.getString(1).length(); } } } double routeC = ms(t);
            t = System.nanoTime(); int campusCount; try (Statement st = c.createStatement();
ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM bus_pass_full WHERE campus_id = 5"))
{ rs.next(); campusCount = rs.getInt(1); } double campusC = ms(t);
            System.out.println();
            System.out.println("(C) ONE DATABASE (table bus_pass_full, campus_id column)");
            System.out.printf("    load all rows (batches of 1,000, one transaction) %.0f ms |
database size %,d bytes (%,d with indexes, index build %.0f ms)%n", load, bytesC, bytesIdx,
idxBuild);
            System.out.printf("    find one pass by pass_id (PRIMARY KEY index) %.3f ms | by
reg_no WITHOUT index %.1f ms | by reg_no WITH index %.3f ms%n", pk, regNoIdx, regIdx);
            System.out.printf("    list one route (%,d rows) %.0f ms (with index on route_no) |
count of one campus (%,d) %.0f ms%n", rows, routeC, campusCount, campusC);
            csv.printf("database,size,%d,bytes%ndatabase,load all,%.1f,ms%ndatabase,find by
primary key,%.4f,ms%ndatabase,find by reg_no without index,%.2f,ms%ndatabase,find by reg_no with
index,%.4f,ms%ndatabase,list one route,%.1f,ms%n", bytesC, load, pk, regNoIdx, regIdx, routeC);
        }
        csv.close();
        System.out.println("(checksum " + sink + ")");
        try (var s = Files.walk(dir)) { s.sorted(Comparator.reverseOrder()).forEach(p ->
p.toFile().delete()); }
    }

    static double timeQuery(Connection c, String sql, String arg, int reps) throws SQLException {
        try (PreparedStatement ps = c.prepareStatement(sql)) {
            for (int i = 0; i < Math.min(reps, 3); i++) { ps.setString(1, arg); try (ResultSet rs
= ps.executeQuery()) { while (rs.next()) sink++; } }
            long t = System.nanoTime();
            for (int i = 0; i < reps; i++) { ps.setString(1, arg); try (ResultSet rs =
ps.executeQuery()) { while (rs.next()) sink++; } }
            return (System.nanoTime() - t) / 1e6 / reps;
        }
    }
}
```

TaskB4/ConcurrencyDemo.java

```
import java.io.File;
import java.nio.file.*;
import java.sql.*;
import java.util.*;
import java.util.concurrent.*;

/** Task B4.3 / alternative: several counters issue passes at the same time. Who gives each new
pass a UNIQUE number? */
public class ConcurrencyDemo {
    static final int COUNTERS = 4, PER_COUNTER = 500;
    static final Object LOCK = new Object();
```

```
    public static void main(String[] args) throws Exception {
        minisqlite.MiniSqlite.install();
        System.out.println("=== " + COUNTERS + " counters issue " + PER_COUNTER + " passes each at
the same moment (expected: " + COUNTERS * PER_COUNTER + " passes, all numbers different) ===");

        // ---- (1) text file, "number = lines in the file + 1", no coordination ----
        for (int attempt = 1; attempt <= 3; attempt++) {
            Path f = Files.createTempFile("issue", ".txt");
            run((counter, i) -> { long next = Files.readAllLines(f).size() + 1;
Files.writeString(f, "P" + next + ",counter" + counter + "\n", StandardOpenOption.APPEND); });
            report("(1) text file, no coordination (run " + attempt + ")", Files.readAllLines(f));
            Files.delete(f);
        }
        // ---- (2) text file with a lock inside ONE program (all counters are threads of the same
JVM) ----
        Path f2 = Files.createTempFile("issue", ".txt");
        run((counter, i) -> { synchronized (LOCK) { long next = Files.readAllLines(f2).size() + 1;
Files.writeString(f2, "P" + next + ",counter" + counter + "\n", StandardOpenOption.APPEND); } });
        report("(2) text file + synchronized (ONE program only)", Files.readAllLines(f2));
        Files.delete(f2);

        // ---- (3) database: the database creates the number (AUTO_INCREMENT / INTEGER PRIMARY
KEY) ----
        File db = File.createTempFile("issued", ".db"); db.delete();
        try (Connection c = DriverManager.getConnection("jdbc:minisqlite:" +
db.getAbsolutePath()); Statement st = c.createStatement()) {
            st.executeUpdate("CREATE TABLE issued (pass_no INTEGER PRIMARY KEY, counter_no INT NOT
NULL)");
        }
        List<Long> keys = Collections.synchronizedList(new ArrayList<>());
        long t = System.nanoTime();
        runDb(db, keys);
        double ms = (System.nanoTime() - t) / 1e6;
        Set<Long> distinct = new HashSet<>(keys);
        int rows;
        try (Connection c = DriverManager.getConnection("jdbc:minisqlite:" +
db.getAbsolutePath()); Statement st = c.createStatement(); ResultSet rs = st.executeQuery("SELECT
COUNT(*) FROM issued")) { rs.next(); rows = rs.getInt(1); }
        System.out.printf("(3) database, each counter has its own connection and the DATABASE
assigns the number%n      passes issued: %d | rows in the table: %d | distinct numbers: %d |
duplicates: %d | time %.0f ms%n", keys.size(), rows, distinct.size(), keys.size() -
distinct.size(), ms);
        db.delete();
    }

    interface Issue { void issue(int counter, int i) throws Exception; }
    static void run(Issue body) throws Exception {
        ExecutorService ex = Executors.newFixedThreadPool(COUNTERS);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<?>> fs = new ArrayList<>();
        for (int c = 0; c < COUNTERS; c++) { final int counter = c + 1; fs.add(ex.submit(() ->
{ go.await(); for (int i = 0; i < PER_COUNTER; i++) body.issue(counter, i); return null; })); }
        go.countDown();
        for (Future<?> f : fs) f.get();
        ex.shutdown();
    }
    static void runDb(File db, List<Long> keys) throws Exception {
        ExecutorService ex = Executors.newFixedThreadPool(COUNTERS);
        CountDownLatch go = new CountDownLatch(1);
        List<Future<?>> fs = new ArrayList<>();
        for (int c = 0; c < COUNTERS; c++) {
            final int counter = c + 1;
            fs.add(ex.submit(() -> {
                try (Connection con = DriverManager.getConnection("jdbc:minisqlite:" +
db.getAbsolutePath())) {
```

```
                    try (Statement st = con.createStatement()) { st.execute("PRAGMA busy_timeout =
20000"); }          // wait for the other counters instead of failing
                    go.await();
                    try (PreparedStatement ps = con.prepareStatement("INSERT INTO
issued(counter_no) VALUES (?)", Statement.RETURN_GENERATED_KEYS)) {
                        for (int i = 0; i < PER_COUNTER; i++) { ps.setInt(1, counter);
ps.executeUpdate(); try (ResultSet rs = ps.getGeneratedKeys()) { rs.next();
keys.add(rs.getLong(1)); } }
                    }
                }
                return null;
            }));
        }
        go.countDown();
        for (Future<?> f : fs) f.get();
        ex.shutdown();
    }
    static void report(String title, List<String> lines) {
        Set<String> ids = new HashSet<>(); int bad = 0;
        for (String l : lines) { if (!l.matches("P\\d+,counter\\d")) bad++; ids.add(l.split(",")
[0]); }
        System.out.printf("%s%n      lines: %d | distinct pass numbers: %d | duplicate numbers: %d
| damaged lines: %d%n", title, lines.size(), ids.size(), lines.size() - ids.size(), bad);
    }
}
```

Appendix A – Repository Layout, How to Run, GitHub Steps

```
CSA09-<RegisterNumber>-Assignment5/
  README.md   GITHUB_STEPS.md
  TaskA1/  design notes (layouts, pseudocode are in the report)        TaskA2/  design notes
  TaskA3/  Dao, AbstractJdbcDao, StudentDao, CourseDao, MarksDao, scattered-SQL classes, A3Demo,
student.sql, output
  TaskA4/  Resident, Hostel, Room, SerializationDemo, v1/ v2/ versioning.sh, FormatBenchmark,
outputs, CSV
  TaskA5/  TransferDemo, wallet.sql, output
  TaskB1/  BusPass, BusPassStore, PassStore (Complete the Code), LineCounter, FixedRecordStore,
tests, benchmarks, outputs, CSV
  TaskB2/  LoginCheck, AdversarialDemo, DamagedFileDemo, users.sql, outputs
  TaskB3/  PassLoader (Complete the Code), PassInserters, benchmarks, FailureDemo, bus_pass.sql,
outputs, CSV
  TaskB4/  Migration, DbInspector, ScaleBenchmark, ConcurrencyDemo, SQL scripts, outputs, CSV
  diagrams/ graphs/ screenshots/ scripts/ tools/ report/
```

How to run the programs

With the test driver (no MySQL needed, Java 21): from the repository root, for example tools/run_jdbc.sh TaskB3 LoaderBenchmark /tmp 1000,10000 both loader_results.csv (the script compiles the task folder together with the test driver and runs the class with --enable-preview --enable-native-access=ALL-UNNAMED).

With MySQL (final submission): (1) install MySQL and create a database; run the .sql scripts of the task (bus_pass.sql, users.sql, student.sql, wallet.sql, bus_pass_full.sql); (2) download MySQL Connector/J and put the jar on the class path; (3) in the program, remove the line minisqlite.MiniSqlite.install(); and replace the URL "jdbc:minisqlite:" + file by "jdbc:mysql://localhost:3306/<database>" with your user name and password (the files Db.java, AdversarialDemo.java, A3Demo.java, TransferDemo.java, MigrationDemo.java, ScaleBenchmark.java and ConcurrencyDemo.java contain the URL); (4) compile and run with javac -cp .:mysql-connector-j-8.x.jar *.java and java -cp .:mysql-connector-j-8.x.jar <MainClass> (use ; instead of : on Windows). The JDBC code uses only java.sql interfaces, so nothing else changes; remember that MySQL needs -- (dash dash space) for SQL comments and that INTEGER PRIMARY KEY in the test table issued is INT AUTO_INCREMENT PRIMARY KEY in MySQL (script issued_mysql.sql).

README.md of the repository

```
# CSA09 – Programming in Java – Assignment 5 (CO5): Persistence – Files, Streams, Serialization, JDBC

| | |
|---|---|
| Name | <YOUR NAME> |
| Register number | <REGISTER NUMBER> |
| Java version | OpenJDK 21 (`java -version`) |
| Repository | CSA09-<RegisterNumber>-Assignment5 |

One folder per task (Part A: TaskA1–A5, Part B: TaskB1–B4): Java source, SQL scripts, captured console
output (`*_output*.txt`),
CSV measurements, and screenshots (`screenshots/`). The report with the marks map is in `report/`.
Part A answers (design and pseudocode) are in the report; TaskA3–A5 hold the supporting programs.

## How to compile and run
| Task | Commands (from the task folder) | Result (short note) |
|---|---|---|
| B1 | `javac *.java` · `java PassStore` · `java BusPassStoreTest` · `java BufferingBenchmark` · `java
GrowthBenchmark` | PassStore prints `[P1003, 192311003, Divya, 7, 3800.0]` then `null`; 11/11 tests pass;
50 MB: 52,428,784 reads unbuffered vs 6,401 buffered |
```

```
| B2 | needs a JDBC database (see below) · `java AdversarialDemo` · `java DamagedFileDemo` | 3 attacks
fool the string-built query; PreparedStatement rejects all; 9 damaged lines reported |
| B3 | `java B3Correctness` · `java LoaderBenchmark <dir> 1000,10000,100000 both loader_results.csv` ·
`java FailureDemo` | one transaction: 0 rows after a failure; auto-commit leaves 500; transaction is
~260x faster for 100,000 rows |
| B4 | `java MigrationDemo` · `java ScaleBenchmark` · `java ConcurrencyDemo` | 8,014 lines → 8,000 loaded
+ 14 rejected with reasons; database lookup 0.025 ms vs 781 ms (text file) |
| A3/A4/A5 | `java A3Demo` · `java SerializationDemo` · `./versioning.sh` · `java FormatBenchmark` ·
`java TransferDemo` | supporting programs of the Part A answers |

## Database
* **With MySQL (use this for the submission):** run the `.sql` scripts, put `mysql-connector-j-8.x.jar`
on the class path, remove the line
  `minisqlite.MiniSqlite.install();` and use a `jdbc:mysql://localhost:3306/<db>` URL (see Appendix A of
the report).
* **Without MySQL (how the evidence in this repository was produced):** `tools/run_jdbc.sh <TaskFolder>
<MainClass> [args]` compiles the task with a
  test-only JDBC driver for SQLite (`tools/sqlite-jdbc-shim`, Java 21 foreign-function API). Real SQLite
engine, JDBC interfaces only.
```

GitHub steps

```
# GitHub submission steps (commit progressively – at least one commit per task)
```
git init
git branch -M main
git remote add origin https://github.com/<username>/CSA09-<RegisterNumber>-Assignment5.git
git add TaskA1 TaskA2 && git commit -m "Part A: A1 and A2 designs"
git add TaskA3 && git commit -m "A3: generic DAO and comparison"
git add TaskA4 && git commit -m "A4: serialization experiments"
git add TaskA5 && git commit -m "A5: transaction and injection analysis"
git add TaskB1 && git commit -m "B1: bus-pass file store, buffering and growth measurements"
git add TaskB2 && git commit -m "B2: adversarial inputs and damaged file"
git add TaskB3 && git commit -m "B3: Statement vs PreparedStatement vs batch, PassLoader"
git add TaskB4 && git commit -m "B4: migration, metadata, scale and concurrency"
git add README.md diagrams graphs screenshots scripts tools report && git commit -m "Add report, diagrams
and tools"
git push -u origin main
```
Commit each task when you finish it on your own computer (the commit history is checked).
```

Appendix B – The Test-Only JDBC Driver and How the Evidence Was Produced

Because no MySQL server was available, the JDBC programs were run against the real SQLite library through a small driver written for this purpose. **It is test equipment, not part of any answer.** It implements java.sql.Driver, Connection, Statement, PreparedStatement, ResultSet, ResultSetMetaData and DatabaseMetaData with dynamic proxies and calls the native SQLite functions (sqlite3_open, sqlite3_exec, sqlite3_prepare_v2, sqlite3_step, sqlite3_bind_*, sqlite3_column_*) through Java 21's foreign-function API. Semantics used by the experiments: Statement.executeUpdate(String) parses and executes the text every time (sqlite3_exec); a PreparedStatement is prepared once (sqlite3_prepare_v2) and re-bound and re-executed; setAutoCommit(false) issues BEGIN; commit and rollback issue COMMIT and ROLLBACK; auto-commit mode leaves every statement as its own transaction (a durable commit each); constraint violations are thrown as SQLIntegrityConstraintViolationException. Any JDBC method that is not implemented throws SQLFeatureNotSupportedException, so a program cannot silently depend on something the driver does not do.

Limitation	Effect on the results
SQLite is not MySQL	Error messages, dialect details (backslash escapes, AUTO_INCREMENT, LOAD DATA, upsert syntax) and the server-side engine differ. The experiments were chosen so that the concepts (parse cost, commit cost, atomicity, constraints, injection) are the same.
No network	Batching and multi-row inserts save more with a remote server than measured here; this is stated where it matters (B3).
Durable commits on this disk	The 0.8 ms per commit depends on the disk. The in-memory runs (Table B3.2) show the same experiments without disk waits.
One machine, one run for the long cases	The 100,000-row auto-commit runs took about 80 s each and were run once; the repeated runs of the short cases show variation of about ±20%.

tools/run_jdbc.sh

```
#!/bin/sh
# Usage: tools/run_jdbc.sh <task-folder> <MainClass> [args...]
# Compiles the task's sources together with the test-only SQLite JDBC shim and runs MainClass
(Java 21, preview API).
# On a computer with MySQL you do NOT need this: use javac/java with mysql-connector-j on the
class path.
ROOT=$(cd "$(dirname "$0")/.." && pwd)
TASK=$1; MAIN=$2; shift 2
OUT=$ROOT/$TASK/.out_jdbc
rm -rf "$OUT"; mkdir -p "$OUT"
java -m jdk.compiler/com.sun.tools.javac.Main -source 21 --enable-preview -sourcepath
"$ROOT/TaskB1" -d "$OUT" $(find $ROOT/tools/sqlite-jdbc-shim/src $ROOT/$TASK -name '*.java' -not -
path '*/.out_*' -not -path '*/v1/*' -not -path '*/v2/*' -not -path '*/errors/*') 2>&1 | grep -v
"^Note:"
cd $ROOT/$TASK
java --enable-preview --enable-native-access=ALL-UNNAMED -cp "$OUT" "$MAIN" "$@" 2>&1 | grep -v
"^WARNING:"
```

tools/sqlite-jdbc-shim/src/minisqlite/MiniSqlite.java - the test-only JDBC driver

```
package minisqlite;

import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.sql.*;
import java.util.*;

/**
```

```
 * TEST-ONLY JDBC driver ("jdbc:minisqlite:<file or :memory:>") that talks to the real SQLite
library through Java's
 * foreign-function API. It exists because no MySQL server / Connector-J jar was available while
preparing this assignment.
 * Application code uses ONLY java.sql interfaces, so for MySQL only the JDBC URL and the driver
jar change.
 * Only the JDBC methods needed by the assignment programs are implemented; any other method
throws SQLFeatureNotSupportedException.
 * Run with:  --enable-preview --enable-native-access=ALL-UNNAMED   (Java 21)
 */
public class MiniSqlite {
    public static final String PREFIX = "jdbc:minisqlite:";

    // ---------------------------------------------------------------- native layer
    static final Linker LINKER = Linker.nativeLinker();
    static final SymbolLookup LIB = SymbolLookup.libraryLookup("libsqlite3.so.0", Arena.global());
    static MethodHandle fn(String name, MemoryLayout ret, MemoryLayout... args) {
        FunctionDescriptor d = ret == null ? FunctionDescriptor.ofVoid(args) :
FunctionDescriptor.of(ret, args);
        return LINKER.downcallHandle(LIB.find(name).orElseThrow(), d);
    }
    static final ValueLayout.OfInt I = ValueLayout.JAVA_INT;
    static final ValueLayout.OfLong L = ValueLayout.JAVA_LONG;
    static final ValueLayout.OfDouble D = ValueLayout.JAVA_DOUBLE;
    static final AddressLayout A = ValueLayout.ADDRESS;
    static final MethodHandle OPEN = fn("sqlite3_open", I, A, A), CLOSE = fn("sqlite3_close", I,
A), EXEC = fn("sqlite3_exec", I, A, A, A, A, A),
        PREPARE = fn("sqlite3_prepare_v2", I, A, A, I, A, A), STEP = fn("sqlite3_step", I, A),
RESET = fn("sqlite3_reset", I, A),
        FINALIZE = fn("sqlite3_finalize", I, A), CLEAR = fn("sqlite3_clear_bindings", I, A),
ERRMSG = fn("sqlite3_errmsg", A, A),
        BIND_TEXT = fn("sqlite3_bind_text", I, A, I, A, I, A), BIND_LONG =
fn("sqlite3_bind_int64", I, A, I, L),
        BIND_DOUBLE = fn("sqlite3_bind_double", I, A, I, D), BIND_NULL = fn("sqlite3_bind_null",
I, A, I),
        COL_COUNT = fn("sqlite3_column_count", I, A), COL_NAME = fn("sqlite3_column_name", A, A,
I), COL_DECL = fn("sqlite3_column_decltype", A, A, I),
        COL_TEXT = fn("sqlite3_column_text", A, A, I), COL_LONG = fn("sqlite3_column_int64", L, A,
I), COL_DOUBLE = fn("sqlite3_column_double", D, A, I),
        COL_TYPE = fn("sqlite3_column_type", I, A, I), CHANGES = fn("sqlite3_changes", I, A),
LAST_ID = fn("sqlite3_last_insert_rowid", L, A),
        VERSION = fn("sqlite3_libversion", A);
    static final MemorySegment TRANSIENT = MemorySegment.ofAddress(-1);

    static String str(MemorySegment p) { return p.address() == 0 ? null :
p.reinterpret(Long.MAX_VALUE).getUtf8String(0); }
    static RuntimeException rt(Throwable t) { return t instanceof RuntimeException r ? r : new
RuntimeException(t); }

    public static String libVersion() { try { return str((MemorySegment) VERSION.invoke()); }
catch (Throwable t) { throw rt(t); } }

    static class Db {
        MemorySegment db;
        Db(String path) throws SQLException {
            try (Arena a = Arena.ofConfined()) {
                MemorySegment cell = a.allocate(A);
                int rc = (int) OPEN.invoke(a.allocateUtf8String(path), cell);
                db = cell.get(A, 0);
                if (rc != 0) throw new SQLException("cannot open " + path);
            } catch (SQLException e) { throw e; } catch (Throwable t) { throw rt(t); }
        }
        SQLException error(int rc, String sql) {
            try {
                String m = str((MemorySegment) ERRMSG.invoke(db));
```

```
                if ((rc & 0xff) == 19) return new SQLIntegrityConstraintViolationException(m,
"23000", rc);
                return new SQLException(m + (sql == null ? "" : "  [" + abbreviate(sql) + "]"),
"HY000", rc);
            } catch (Throwable t) { throw rt(t); }
        }
        static String abbreviate(String s) { return s.length() > 90 ? s.substring(0, 90) + "..." :
s; }
        int exec(String sql) throws SQLException {            // full parse + execute every time
(what a JDBC Statement does)
            try (Arena a = Arena.ofConfined()) {
                int rc = (int) EXEC.invoke(db, a.allocateUtf8String(sql), MemorySegment.NULL,
MemorySegment.NULL, MemorySegment.NULL);
                if (rc != 0) throw error(rc, sql);
                return (int) CHANGES.invoke(db);
            } catch (SQLException e) { throw e; } catch (Throwable t) { throw rt(t); }
        }
        MemorySegment prepare(String sql) throws SQLException {
            try (Arena a = Arena.ofConfined()) {
                MemorySegment cell = a.allocate(A);
                int rc = (int) PREPARE.invoke(db, a.allocateUtf8String(sql), -1, cell,
MemorySegment.NULL);
                if (rc != 0) throw error(rc, sql);
                return cell.get(A, 0);
            } catch (SQLException e) { throw e; } catch (Throwable t) { throw rt(t); }
        }
        int step(MemorySegment st, String sql) throws SQLException {
            try { int rc = (int) STEP.invoke(st); if (rc != 100 && rc != 101) throw error(rc,
sql); return rc; }
            catch (SQLException e) { throw e; } catch (Throwable t) { throw rt(t); }
        }
        void close() { try { CLOSE.invoke(db); } catch (Throwable t) { throw rt(t); } }
    }

    // ---------------------------------------------------------------- JDBC proxies
    @SuppressWarnings("unchecked")
    static <T> T proxy(Class<T> c, InvocationHandler h) { return (T)
Proxy.newProxyInstance(MiniSqlite.class.getClassLoader(), new Class<?>[]{c}, h); }
    static Object objectMethod(Object self, Method m, Object[] a, String name) {
        switch (m.getName()) {
            case "toString": return name + "@" +
Integer.toHexString(System.identityHashCode(self));
            case "hashCode": return System.identityHashCode(self);
            case "equals": return self == a[0];
            default: return null;
        }
    }
    static boolean isObjectMethod(Method m) { return m.getDeclaringClass() == Object.class; }
    static SQLFeatureNotSupportedException unsupported(Method m) { return new
SQLFeatureNotSupportedException("MiniSqlite does not implement " +
m.getDeclaringClass().getSimpleName() + "." + m.getName()); }

    /** Registers the driver with java.sql.DriverManager. */
    public static void install() throws SQLException {
        Driver d = proxy(Driver.class, (self, m, a) -> {
            if (isObjectMethod(m)) return objectMethod(self, m, a, "MiniSqliteDriver");
            switch (m.getName()) {
                case "acceptsURL": return a[0] != null && ((String) a[0]).startsWith(PREFIX);
                case "connect": return ((String) a[0]).startsWith(PREFIX) ? new Conn(((String)
a[0]).substring(PREFIX.length()+)).jdbc() : null;
                case "getMajorVersion": return 0; case "getMinorVersion": return 1; case
"jdbcCompliant": return false;
                default: throw unsupported(m);
            }
        });
```

```
        DriverManager.registerDriver(d);
    }

    static class Conn {
        final Db db; boolean autoCommit = true, inTx = false, closed = false; Connection jdbc;
        Conn(String path) throws SQLException { db = new Db(path); }
        Connection jdbc() {
            jdbc = proxy(Connection.class, (self, m, a) -> {
                if (isObjectMethod(m)) return objectMethod(self, m, a, "MiniSqliteConnection");
                switch (m.getName()) {
                    case "createStatement": return new Stmt(this).jdbc();
                    case "prepareStatement": return new Prep(this, (String) a[0]).jdbc();
                    case "setAutoCommit": {
                        boolean on = (Boolean) a[0];
                        if (on && inTx) { db.exec("COMMIT"); inTx = false; }
                        if (!on && !inTx) { db.exec("BEGIN"); inTx = true; }
                        autoCommit = on; return null; }
                    case "getAutoCommit": return autoCommit;
                    case "commit": if (inTx) { db.exec("COMMIT"); inTx = false; if (!autoCommit) {
db.exec("BEGIN"); inTx = true; } } return null;
                    case "rollback": if (inTx) { db.exec("ROLLBACK"); inTx = false; if (!
autoCommit) { db.exec("BEGIN"); inTx = true; } } return null;
                    case "close": if (!closed) { if (inTx) { try { db.exec("ROLLBACK"); } catch
(SQLException ignore) { } } db.close(); closed = true; } return null;
                    case "isClosed": return closed;
                    case "getMetaData": return meta();
                    case "isValid": return !closed;
                    default: throw unsupported(m);
                }
            });
            return jdbc;
        }
        DatabaseMetaData meta() {
            return proxy(DatabaseMetaData.class, (self, m, a) -> {
                if (isObjectMethod(m)) return objectMethod(self, m, a, "MiniSqliteMetaData");
                switch (m.getName()) {
                    case "getDatabaseProductName": return "SQLite (via MiniSqlite test driver)";
                    case "getDatabaseProductVersion": return libVersion();
                    case "getDriverName": return "MiniSqlite JDBC test driver
(java.lang.foreign)";
                    case "getDriverVersion": return "0.1";
                    case "getURL": return PREFIX + "...";
                    case "getUserName": return "(none)";
                    case "getTables": {
                        PreparedStatement ps = new Prep(this, "SELECT NULL AS TABLE_CAT, NULL AS
TABLE_SCHEM, name AS TABLE_NAME, 'TABLE' AS TABLE_TYPE FROM sqlite_master WHERE type = 'table' AND
name NOT LIKE 'sqlite_%' ORDER BY name").jdbc();
                        return ps.executeQuery(); }
                    default: throw unsupported(m);
                }
            });
        }
    }

    static class Stmt {
        final Conn c; final List<String> batch = new ArrayList<>(); boolean closed;
        Stmt(Conn c) { this.c = c; }
        Statement jdbc() {
            return proxy(Statement.class, (self, m, a) -> {
                if (isObjectMethod(m)) return objectMethod(self, m, a, "MiniSqliteStatement");
                switch (m.getName()) {
                    case "executeUpdate": return c.db.exec((String) a[0]);
                    case "execute": { String sql = (String) a[0]; c.db.exec(sql); return false; }
                    case "executeQuery": return new Rs(c, c.db.prepare((String) a[0]), (String)
a[0], true).jdbc();
```

```
                    case "addBatch": batch.add((String) a[0]); return null;
                    case "clearBatch": batch.clear(); return null;
                    case "executeBatch": { int[] r = new int[batch.size()]; for (int i = 0; i <
r.length; i++) r[i] = c.db.exec(batch.get(i)); batch.clear(); return r; }
                    case "getConnection": return c.jdbc;
                    case "close": closed = true; return null;
                    case "isClosed": return closed;
                    default: throw unsupported(m);
                }
            });
        }
    }

    static class Prep {
        final Conn c; final String sql; final MemorySegment st; final int nParams; final Object[]
params;
        final List<Object[]> batch = new ArrayList<>(); boolean closed;
        Prep(Conn c, String sql) throws SQLException {
            this.c = c; this.sql = sql; this.st = c.db.prepare(sql);
            int n = 0; for (char ch : sql.toCharArray()) if (ch == '?') n++;
            nParams = n; params = new Object[n + 1];
        }
        void bindAll(Object[] p) throws SQLException {
            try {
                RESET.invoke(st); CLEAR.invoke(st);
                for (int i = 1; i <= nParams; i++) {
                    Object v = p[i];
                    int rc;
                    if (v == null) rc = (int) BIND_NULL.invoke(st, i);
                    else if (v instanceof Integer || v instanceof Long) rc = (int)
BIND_LONG.invoke(st, i, ((Number) v).longValue());
                    else if (v instanceof Double || v instanceof Float) rc = (int)
BIND_DOUBLE.invoke(st, i, ((Number) v).doubleValue());
                    else try (Arena a = Arena.ofConfined()) { rc = (int) BIND_TEXT.invoke(st, i,
a.allocateUtf8String(v.toString()), -1, TRANSIENT); }
                    if (rc != 0) throw c.db.error(rc, sql);
                }
            } catch (SQLException e) { throw e; } catch (Throwable t) { throw rt(t); }
        }
        int runUpdate(Object[] p) throws SQLException {
            bindAll(p);
            try { c.db.step(st, sql); return (int) CHANGES.invoke(c.db.db); }
            catch (SQLException e) { try { RESET.invoke(st); } catch (Throwable ignore) { } throw
e; }
            catch (Throwable t) { throw rt(t); }
        }
        PreparedStatement jdbc() {
            return proxy(PreparedStatement.class, (self, m, a) -> {
                if (isObjectMethod(m)) return objectMethod(self, m, a,
"MiniSqlitePreparedStatement");
                String n = m.getName();
                switch (n) {
                    case "setString": case "setInt": case "setLong": case "setDouble": case
"setObject": case "setBigDecimal": case "setFloat": {
                        int idx = (Integer) a[0]; if (idx < 1 || idx > nParams) throw new
SQLException("parameter index out of range: " + idx);
                        params[idx] = a[1]; return null; }
                    case "setNull": params[(Integer) a[0]] = null; return null;
                    case "clearParameters": Arrays.fill(params, null); return null;
                    case "executeUpdate": return runUpdate(params);
                    case "execute": runUpdate(params); return false;
                    case "executeQuery": { bindAll(params); return new Rs(c, st, sql,
false).jdbc(); }
                    case "addBatch": batch.add(params.clone()); return null;
                    case "clearBatch": batch.clear(); return null;
```

```
                    case "executeBatch": { int[] r = new int[batch.size()]; try { for (int i = 0;
i < r.length; i++) r[i] = runUpdate(batch.get(i)); } finally { batch.clear(); } return r; }
                    case "getGeneratedKeys": return new Prep(c, "SELECT last_insert_rowid() AS
GENERATED_KEY").jdbc().executeQuery();
                    case "getConnection": return c.jdbc;
                    case "close": if (!closed) { closed = true; FINALIZE.invoke(st); } return
null;
                    case "isClosed": return closed;
                    default: throw unsupported(m);
                }
            });
        }
    }

    static class Rs {
        final Conn c; final MemorySegment st; final String sql; final boolean ownsStmt; boolean
done, closed, wasNull;
        Rs(Conn c, MemorySegment st, String sql, boolean ownsStmt) { this.c = c; this.st = st;
this.sql = sql; this.ownsStmt = ownsStmt; }
        int colIndex(Object o) throws Throwable {
            if (o instanceof Integer i) return i - 1;
            int n = (int) COL_COUNT.invoke(st);
            for (int i = 0; i < n; i++) if (str((MemorySegment) COL_NAME.invoke(st,
i)).equalsIgnoreCase((String) o)) return i;
            throw new SQLException("no such column: " + o);
        }
        ResultSet jdbc() {
            return proxy(ResultSet.class, (self, m, a) -> {
                if (isObjectMethod(m)) return objectMethod(self, m, a, "MiniSqliteResultSet");
                switch (m.getName()) {
                    case "next": { if (done) return false; int rc = c.db.step(st, sql); if (rc ==
101) { done = true; return false; } return true; }
                    case "getString": { MemorySegment p = (MemorySegment) COL_TEXT.invoke(st,
colIndex(a[0])); wasNull = p.address() == 0; return str(p); }
                    case "getInt": { long v = (long) COL_LONG.invoke(st, colIndex(a[0])); return
(int) v; }
                    case "getLong": return (long) COL_LONG.invoke(st, colIndex(a[0]));
                    case "getDouble": return (double) COL_DOUBLE.invoke(st, colIndex(a[0]));
                    case "wasNull": return wasNull;
                    case "getMetaData": return meta();
                    case "close": if (!closed) { closed = true; if (ownsStmt) FINALIZE.invoke(st);
else RESET.invoke(st); } return null;
                    case "isClosed": return closed;
                    default: throw unsupported(m);
                }
            });
        }
        ResultSetMetaData meta() {
            return proxy(ResultSetMetaData.class, (self, m, a) -> {
                if (isObjectMethod(m)) return objectMethod(self, m, a,
"MiniSqliteResultSetMetaData");
                switch (m.getName()) {
                    case "getColumnCount": return (int) COL_COUNT.invoke(st);
                    case "getColumnName": case "getColumnLabel": return str((MemorySegment)
COL_NAME.invoke(st, (Integer) a[0] - 1));
                    case "getColumnTypeName": { String d = str((MemorySegment) COL_DECL.invoke(st,
(Integer) a[0] - 1)); return d == null ? "(expression)" : d; }
                    case "getColumnType": { String d = str((MemorySegment) COL_DECL.invoke(st,
(Integer) a[0] - 1)); d = d == null ? "" : d.toUpperCase();
                        return d.startsWith("INT") ? Types.INTEGER : d.startsWith("DECIMAL") ?
Types.DECIMAL : d.startsWith("DOUBLE") ? Types.DOUBLE : Types.VARCHAR; }
                    default: throw unsupported(m);
                }
            });
        }
```

```
    }
}
```

Appendix C – Supporting Programs of Part A, Viva Preparation, Checklist

Part A answers contain pseudocode only, as required. The following Java programs implement and test the Part A designs; their outputs are quoted in the answers.

C.1 Task A3 – generic DAO

TaskA3/student.sql

```
-- Task A3: tables used by the DAO classes (MySQL / SQLite compatible)
CREATE TABLE student (
    reg_no  VARCHAR(12) PRIMARY KEY,       -- Student.regNo
    name    VARCHAR(60) NOT NULL,          -- Student.name
    email   VARCHAR(80) NOT NULL,          -- Student.email
    year_no INT         NOT NULL           -- Student.year
);
CREATE TABLE course (course_code VARCHAR(10) PRIMARY KEY, title VARCHAR(80) NOT NULL, credits INT
NOT NULL);
CREATE TABLE marks  (mark_id INT PRIMARY KEY, reg_no VARCHAR(12) NOT NULL, course_code VARCHAR(10)
NOT NULL, marks INT NOT NULL);
```

TaskA3/Dao.java

```
import java.sql.SQLException;
import java.util.List;

/** Generic data-access interface: T = the entity class, K = the type of its primary key. */
public interface Dao<T, K> {
    void insert(T entity) throws SQLException;
    T findById(K key) throws SQLException;          // null when there is no such row
    List<T> findAll() throws SQLException;
    int update(T entity) throws SQLException;       // number of rows changed
    int delete(K key) throws SQLException;          // number of rows deleted
}
```

TaskA3/AbstractJdbcDao.java

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * The steps that were copied into StudentDB, CourseDB and MarksDB are written ONCE here:
 * open connection -> prepare SQL -> bind parameters -> execute -> map rows to objects -> close
everything.
 * A subclass supplies only what differs: the SQL text, how to bind an entity, and how to turn a
row into an object.
 */
public abstract class AbstractJdbcDao<T, K> implements Dao<T, K> {
    public interface ConnectionFactory { Connection open() throws SQLException; }
    public interface Binder { void bind(PreparedStatement ps) throws SQLException; }
    public interface RowMapper<R> { R map(ResultSet rs) throws SQLException; }

    private final ConnectionFactory connections;
    protected AbstractJdbcDao(ConnectionFactory connections) { this.connections = connections; }

    // ----- the parts that differ (one small method each) -----
    protected abstract String insertSql();
    protected abstract void bindInsert(PreparedStatement ps, T e) throws SQLException;
    protected abstract String selectByIdSql();
    protected abstract String selectAllSql();
    protected abstract String updateSql();
    protected abstract void bindUpdate(PreparedStatement ps, T e) throws SQLException;
    protected abstract String deleteSql();
    protected abstract T mapRow(ResultSet rs) throws SQLException;
```

```
    protected abstract void bindKey(PreparedStatement ps, K key) throws SQLException;

    // ----- the repeated steps, written once -----
    protected <R> R query(String sql, Binder binder, RowMapper<R> mapper, boolean many, List<R>
into) throws SQLException {
        try (Connection c = connections.open(); PreparedStatement ps = c.prepareStatement(sql)) {
            binder.bind(ps);
            try (ResultSet rs = ps.executeQuery()) {
                R first = null;
                while (rs.next()) { R r = mapper.map(rs); if (first == null) first = r; if (many)
into.add(r); else break; }
                return first;
            }
        }
    }
    protected int execute(String sql, Binder binder) throws SQLException {
        try (Connection c = connections.open(); PreparedStatement ps = c.prepareStatement(sql))
{ binder.bind(ps); return ps.executeUpdate(); }
    }

    @Override public void insert(T e) throws SQLException { execute(insertSql(), ps ->
bindInsert(ps, e)); }
    @Override public T findById(K key) throws SQLException { return query(selectByIdSql(), ps ->
bindKey(ps, key), this::mapRow, false, null); }
    @Override public List<T> findAll() throws SQLException { List<T> all = new ArrayList<>();
query(selectAllSql(), ps -> { }, this::mapRow, true, all); return all; }
    @Override public int update(T e) throws SQLException { return execute(updateSql(), ps ->
bindUpdate(ps, e)); }
    @Override public int delete(K key) throws SQLException { return execute(deleteSql(), ps ->
bindKey(ps, key)); }
}
```

TaskA3/Student.java

```
public class Student {
    public String regNo, name, email;
    public int year;
    public Student(String regNo, String name, String email, int year) { this.regNo = regNo;
this.name = name; this.email = email; this.year = year; }
    @Override public String toString() { return regNo + " " + name + " <" + email + "> year " +
year; }
}
```

TaskA3/StudentDao.java

```
import java.sql.*;

/** Object -> table mapping:  regNo -> reg_no (PRIMARY KEY), name -> name, email -> email, year ->
year_no. */
public class StudentDao extends AbstractJdbcDao<Student, String> {
    public StudentDao(ConnectionFactory f) { super(f); }
    @Override protected String insertSql()     { return "INSERT INTO student(reg_no, name, email,
year_no) VALUES (?, ?, ?, ?)"; }
    @Override protected String selectByIdSql() { return "SELECT reg_no, name, email, year_no FROM
student WHERE reg_no = ?"; }
    @Override protected String selectAllSql()  { return "SELECT reg_no, name, email, year_no FROM
student ORDER BY reg_no"; }
    @Override protected String updateSql()     { return "UPDATE student SET name = ?, email = ?,
year_no = ? WHERE reg_no = ?"; }
    @Override protected String deleteSql()     { return "DELETE FROM student WHERE reg_no = ?"; }
    @Override protected void bindInsert(PreparedStatement ps, Student s) throws SQLException
{ ps.setString(1, s.regNo); ps.setString(2, s.name); ps.setString(3, s.email); ps.setInt(4,
s.year); }
    @Override protected void bindUpdate(PreparedStatement ps, Student s) throws SQLException
{ ps.setString(1, s.name); ps.setString(2, s.email); ps.setInt(3, s.year); ps.setString(4,
s.regNo); }
```

```
    @Override protected void bindKey(PreparedStatement ps, String key) throws SQLException
{ ps.setString(1, key); }
    @Override protected Student mapRow(ResultSet rs) throws SQLException { return new
Student(rs.getString("reg_no"), rs.getString("name"), rs.getString("email"),
rs.getInt("year_no")); }
}
```

TaskA3/CourseDao.java

```
import java.sql.*;

public class CourseDao extends AbstractJdbcDao<CourseDao.Course, String> {
    public static class Course { public String code, title; public int credits; public
Course(String c, String t, int cr) { code = c; title = t; credits = cr; } public String toString()
{ return code + " " + title + " (" + credits + " credits)"; } }
    public CourseDao(ConnectionFactory f) { super(f); }
    @Override protected String insertSql()     { return "INSERT INTO course(course_code, title,
credits) VALUES (?, ?, ?)"; }
    @Override protected String selectByIdSql() { return "SELECT course_code, title, credits FROM
course WHERE course_code = ?"; }
    @Override protected String selectAllSql()  { return "SELECT course_code, title, credits FROM
course ORDER BY course_code"; }
    @Override protected String updateSql()     { return "UPDATE course SET title = ?, credits = ?
WHERE course_code = ?"; }
    @Override protected String deleteSql()     { return "DELETE FROM course WHERE course_code
= ?"; }
    @Override protected void bindInsert(PreparedStatement ps, Course c) throws SQLException
{ ps.setString(1, c.code); ps.setString(2, c.title); ps.setInt(3, c.credits); }
    @Override protected void bindUpdate(PreparedStatement ps, Course c) throws SQLException
{ ps.setString(1, c.title); ps.setInt(2, c.credits); ps.setString(3, c.code); }
    @Override protected void bindKey(PreparedStatement ps, String k) throws SQLException
{ ps.setString(1, k); }
    @Override protected Course mapRow(ResultSet rs) throws SQLException { return new
Course(rs.getString(1), rs.getString(2), rs.getInt(3)); }
}
```

TaskA3/MarksDao.java

```
import java.sql.*;

public class MarksDao extends AbstractJdbcDao<MarksDao.Mark, Integer> {
    public static class Mark { public int id; public String regNo, courseCode; public int marks;
public Mark(int i, String r, String c, int m) { id = i; regNo = r; courseCode = c; marks = m; }
public String toString() { return "#" + id + " " + regNo + " " + courseCode + " = " + marks; } }
    public MarksDao(ConnectionFactory f) { super(f); }
    @Override protected String insertSql()     { return "INSERT INTO marks(mark_id, reg_no,
course_code, marks) VALUES (?, ?, ?, ?)"; }
    @Override protected String selectByIdSql() { return "SELECT mark_id, reg_no, course_code,
marks FROM marks WHERE mark_id = ?"; }
    @Override protected String selectAllSql()  { return "SELECT mark_id, reg_no, course_code,
marks FROM marks ORDER BY mark_id"; }
    @Override protected String updateSql()     { return "UPDATE marks SET reg_no = ?, course_code
= ?, marks = ? WHERE mark_id = ?"; }
    @Override protected String deleteSql()     { return "DELETE FROM marks WHERE mark_id = ?"; }
    @Override protected void bindInsert(PreparedStatement ps, Mark m) throws SQLException
{ ps.setInt(1, m.id); ps.setString(2, m.regNo); ps.setString(3, m.courseCode); ps.setInt(4,
m.marks); }
    @Override protected void bindUpdate(PreparedStatement ps, Mark m) throws SQLException
{ ps.setString(1, m.regNo); ps.setString(2, m.courseCode); ps.setInt(3, m.marks); ps.setInt(4,
m.id); }
    @Override protected void bindKey(PreparedStatement ps, Integer k) throws SQLException
{ ps.setInt(1, k); }
    @Override protected Mark mapRow(ResultSet rs) throws SQLException { return new
Mark(rs.getInt(1), rs.getString(2), rs.getString(3), rs.getInt(4)); }
}
```

TaskA3/ScatteredStudentService.java - the alternative

```
import java.sql.*;
/** ALTERNATIVE design: SQL inside the service classes (every class repeats open / prepare / map /
close). */
public class ScatteredStudentService {
    private final AbstractJdbcDao.ConnectionFactory f;
    public ScatteredStudentService(AbstractJdbcDao.ConnectionFactory f) { this.f = f; }
    public Student find(String regNo) throws SQLException {
        try (Connection c = f.open(); PreparedStatement ps = c.prepareStatement("SELECT reg_no,
name, email, year_no FROM student WHERE reg_no = ?")) {
            ps.setString(1, regNo);
            try (ResultSet rs = ps.executeQuery()) { return rs.next() ? new
Student(rs.getString("reg_no"), rs.getString("name"), rs.getString("email"), rs.getInt("year_no"))
: null; }
        }
    }
    public int rename(String regNo, String name) throws SQLException {
        try (Connection c = f.open(); PreparedStatement ps = c.prepareStatement("UPDATE student
SET name = ? WHERE reg_no = ?")) { ps.setString(1, name); ps.setString(2, regNo); return
ps.executeUpdate(); }
    }
}
```

TaskA3/ScatteredReportScreen.java

```
import java.sql.*;
public class ScatteredReportScreen {
    private final AbstractJdbcDao.ConnectionFactory f;
    public ScatteredReportScreen(AbstractJdbcDao.ConnectionFactory f) { this.f = f; }
    public String studentList() throws SQLException {
        StringBuilder sb = new StringBuilder();
        try (Connection c = f.open(); PreparedStatement ps = c.prepareStatement("SELECT reg_no,
name FROM student ORDER BY reg_no"); ResultSet rs = ps.executeQuery()) {
            while (rs.next()) sb.append(rs.getString("reg_no")).append('
').append(rs.getString("name")).append('\n');
        }
        return sb.toString();
    }
}
```

TaskA3/ScatteredRegistrationScreen.java

```
import java.sql.*;
public class ScatteredRegistrationScreen {
    private final AbstractJdbcDao.ConnectionFactory f;
    public ScatteredRegistrationScreen(AbstractJdbcDao.ConnectionFactory f) { this.f = f; }
    public void register(String regNo, String name, String email, int year) throws SQLException {
        try (Connection c = f.open(); PreparedStatement ps = c.prepareStatement("INSERT INTO
student(reg_no, name, email, year_no) VALUES (?, ?, ?, ?)")) {
            ps.setString(1, regNo); ps.setString(2, name); ps.setString(3, email); ps.setInt(4,
year); ps.executeUpdate();
        }
    }
    public boolean exists(String regNo) throws SQLException {
        try (Connection c = f.open(); PreparedStatement ps = c.prepareStatement("SELECT COUNT(*)
FROM student WHERE reg_no = ?")) { ps.setString(1, regNo); try (ResultSet rs = ps.executeQuery())
{ rs.next(); return rs.getInt(1) > 0; } }
    }
}
```

TaskA3/ScatteredMarksService.java

```
import java.sql.*;
public class ScatteredMarksService {
    private final AbstractJdbcDao.ConnectionFactory f;
    public ScatteredMarksService(AbstractJdbcDao.ConnectionFactory f) { this.f = f; }
    public double average(String regNo) throws SQLException {
```

```
        try (Connection c = f.open(); PreparedStatement ps = c.prepareStatement("SELECT
AVG(m.marks) FROM marks m JOIN student s ON s.reg_no = m.reg_no WHERE s.reg_no = ?")) {
            ps.setString(1, regNo); try (ResultSet rs = ps.executeQuery()) { rs.next(); return
rs.getDouble(1); }
        }
    }
    public int deleteMarksOf(String regNo) throws SQLException {
        try (Connection c = f.open(); PreparedStatement ps = c.prepareStatement("DELETE FROM marks
WHERE reg_no = ?")) { ps.setString(1, regNo); return ps.executeUpdate(); }
    }
}
```

TaskA3/A3Demo.java

```
import java.io.File;
import java.lang.management.ManagementFactory;
import java.nio.file.*;
import java.sql.*;
import java.util.List;

public class A3Demo {
    static long sink = 0;
    static long allocated() { com.sun.management.ThreadMXBean b =
(com.sun.management.ThreadMXBean) ManagementFactory.getThreadMXBean(); return
b.getThreadAllocatedBytes(Thread.currentThread().getId()); }
    static int occurrences(String text, String word) { int n = 0, i = 0; while ((i =
text.indexOf(word, i)) >= 0) { n++; i += word.length(); } return n; }

    public static void main(String[] args) throws Exception {
        minisqlite.MiniSqlite.install();
        File db = File.createTempFile("a3demo", ".db"); db.delete();
        String url = "jdbc:minisqlite:" + db.getAbsolutePath();
        AbstractJdbcDao.ConnectionFactory f = () -> DriverManager.getConnection(url);
        try (Connection c = f.open(); Statement st = c.createStatement()) { for (String sql :
Files.readString(Path.of("student.sql")).split(";")) if (!sql.isBlank()) st.executeUpdate(sql); }

        System.out.println("=== A3.2 One generic design, three entities ===");
        StudentDao students = new StudentDao(f);
        students.insert(new Student("192311001", "Arun", "arun@example.com", 2));
        students.insert(new Student("192311002", "Divya", "divya@example.com", 3));
        System.out.println("findById(192311002)      : " + students.findById("192311002"));
        System.out.println("findById(000000000)      : " + students.findById("000000000") + "
(no such row)");
        System.out.println("update Arun's year       : rows changed = " + students.update(new
Student("192311001", "Arun", "arun@example.com", 3)));
        System.out.println("findAll()                : " + students.findAll());
        System.out.println("delete(192311002)        : rows deleted = " +
students.delete("192311002") + ", remaining = " + students.findAll().size());
        CourseDao courses = new CourseDao(f);
        courses.insert(new CourseDao.Course("CSA09", "Programming in Java", 4));
        System.out.println("CourseDao.findAll()      : " + courses.findAll());
        MarksDao marks = new MarksDao(f);
        marks.insert(new MarksDao.Mark(1, "192311001", "CSA09", 88)); marks.insert(new
MarksDao.Mark(2, "192311001", "CSA10", 76));
        System.out.println("MarksDao.findAll()       : " + marks.findAll());

        System.out.println();
        System.out.println("=== Alternative: SQL written inside service / GUI classes ===");
        ScatteredStudentService alt = new ScatteredStudentService(f);
        System.out.println("ScatteredStudentService.find(192311001) : " + alt.find("192311001") +
"   (same result)");

        // ----- how many places change when a column of the STUDENT table is renamed? (counted
from the real source files) -----
        System.out.println();
```

```
        System.out.println("=== Rename a column of the student table: SQL statements that mention
the table, and in how many classes ===");
        String[] all = {"StudentDao.java", "CourseDao.java", "MarksDao.java",
"ScatteredStudentService.java", "ScatteredReportScreen.java", "ScatteredRegistrationScreen.java",
"ScatteredMarksService.java"};
        int daoStatements = 0, daoClasses = 0, altStatements = 0, altClasses = 0;
        for (String fn : all) {
            int n = studentStatements(Files.readString(Path.of(fn)));
            if (n == 0) continue;
            boolean generic = !fn.startsWith("Scattered");
            System.out.println("  " + (generic ? "generic DAO design : " : "scattered design   :
") + fn + " -> " + n + " statement(s) on the student table");
            if (generic) { daoStatements += n; daoClasses++; } else { altStatements += n;
altClasses++; }
        }
        System.out.println("  generic DAO : " + daoStatements + " statements, all in " +
daoClasses + " class (StudentDao)");
        System.out.println("  scattered   : " + altStatements + " statements spread over " +
altClasses + " classes (every service / screen that touches the table)");

        // ----- cost per operation -----
        System.out.println();
        System.out.println("=== Cost per findById (new connection each time, as in the original
StudentDB class) ===");
        ScatteredStudentService direct = alt;
        for (int i = 0; i < 300; i++) { sink += students.findById("192311001") != null ? 1 : 0;
sink += direct.find("192311001") != null ? 1 : 0; }
        int n = 3000; double[] us = new double[2]; long[] bytes = new long[2];
        for (int d = 0; d < 2; d++) {
            double best = 1e18;
            for (int r = 0; r < 5; r++) {
                long a0 = allocated(), t0 = System.nanoTime();
                for (int i = 0; i < n; i++) sink += (d == 0 ? students.findById("192311001") :
direct.find("192311001")) != null ? 1 : 0;
                best = Math.min(best, (System.nanoTime() - t0) / 1e3 / n);
                if (r == 0) bytes[d] = (allocated() - a0) / n;
            }
            us[d] = best;
        }
        System.out.printf("generic Dao<Student,String>.findById : %.1f microseconds per call, %,d
bytes allocated per call%n", us[0], bytes[0]);
        System.out.printf("SQL inside the service class         : %.1f microseconds per call, %,d
bytes allocated per call%n", us[1], bytes[1]);
        System.out.println("(checksum " + sink + ")");
        db.delete();
    }

    /** counts SQL statements (in code lines, not comments) that read or write the student table
*/
    static int studentStatements(String src) {
        int n = 0;
        for (String line : src.split("\n")) {
            String t = line.trim();
            if (t.startsWith("//") || t.startsWith("*") || t.startsWith("/*")) continue;
            java.util.regex.Matcher m = java.util.regex.Pattern.compile("(FROM|INTO|UPDATE|JOIN)
student\\b").matcher(line);
            while (m.find()) n++;
        }
        return n;
    }
}
```

C.2 Task A4 – serialization

TaskA4/Resident.java

```
import java.io.Serializable;
import java.sql.Connection;

/** A resident of the hostel. password and conn are transient: they must NOT be written to the
file. */
public class Resident implements Serializable {
    private static final long serialVersionUID = 1L;
    public String name, regNo;
    public transient String password;          // secret: never stored in the clear on disk (re-
entered or stored as a hash elsewhere)
    public transient Connection conn;          // live resource: meaningless after the program
ends, and Connection is not Serializable

    public Resident(String name, String regNo, String password, Connection conn) { this.name =
name; this.regNo = regNo; this.password = password; this.conn = conn; }
    @Override public String toString() { return name + "(" + regNo + ", pwd=" + password + ",
conn=" + (conn == null ? "null" : "live") + ")"; }
}
```

TaskA4/BadResident.java

```
import java.io.Serializable;
import java.sql.Connection;

/** The WRONG design: the live Connection is an ordinary field. */
public class BadResident implements Serializable {
    public String name, regNo;
    public String password;
    public Connection conn;                   // NOT transient
    public BadResident(String name, String regNo, String password, Connection conn) { this.name =
name; this.regNo = regNo; this.password = password; this.conn = conn; }
}
```

TaskA4/Room.java

```
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

public class Room implements Serializable {
    private static final long serialVersionUID = 1L;
    public int number;
    public List<Resident> residents = new ArrayList<>();
    public Room(int number) { this.number = number; }
}
```

TaskA4/Hostel.java

```
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

public class Hostel implements Serializable {
    private static final long serialVersionUID = 1L;
    public String name;
    public List<Room> rooms = new ArrayList<>();
    public Hostel(String name) { this.name = name; }
}
```

TaskA4/SerializationDemo.java

```
import java.io.*;
import java.lang.reflect.Proxy;
import java.sql.Connection;
import java.util.*;
```

```
/** Task A4.1: what is written when a Hostel is serialized, what transient does, and what happens
with a live Connection. */
public class SerializationDemo {
    static Connection fakeLiveConnection() {                              // an object that
implements Connection but is NOT Serializable
        return (Connection) Proxy.newProxyInstance(SerializationDemo.class.getClassLoader(), new
Class<?>[]{Connection.class}, (p, m, a) -> null);
    }
    static int count;
    public static void main(String[] args) throws Exception {
        Connection live = fakeLiveConnection();
        Hostel h = new Hostel("Block A");
        Room r101 = new Room(101), r102 = new Room(102);
        Resident arun = new Resident("Arun", "192311001", "secret123", live);
        Resident divya = new Resident("Divya", "192311002", "pass456", live);
        r101.residents.add(arun); r101.residents.add(divya);
        r102.residents.add(arun);                                          // the SAME Resident
object is also in room 102
        h.rooms.add(r101); h.rooms.add(r102);

        System.out.println("=== 1. Serialize the Hostel (one writeObject call) ===");
        ByteArrayOutputStream bos = new ByteArrayOutputStream();
        try (ObjectOutputStream out = new ObjectOutputStream(bos)) { out.writeObject(h); }
        byte[] bytes = bos.toByteArray();
        System.out.println("bytes written: " + bytes.length);
        Hostel back;
        try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(bytes))) { back
= (Hostel) in.readObject(); }
        System.out.println("hostel name after reading : " + back.name + ", rooms = " +
back.rooms.size());
        for (Room r : back.rooms) System.out.println("  room " + r.number + " -> " + r.residents);
        Resident a1 = back.rooms.get(0).residents.get(0), a2 = back.rooms.get(1).residents.get(0);
        System.out.println("transient fields after reading: password = " + a1.password + ", conn =
" + a1.conn + "   (not stored, reset to null)");
        System.out.println("Arun in room 101 and Arun in room 102 are the same object after
reading? " + (a1 == a2) + "   (a shared reference is written once and shared again)");
        System.out.println("-> writeObject(hostel) also wrote ALL reachable objects: 1 Hostel, " +
back.rooms.size() + " Rooms and 2 distinct Residents (3 references).");
        System.out.println("password visible in the file bytes? " + new String(bytes, "ISO-8859-
1").contains("secret123"));

        System.out.println();
        System.out.println("=== 2. The wrong design: a live Connection that is not transient
===");
        BadResident bad = new BadResident("Arun", "192311001", "secret123", live);
        try (ObjectOutputStream out = new ObjectOutputStream(new ByteArrayOutputStream()))
{ out.writeObject(bad); }
        catch (NotSerializableException e) { System.out.println("NotSerializableException: " +
e.getMessage().replaceAll("\\$Proxy[0-9]+", "<proxy>")); }
        BadResident bad2 = new BadResident("Arun", "192311001", "secret123", null);
        ByteArrayOutputStream b2 = new ByteArrayOutputStream();
        try (ObjectOutputStream out = new ObjectOutputStream(b2)) { out.writeObject(bad2); }
        System.out.println("with conn = null the same class is written, but the password IS in the
file: " + new String(b2.toByteArray(), "ISO-8859-1").contains("secret123"));
    }
}
```

TaskA4/v1/ResidentFixed.java

```
import java.io.Serializable;
public class ResidentFixed implements Serializable {
    private static final long serialVersionUID = 1L;              // explicit version number
    public String name, regNo;
    public ResidentFixed(String n, String r) { name = n; regNo = r; }
}
```

TaskA4/v1/ResidentImplicit.java

```
import java.io.Serializable;
public class ResidentImplicit implements Serializable {          // NO serialVersionUID: the
compiler computes one from the class structure
    public String name, regNo;
    public ResidentImplicit(String n, String r) { name = n; regNo = r; }
}
```

TaskA4/v1/Writer.java

```
import java.io.*;
public class Writer { public static void main(String[] a) throws Exception {
    try (ObjectOutputStream o = new ObjectOutputStream(new FileOutputStream("fixed_v1.ser")))
{ o.writeObject(new ResidentFixed("Arun", "192311001")); }
    try (ObjectOutputStream o = new ObjectOutputStream(new FileOutputStream("implicit_v1.ser"))) {
o.writeObject(new ResidentImplicit("Arun", "192311001")); }
    System.out.println("version 1 (no phone field): wrote fixed_v1.ser and implicit_v1.ser");
    System.out.println("computed serialVersionUID of ResidentImplicit v1 = " +
java.io.ObjectStreamClass.lookup(ResidentImplicit.class).getSerialVersionUID());
} }
```

TaskA4/v2/ResidentFixed.java

```
import java.io.Serializable;
public class ResidentFixed implements Serializable {
    private static final long serialVersionUID = 1L;              // same number: "this is still
the same kind of object"
    public String name, regNo;
    public String phone;                                          // NEW FIELD
    public ResidentFixed(String n, String r) { name = n; regNo = r; }
}
```

TaskA4/v2/ResidentImplicit.java

```
import java.io.Serializable;
public class ResidentImplicit implements Serializable {
    public String name, regNo;
    public String phone;                                          // NEW FIELD (changes the
computed serialVersionUID)
    public ResidentImplicit(String n, String r) { name = n; regNo = r; }
}
```

TaskA4/v2/Reader.java

```
import java.io.*;
public class Reader { public static void main(String[] a) throws Exception {
    System.out.println("version 2 (class now has a phone field) reads the OLD files:");
    System.out.println("computed serialVersionUID of ResidentImplicit v2 = " +
java.io.ObjectStreamClass.lookup(ResidentImplicit.class).getSerialVersionUID());
    try (ObjectInputStream i = new ObjectInputStream(new FileInputStream("fixed_v1.ser")))
{ ResidentFixed r = (ResidentFixed) i.readObject();
        System.out.println("  explicit serialVersionUID = 1L : OK  -> name=" + r.name + ", regNo="
+ r.regNo + ", phone=" + r.phone + "   (new field gets its default value)"); }
    try (ObjectInputStream i = new ObjectInputStream(new FileInputStream("implicit_v1.ser")))
{ ResidentImplicit r = (ResidentImplicit) i.readObject(); System.out.println("  no
serialVersionUID           : OK  -> " + r.name); }
    catch (InvalidClassException e) { System.out.println("  no serialVersionUID           :
InvalidClassException -> " + e.getMessage()); }
} }
```

TaskA4/versioning.sh

```
#!/bin/sh
# Task A4.2: write with version 1 of the classes, read with version 2.
JC="java -m jdk.compiler/com.sun.tools.javac.Main"
rm -rf out_v1 out_v2; mkdir out_v1 out_v2
$JC -d out_v1 v1/*.java && $JC -d out_v2 v2/*.java || exit 1
java -cp out_v1 Writer
java -cp out_v2 Reader
rm -f fixed_v1.ser implicit_v1.ser
```

TaskA4/FormatBenchmark.java

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;

/** Alternative of A4: Java serialization  versus  CSV  versus  JSON text, for n residents. */
public class FormatBenchmark {
    static class Res implements Serializable {
        private static final long serialVersionUID = 1L;
        String name, regNo, room, phone;
        Res(String n, String r, String rm, String p) { name = n; regNo = r; room = rm; phone =
p; }
    }
    static long sink = 0;
    static List<Res> make(int n) { List<Res> l = new ArrayList<>(n); for (int i = 0; i < n; i++)
l.add(new Res("Resident Name " + i, String.format("19%07d", i), "A-" + (100 + i % 400),
String.format("98%08d", i))); return l; }

    static void saveSer(List<Res> l, Path f) throws IOException { try (ObjectOutputStream o = new
ObjectOutputStream(new BufferedOutputStream(Files.newOutputStream(f)))) { o.writeObject(l); } }
    @SuppressWarnings("unchecked") static List<Res> loadSer(Path f) throws Exception { try
(ObjectInputStream i = new ObjectInputStream(new BufferedInputStream(Files.newInputStream(f))))
{ return (List<Res>) i.readObject(); } }
    static String csvEsc(String s) { return (s.contains(",") || s.contains("\"")) ? "\"" +
s.replace("\"", "\"\"") + "\"" : s; }
    static void saveCsv(List<Res> l, Path f) throws IOException {
        try (BufferedWriter w = Files.newBufferedWriter(f)) { w.write("name,regNo,room,phone\n");
for (Res r : l) w.write(csvEsc(r.name) + "," + r.regNo + "," + r.room + "," + r.phone + "\n"); }
    }
    static List<Res> loadCsv(Path f) throws IOException {
        List<Res> l = new ArrayList<>();
        try (BufferedReader r = Files.newBufferedReader(f)) { r.readLine(); String s; while ((s =
r.readLine()) != null) { String[] p = s.split(",", -1); l.add(new Res(p[0], p[1], p[2],
p[3])); } }
        return l;
    }
    static String jsonEsc(String s) { return s.replace("\\", "\\\\").replace("\"", "\\\""); }
    static void saveJson(List<Res> l, Path f) throws IOException {
        try (BufferedWriter w = Files.newBufferedWriter(f)) {
            w.write("[\n");
            for (int i = 0; i < l.size(); i++) { Res r = l.get(i); w.write("{\"name\":\"" +
jsonEsc(r.name) + "\",\"regNo\":\"" + r.regNo + "\",\"room\":\"" + r.room + "\",\"phone\":\"" +
r.phone + "\"}" + (i + 1 < l.size() ? ",\n" : "\n")); }
            w.write("]\n");
        }
    }
    static String field(String line, String key) { int i = line.indexOf("\"" + key + "\":\""); if
(i < 0) return null; i += key.length() + 4; return line.substring(i, line.indexOf('"', i)); }
    static List<Res> loadJson(Path f) throws IOException {
        List<Res> l = new ArrayList<>();
        try (BufferedReader r = Files.newBufferedReader(f)) { String s; while ((s =
r.readLine()) != null) if (s.startsWith("{")) l.add(new Res(field(s, "name"), field(s, "regNo"),
field(s, "room"), field(s, "phone"))); }
        return l;
    }

    public static void main(String[] args) throws Exception {
        PrintWriter csv = new PrintWriter("format_results.csv");
        csv.println("n,format,file_bytes,save_ms,load_ms");
        System.out.println("=== Serialization vs CSV vs JSON for n residents (best of 5 runs)
===");
        System.out.printf("%9s %-14s %14s %12s %12s%n", "n", "format", "file bytes", "save (ms)",
"load (ms)");
```

```
        for (int w = 0; w < 3; w++) { List<Res> l = make(5000); Path f = Files.createTempFile("w",
".x"); saveSer(l, f); loadSer(f); saveCsv(l, f); loadCsv(f); saveJson(l, f); loadJson(f);
Files.delete(f); }
        for (int n : new int[]{1000, 10000, 100000}) {
            List<Res> data = make(n);
            for (String fmt : new String[]{"serialization", "CSV", "JSON"}) {
                Path f = Files.createTempFile("res", ".x");
                double save = 1e18, load = 1e18; int size = 0;
                for (int r = 0; r < 5; r++) {
                    long t = System.nanoTime();
                    if (fmt.equals("serialization")) saveSer(data, f); else if (fmt.equals("CSV"))
saveCsv(data, f); else saveJson(data, f);
                    save = Math.min(save, (System.nanoTime() - t) / 1e6);
                    size = (int) Files.size(f);
                    t = System.nanoTime();
                    List<Res> back = fmt.equals("serialization") ? loadSer(f) :
fmt.equals("CSV") ? loadCsv(f) : loadJson(f);
                    load = Math.min(load, (System.nanoTime() - t) / 1e6);
                    if (back.size() != n || !back.get(n - 1).name.equals(data.get(n - 1).name))
throw new IllegalStateException("round trip failed for " + fmt);
                }
                System.out.printf("%,9d %-14s %,14d %12.1f %12.1f%n", n, fmt, size, save, load);
                csv.printf("%d,%s,%d,%.2f,%.2f%n", n, fmt, size, save, load);
                Files.delete(f);
            }
        }
        csv.close();

        System.out.println();
        System.out.println("=== A NEW FIELD (building) is added after files were saved: old file
read by the new program ===");
        String oldCsv = "name,regNo,room,phone\nArun,192311001,A-101,9800000001\n";
        String[] header = oldCsv.split("\n")[0].split(","), row = oldCsv.split("\n")
[1].split(",");
        try { String building = row[4]; System.out.println("CSV, positional reader expecting 5
columns : " + building); }
        catch (ArrayIndexOutOfBoundsException e) { System.out.println("CSV, positional reader
expecting 5 columns : ArrayIndexOutOfBoundsException (" + e.getMessage() + ")"); }
        Map<String, String> byName = new HashMap<>(); for (int i = 0; i < header.length; i++)
byName.put(header[i], row[i]);
        System.out.println("CSV, header-aware reader (looks up 'building')      : " +
byName.getOrDefault("building", "(missing -> default)") + "  (works, if the reader is written that
way)");
        String oldJson = "{\"name\":\"Arun\",\"regNo\":\"192311001\",\"room\":\"A-
101\",\"phone\":\"9800000001\"}";
        System.out.println("JSON, reader looks up the key 'building'            : " +
field(oldJson, "building") + "  (missing key -> default; extra keys in the file are ignored)");
        System.out.println("Java serialization with fixed serialVersionUID      : new field =
default value; WITHOUT a fixed UID: InvalidClassException (see versioning output)");
        System.out.println("(checksum " + sink + ")");
    }
}
```

C.3 Task A5 – transfer

TaskA5/wallet.sql

```
-- Task A5 / B3: two accounts of one student; the sum of both balances must never change by a
transfer
DROP TABLE IF EXISTS account;
CREATE TABLE account (
    acc_id  VARCHAR(16)   PRIMARY KEY,
    balance DECIMAL(10,2) NOT NULL CHECK (balance >= 0)       -- an account can never go negative
);
INSERT INTO account(acc_id, balance) VALUES ('HOSTEL-192311001', 1000.00);
```

```
INSERT INTO account(acc_id, balance) VALUES ('CANTEEN-192311001', 50.00);
```

TaskA5/TransferDemo.java

```
import java.io.File;
import java.nio.file.*;
import java.sql.*;

/** Task A5: transfer money between two accounts: ONE transaction against two independent auto-
committed statements. */
public class TransferDemo {
    static final String FROM = "HOSTEL-192311001", TO = "CANTEEN-192311001";

    static Connection fresh(File f) throws Exception {
        minisqlite.MiniSqlite.install();
        Connection c = DriverManager.getConnection("jdbc:minisqlite:" + f.getAbsolutePath());
        try (Statement st = c.createStatement()) { for (String s :
Files.readString(Path.of("wallet.sql")).split(";\\s*\\n")) if (!s.isBlank() && !
s.trim().startsWith("--")) st.executeUpdate(s); }
        return c;
    }
    static double bal(Connection c, String id) throws SQLException { try (PreparedStatement ps =
c.prepareStatement("SELECT balance FROM account WHERE acc_id = ?")) { ps.setString(1, id); try
(ResultSet rs = ps.executeQuery()) { rs.next(); return rs.getDouble(1); } } }
    static String state(Connection c) throws SQLException { double a = bal(c, FROM), b = bal(c,
TO); return String.format("hostel=%.2f canteen=%.2f total=%.2f", a, b, a + b); }
    static void update(Connection c, String id, double delta) throws SQLException {
        try (PreparedStatement ps = c.prepareStatement("UPDATE account SET balance = balance + ?
WHERE acc_id = ?")) { ps.setDouble(1, delta); ps.setString(2, id); ps.executeUpdate(); }
    }

    /** One transaction: both updates or neither. crashAfterFirst simulates a failure between the
two statements. */
    static void transferTx(Connection c, double amt, boolean debitFirst, boolean crashAfterFirst)
throws SQLException {
        boolean old = c.getAutoCommit();
        c.setAutoCommit(false);
        try {
            update(c, debitFirst ? FROM : TO, debitFirst ? -amt : amt);
            if (crashAfterFirst) throw new IllegalStateException("simulated crash between the two
updates");
            update(c, debitFirst ? TO : FROM, debitFirst ? amt : -amt);
            c.commit();
        } catch (SQLException | RuntimeException e) { c.rollback(); throw e; }
        finally { c.setAutoCommit(old); }
    }
    /** Two independent statements, each committed by itself (auto-commit ON). */
    static void transferAuto(Connection c, double amt, boolean debitFirst, boolean
crashAfterFirst) throws SQLException {
        c.setAutoCommit(true);
        update(c, debitFirst ? FROM : TO, debitFirst ? -amt : amt);
        if (crashAfterFirst) throw new IllegalStateException("simulated crash between the two
updates");
        update(c, debitFirst ? TO : FROM, debitFirst ? amt : -amt);
    }
    interface Op { void run(Connection c) throws Exception; }
    static void scenario(String title, boolean tx, Op op) throws Exception {
        File f = File.createTempFile("a5wallet", ".db"); f.delete();
        try (Connection c = fresh(f)) {
            String before = state(c), err = "none";
            try { op.run(c); } catch (Exception e) { err = e.getClass().getSimpleName() +
(e.getMessage() == null ? "" : ": " + e.getMessage().replaceAll("\\s+\\[.*", "")); }
            System.out.printf("%-58s%n    before: %s%n    error : %s%n    after : %s%n", title,
before, err, state(c));
        } finally { f.delete(); }
    }
```

```
    public static void main(String[] args) throws Exception {
        System.out.println("=== A5.3 Transfer Rs.300 from the hostel wallet to the canteen wallet
===");
        scenario("1. transaction, no failure", true, c -> transferTx(c, 300, true, false));
        scenario("2. TRANSACTION, crash between the two updates", true, c -> transferTx(c, 300,
true, true));
        scenario("3. AUTO-COMMIT, crash between the two updates (debit first)", false, c ->
transferAuto(c, 300, true, true));
        scenario("4. AUTO-COMMIT, credit first, then the debit fails (balance too low: 2,000)",
false, c -> transferAuto(c, 2000, false, false));
        scenario("5. TRANSACTION, credit first, then the debit fails (balance too low: 2,000)",
true, c -> transferTx(c, 2000, false, false));

        System.out.println();
        System.out.println("=== Cost: 2,000 transfers of Rs.1 (database on disk, every commit is
made durable) ===");
        for (int w = 0; w < 2; w++) timeRun(200, true);
        double tx = timeRun(2000, true), auto = timeRun(2000, false);
        System.out.printf("one transaction per transfer (1 commit)     : %.1f ms in total, %.0f us
per transfer%n", tx, tx * 1000 / 2000);
        System.out.printf("two auto-committed updates (2 commits)      : %.1f ms in total, %.0f us
per transfer%n", auto, auto * 1000 / 2000);
        System.out.printf("ratio auto-commit / transaction             : %.2f%n", auto / tx);
    }
    static double timeRun(int n, boolean tx) throws Exception {
        File f = File.createTempFile("a5time", ".db"); f.delete();
        try (Connection c = fresh(f)) {
            update(c, FROM, 100000);
            long t = System.nanoTime();
            for (int i = 0; i < n; i++) { if (tx) transferTx(c, 1, true, false); else
transferAuto(c, 1, true, false); }
            return (System.nanoTime() - t) / 1e6;
        } finally { f.delete(); }
    }
}
```

ER diagrams - Task A5 (wallet transfer) and Task B2 (login)

USERS (test database only)	
PK	user_no INT
UK	id VARCHAR(20)
	pwd VARCHAR(40)
	role VARCHAR(10)

ACCOUNT	
PK	acc_id VARCHAR(16)
	balance DECIMAL(10,2) CHECK (balance >= 0)

Figure C.1 – ER diagrams of the tables used in A5 (account) and B2 (users)

C.4 Viva preparation – short answers

Question	Answer
Why is reading a file with read() one byte at a time slow?	Every call is a system call. 50 MB = 52,428,784 calls (measured, 16 s). A buffer (8 KB or 64 KB) divides the number of system calls by the buffer size: 6,401 and 801 calls.
When does BufferedInputStream not help?	When the program already reads blocks at least as large as the buffer, when the data is already in memory, and for random access.
What does a transaction guarantee?	Atomicity: all statements take effect together (commit) or none (rollback). Measured: a failure at row 500 leaves 0 rows with a transaction and 500 rows without.

Question	Answer
Why is auto-commit slow for bulk loads?	Each statement is a transaction with a durable commit (about 0.8 ms on this disk); 100,000 rows = 80 s; one transaction = 0.3 s.
Why is PreparedStatement safe?	The SQL structure is parsed before the data exists and the data are sent as values, so input can never become SQL. Verified with three attacks.
Why is escaping quotes not enough?	It fails in numeric contexts (no quotes), depends on the dialect (MySQL backslash) and must be remembered for every value; regex validation blocks honest input such as it's-me.
What is serialVersionUID for?	Compatibility check between the class and the stream. Declared explicitly, adding a field is compatible (phone = null); computed automatically, it changes and the old file gives InvalidClassException.
What is transient?	A field that serialization skips (password, Connection); after reading it is null/0 and must be recreated.
What is a DAO and why use it?	A class that owns all database access of one entity. The repeated JDBC steps are written once in a template class; a column change touches one class instead of four.
Which design for 12 campuses?	A shared relational database with indexes: 0.025 ms per lookup against 781 ms in the text file, unique numbers from AUTO_INCREMENT, transactions, no heap proportional to the data.

C.5 Final checklist before submission

- Replace every <placeholder> (name, register number, college, GitHub link, date).
- Run every program on your own computer; run the JDBC programs with MySQL (Appendix A) and add a screenshot of the MySQL tables; retake the screenshots from your own terminal.
- Check that the pseudocode keywords match the official SIMATS pseudocode standard.
- Create the GitHub repository CSA09-<RegisterNumber>-Assignment5 and commit task by task (at least one commit per task).